

2024 / 2025

Introduction au Langage de Programmation Python



M. Marc TEJIOGNI

Email : mtejiogni@yahoo.fr

Tél : 693 90 91 21 / 682 66 37 31

PLAN DU COURS

OBJECTIF GENERAL	6
OBJECTIFS SPECIFIQUES	6
CHAPITRE I : INTRODUCTION A PYTHON ET A L'INGENIERIE	7
Objectif du chapitre.....	7
Objectifs spécifiques.....	7
I. Présentation de Python	7
I.1. Qu'est-ce que Python ?	7
I.2. Que fait Python ?	8
I.3. Pourquoi Python pour l'ingénierie ?	9
II. Installation de l'environnement.....	11
III. L'interpréteur python	14
III.1. Accéder à l'interpréteur python	14
III.2. Utilisation de l'interpréteur python.....	14
Tests de connaissances.....	15
CHAPITRE II : GEREZ VOS DONNEES AVEC LES VARIABLES	17
Objectif du chapitre.....	17
Objectifs spécifiques.....	17
I. Enregistrez vos données avec des variables	17
I.1 Qu'est ce qu'une variable ?	17
I.2 Afficher une ou plusieurs variables	18
I.3. Nommez une variable	19
II. Classez vos données avec les types de données	19
II.1. Qu'est ce qu'un type de données ?	19
II.2. Les types de données numériques : entiers et virgules flottantes.....	20
II.3. Les chaînes de caractères	20
II.4. Les booléens	21
II.5. La fonction « type() »	21
III. Enregistrez des groupes de données avec des listes.....	22
III.1. Qu'est ce qu'une liste ?	22
III.2. Accédez aux éléments d'une liste	22
III.2. Accédez aux caractères d'une chaîne.....	23
III.3. Modifiez les éléments d'une liste.....	23
III.4. Ajoutez, retirez et triez les listes	24
III.5. Les tuples	25

III.6. Trouvez un élément.....	25
IV. Enregistrez des données complexes avec des dictionnaires	26
IV.1. Qu'est ce qu'un dictionnaire ?	26
IV.2. Créez un dictionnaire	26
IV.3. Accédez à une valeur d'un dictionnaire.....	27
IV.4. Réalisez des opérations avec les dictionnaires	27
Tests de connaissances.....	28
CHAPITRE III : GEREZ LA LOGIQUE DE VOS PROGRAMMES AVEC PYTHON	30
Objectif du chapitre.....	30
Objectifs spécifiques.....	30
I. Contrôlez le déroulement du programme avec des conditions.....	30
I.1. Définition.....	30
I.2. Les conditions avec les instructions if/else.....	31
I.3. Les conditions alternatives avec la clause elif.....	31
I.4. Les conditions multiples avec des opérateurs logiques	32
I.5. Les conditions complexes avec des expressions comparatives	32
I.5. Simplifiez le code avec les matchs cases.....	33
II. Répétez des tâches facilement à l'aide de boucles	34
II.1. Définition.....	34
II.2. Utilisez la boucle for	34
II.3. Utilisez la boucle while	35
II.4. Utilisez les mot-clés break et continue.....	36
III. Regroupez des tâches en utilisant des fonctions	37
III.1. Qu'est ce qu'une fonction ?	37
III.2. Utilisez une fonction sans paramètres.....	38
III.3. Utilisez une fonction avec paramètres	38
III.4. Utilisez une fonction avec une valeur de retour.....	39
IV. Écrivez du code en évitant les erreurs courantes	40
IV.1. Pourquoi écrire du code de façon structurée ?	40
IV.2. Comment écrire du code de façon structurée.....	40
CHAPITRE IV : CALCUL NUMERIQUE ET VISUALISATION STATIQUE DES DONNEES	44
Objectif du chapitre.....	44
Objectifs spécifiques.....	44
I. Introduction à NumPy.....	44
I.1. Présentation de la bibliothèque.....	44
I.2. Différenciez les arrays des listes classiques	45
I.2. Créez un array NumPy.....	46

I.3. Sélectionnez des éléments au sein d'un array.....	47
I.4. Utilisez les méthodes d'array.....	49
I.5. Créez un tableau avec NumPy	50
I.6. Découvrez l'analogie entre les arrays et les matrices	51
II. Introduction à SciPy	52
II.1. Présentation de la bibliothèque.....	52
II.2. Algèbre linéaire avec SciPy	53
II.3. Equations différentielles avec SciPy	54
III. Construisez des data visualisations avec Matplotlib.....	56
III.1. Comprenez l'importance de la représentation graphique	56
III.2. Présentation de la bibliothèque	57
III.3. Tracez vos premiers graphiques.....	58
III.3.1. Le nuage de points	58
III.3.2. Le diagramme circulaire	59
III.3.3. Le diagramme à barres	61
III.3.4. L'histogramme	62
III.3.5. Les courbes	63
III.3.6. Créez plusieurs graphiques sur une même fenêtre.....	65
CHAPITRE V : ANALYSE ET VISUALISATION DYNAMIQUES DES DONNEES.....	67
Objectif du chapitre.....	67
Objectifs spécifiques.....	67
I. Analyse des données avec Pandas	67
I.1. Présentation de la bibliothèque.....	67
I.2. Créez votre premier data frame avec Pandas.....	69
I.2.1. Préparez votre jeu de données	69
I.2.2. Générez votre premier data frame depuis des jeux de données au format .csv, .json, .xlsx.....	71
I.3. Manipuler le data frame	72
I.3.1. Naviguez dans le data frame	72
I.3.2. Découvrez l'objet Series de Pandas.....	73
I.3.3. Manipulez les colonnes	73
I.4. Filtrez les données du data frame	77
I.4.1. Appliquez la sélection via des indices	77
I.4.2. Utilisez la sélection via des conditions.....	78
I.4.3. Modifiez une sélection.....	80
I.5. Agrégez des données avec Pandas.....	81
I.5.1. Créez un jeu de données fictif	82

I.5.2. Agrégez plusieurs lignes.....	83
I.5.2. Agrégez des lignes et des colonnes	85
I.6. Fusionnez des données avec Pandas.....	87
I.6.1. Comprenez l'intérêt de la fusion de données.....	87
I.6.1. Fusionnez des données avec merge	88
I.6.2. Concaténez des données avec Pandas.....	90
II. Introduction à Plotly.....	91
II.1. Présentation de la bibliothèque.....	91
II.2. Implémentez un graphique avec Plotly	91
III. Introduction à Mayavi.....	96
II.1. Présentation de la bibliothèque.....	96
II.2. Plotly vs Mayavi.....	97
II.3. Cas pratique.....	98

OBJECTIF GENERAL

Bienvenue dans le cours d'introduction au langage de programmation de Python. Les premières étapes de tout apprentissage sont généralement les plus difficiles. Toutes nos félicitations pour vous être lancé ! Ce cours initie les apprenants à Python comme langage polyvalent pour résoudre des problèmes complexes en ingénierie. À travers des exemples concrets tirés des mathématiques (calcul différentiel, intégrales), de la physique (mécanique, thermodynamique) et de la chimie (réactions, équilibres), les apprenants apprendront à coder, analyser des données et simuler des systèmes. Le cours comprend des laboratoires pratiques, des projets d'équipe et des devoirs pour renforcer les compétences techniques et appliquées.

OBJECTIFS SPECIFIQUES

Au terme de ce cours l'apprenant sera capable de :

- Comprendre les bases de Python et son rôle en ingénierie
- Maîtriser les structures conditionnelles et boucles pour résoudre des problèmes logiques
- Apprendre à créer des fonctions pour modéliser des problèmes mathématiques
- Gérer des données complexes pour des applications en ingénierie
- Utiliser Python pour les calculs numériques et la visualisation
- Appliquer Python à des problèmes de physique et d'ingénierie complexes
- Utiliser Python pour analyser des données expérimentales
- Explorer des outils avancés pour l'ingénierie (Applications avancées et visualisation 3D)

CHAPITRE I : INTRODUCTION A PYTHON ET A L'INGENIERIE

Objectif du chapitre

L'objectif de ce chapitre est de fournir aux apprenants une compréhension approfondie des fondamentaux de Python et de ses applications dans le domaine de l'ingénierie, en les accompagnant dans l'installation et la configuration de leur environnement de développement.

Objectifs spécifiques

- Présentation de Python : Pourquoi Python pour l'ingénierie ?
- Installation de l'environnement (Anaconda, Jupyter Notebook, VS Code)
- Utilisation de l'interpréteur Python

I. Présentation de Python

I.1. Qu'est-ce que Python ?

Python est un langage de programmation inventé par **Guido van Rossum**. La première version de Python est sortie en **1991**.

Python est un langage de programmation **interprété**, c'est-à-dire qu'il n'est pas nécessaire de le compiler avant de l'exécuter. Contrairement aux langages compilés comme le C, qui nécessitent une étape de compilation (transformation du code source en code machine avant l'exécution), un langage interprété exécute le code directement ligne par ligne. Cette caractéristique permet aux développeurs de tester et de modifier leur code rapidement, sans avoir à passer par une étape de compilation.

Si vous avez déjà touché un peu à la programmation, vous verrez que le langage Python possède une certaine **poésie** : Cette affirmation souligne la beauté et l'élégance du code Python. De nombreux programmeurs trouvent que Python permet d'écrire du code qui est non seulement fonctionnel, mais aussi esthétique. Cela fait référence au principe de lisibilité qui est au cœur de la conception de Python, où les instructions sont souvent faciles à lire et à comprendre.

I.2. Que fait Python ?

Python est à la fois simple et puissant, il vous permet d'écrire des scripts très simples et grâce à ses nombreuses bibliothèques, vous pouvez travailler sur des projets plus ambitieux. **Python** est un langage de programmation polyvalent, ce qui signifie qu'il peut être utilisé pour développer une large gamme d'applications dans divers domaines. Voici quelques types d'applications courantes que vous pouvez coder avec **Python** :

- **Applications Web** : Python est couramment utilisé pour créer des applications web dynamiques. Les frameworks tels que **Django** et **Flask** facilitent le développement de logiciels web complexes.

Technologies nécessaires :

- Frameworks : Django, Flask, FastAPI.
- Base de données : SQLite, PostgreSQL, MySQL.
- Front-end : HTML, CSS, JavaScript (pour intégrer avec le back-end Python).
- Serveurs : Apache, Nginx

- **Développement de Logiciels de Bureau** : Python peut être utilisé pour développer des logiciels de bureau avec des interfaces graphiques (GUI) grâce à des bibliothèques comme **Tkinter**, **PyQt** ou **wxPython**.

Technologies nécessaires :

- GUI Libraries : Tkinter, PyQt, Kivy, wxPython.
- Distribution : PyInstaller, cx_Freeze (pour empaqueter l'application).

- **Applications de Science des Données et Analyse** : Python est le langage de choix pour les scientifiques et analystes en raison de son écosystème riche en bibliothèques dédiées à l'analyse de données et à la science des données.

Technologies nécessaires :

- Bibliothèques : NumPy, pandas, Matplotlib, Seaborn, SciPy.
- Environnements : Jupyter Notebook, Anaconda (pour la gestion des packages et des environnements).

- **Apprentissage Automatique et Intelligence Artificielle** : Python est largement utilisé dans le domaine de l'apprentissage automatique et de l'intelligence artificielle grâce à ses bibliothèques robustes et flexibles.

Technologies nécessaires :

- Bibliothèques : TensorFlow, Keras, PyTorch, scikit-learn, XGBoost.
- Outils de visualisation : Matplotlib, Plotly.
- Environnements : Jupyter Notebook, Google Colab (pour le calcul dans le cloud).

- **Automatisation et Scripting** : Python est souvent utilisé pour automatiser des tâches répétitives, simple à écrire et à lire, ce qui en fait un excellent choix pour les scripteurs.

Technologies nécessaires :

- Scripting : Scripts Python de base, modules Python standards comme os, subprocess, shutil.
- Outils d'automatisation : Selenium (pour l'automatisation de navigateurs), BeautifulSoup (pour le scraping web).

- **Développement de Jeux** : Bien que moins courant que d'autres langages comme C++, Python peut être utilisé pour créer des jeux grâce à des bibliothèques comme Pygame.

Technologies nécessaires :

- Bibliothèques : Pygame (pour le développement de jeux 2D).
- Moteurs de jeu : Godot (qui prend également en charge le langage Python via GDScript).

- **Internet des Objets (IoT)** : Python peut être utilisé pour développer des applications pour des périphériques IoT, notamment ceux utilisant Raspberry Pi.

Technologies nécessaires :

- Plateformes IoT : Raspberry Pi avec des bibliothèques comme GPIO, MicroPython pour des microcontrôleurs.
- Protocole de communication : MQTT, HTTP.

- **Applications Blockchain et Cryptographie** : Python peut être utilisé pour développer des applications sur la blockchain et pour des systèmes de cryptographie. La **Blockchain** (ou chaîne de blocs en français) est une technologie de stockage et de transmission d'informations qui se caractérise par plusieurs éléments fondamentaux. Elle est généralement utilisée dans les domaines tels que : Cryptomonnaie, Vote électronique, Traçabilité des chaînes d'approvisionnement, Gestion des identités

Technologies nécessaires :

- Bibliothèques : Web3.py (pour interagir avec la blockchain Ethereum), PyCryptodome pour la cryptographie.
- Frameworks de Blockchain : Django, Flask (pour les applications basées sur blockchain).

- **Tests Automatisés** : Python est largement utilisé pour écrire des tests automatisés pour les applications logicielles, permettant d'assurer la qualité.

Technologies nécessaires :

- Bibliothèques de test : unittest, pytest, Selenium (pour les tests d'interfaces).

I.3. Pourquoi Python pour l'ingénierie ?

Python joue un rôle essentiel dans le domaine de l'ingénierie grâce à sa polyvalence. Il y a plusieurs raisons convaincantes pour lesquelles Python est devenu un choix privilégié pour l'ingénierie. Voici quelques-unes des principales motivations :

Simplicité et Lisibilité

- **Syntaxe claire** : Python a une syntaxe simple et intuitive qui facilite l'écriture et la compréhension du code. Cela permet aux ingénieurs, qu'ils soient expérimentés ou novices, de se concentrer sur la résolution de problèmes sans être submergés par des complexités de syntaxe.
- **Lisibilité** : Le code Python est souvent décrit comme "lisible comme de l'anglais". Cette caractéristique favorise le travail collaboratif et la maintenance du code, car d'autres personnes peuvent facilement comprendre et adapter le code existant.

Polyvalence

Large éventail d'applications : Python peut être utilisé dans divers domaines d'ingénierie, y compris le développement d'applications web, l'analyse de données, la science des données, l'automatisation, le prototypage IoT, et bien plus encore. Sa polyvalence en fait un outil intemporel pour les ingénieurs.

Richesse des Bibliothèques et Frameworks

Python dispose d'une vaste bibliothèque de modules et de packages qui simplifient le développement d'applications. Ces outils permettent aux ingénieurs de travailler plus efficacement, en évitant de réinventer la roue.

Support de l'Intelligence Artificielle et de l'Apprentissage Automatique

Python est souvent considéré comme le langage de choix pour l'intelligence artificielle (IA) et l'apprentissage automatique (ML). Grâce à des bibliothèques telles que TensorFlow, Keras et scikit-learn, les ingénieurs peuvent développer des modèles prédictifs et des systèmes intelligents avec facilité.

Développement Rapide et Prototypage

La capacité de Python à permettre un développement rapide est particulièrement bénéfique pour les ingénieurs. Ils peuvent tester des idées, créer des prototypes et apporter des modifications rapidement, ce qui est crucial dans les environnements de travail agiles.

Communauté Active

- **Large communauté** : Python possède une communauté mondiale active qui contribue à son développement et à ses extensions. Cela signifie que les ingénieurs peuvent obtenir facilement de l'aide, des ressources et des conseils, et que de nombreuses ressources d'apprentissage sont disponibles.
- **Support et documentation** : La documentation de Python et des bibliothèques est généralement bien structurée et facile à suivre, ce qui aide les utilisateurs à apprendre et à résoudre des problèmes.

Intégration avec d'autres Technologies

- **Interopérabilité** : Python peut facilement s'intégrer avec d'autres langages de programmation et technologies, ce qui permet aux ingénieurs de l'utiliser comme langage de glissement entre différents systèmes.
- **API et microservices** : Python peut également être utilisé pour construire des API et des microservices, facilitant ainsi l'interconnexion entre différentes applications et systèmes.

Applicabilité dans des Domaines Spécifiques

- **Ingénierie financière** : Utilisé pour le calcul des risques, l'analyse des investissements et la modélisation financière.
- **Ingénierie logicielle** : Python est souvent utilisé dans le développement de logiciels, les tests automatisés, et la gestion de l'infrastructure.

- **Ingénierie électrique et mécanique** : Pour la simulation, la modélisation de circuits, et l'analyse de la dynamique des systèmes.

II. Installation de l'environnement

Il existe plusieurs environnements dans lesquels vous pouvez coder en Python, chacun ayant ses propres caractéristiques et avantages. Voici un aperçu des environnements les plus courants pour le développement en Python :

Environnements de Développement Intégré (IDE)

Les IDE offrent des outils complets pour le développement, y compris un éditeur de code, des fonctionnalités de débogage, et souvent un terminal intégré.

- **PyCharm** : IDE populaire pour le développement Python. Offre une multitude de fonctionnalités telles que l'autocomplétion, le refactoring de code, et le débogage intégré. Il est disponible en version Community (gratuite) et Professional (payante).
- **Visual Studio Code (VS Code)** : Éditeur de code léger et extensible avec un support Python via des extensions. Il dispose de nombreuses fonctionnalités comme le débogage intégré, la coloration syntaxique, et les snippets de code. Il est très personnalisable avec des extensions pour répondre à différents besoins.
- **Spyder** : IDE conçu principalement pour les scientifiques et les ingénieurs. Il inclut des fonctionnalités de visualisation de données et d'analyse de données. Intégré dans la distribution Anaconda.

Environnements de Codage en Ligne

Les environnements de codage en ligne permettent d'écrire et d'exécuter du code Python directement dans un navigateur sans avoir besoin d'installer quoi que ce soit.

- **Google Colab** : Outil basé sur Jupyter Notebook, populaire pour le machine learning et l'analyse de données. Il permet d'exécuter du code Python dans le cloud, avec accès à des GPU gratuitement. Il est idéal pour partager des notebooks et collaborer en temps réel.
- **Repl.it** : Environnement de codage en ligne qui prend en charge de nombreux langages, dont Python. Il permet de coder, exécuter et partager des projets en ligne facilement, idéal pour les petites applications et les sprints de codage.
- **Jupyter Notebook** : Application web basée sur un navigateur pour créer et partager des documents contenant du code exécuté, des visualisations et du texte en langage Markdown. Il est très utilisé dans le domaine de la science des données et de l'enseignement.

Éditeurs de Texte

Pour les développeurs qui préfèrent un environnement plus léger, les éditeurs de texte permettent de coder rapidement sans les fonctionnalités complètes des IDE.

- **Sublime Text** : Éditeur de texte rapide et léger, avec support pour la coloration syntaxique de Python. Il est personnalisable avec des plugins et des thèmes.
- **Atom** : Éditeur de texte open-source développé par GitHub. Il est extensible avec des packages pour ajouter des fonctionnalités, y compris le support Python.
- **Notepad++** : Simplicité et légèreté, idéal pour ceux qui souhaitent écrire du code sans fonctionnalités avancées.

Distribution Anaconda

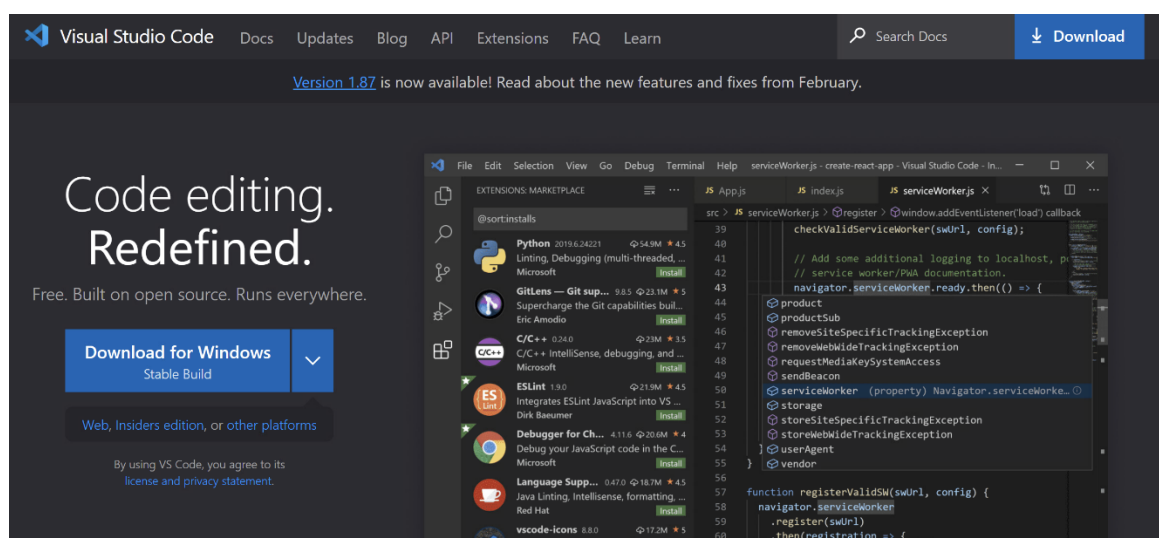
Anaconda est une distribution Python spécialement conçue pour le traitement de données et l'apprentissage automatique. Il inclut un gestionnaire de paquets (conda) et un environnement virtuel pour gérer les dépendances de projet.

Il comprend **Jupyter Notebook**, **Spyder** et de nombreuses bibliothèques populaires préinstallées (comme NumPy, pandas, et Matplotlib).

Terminal et Environnement Virtuel

- **Terminal/Command Line** : Vous pouvez exécuter Python directement à partir de la ligne de commande ou du terminal en utilisant l'interpréteur Python. Pratique pour des scripts rapides ou des administrations système.
- **Environnements Virtuels** : Les outils comme **venv** ou **virtualenv** vous permettent de créer des environnements isolés pour chaque projet Python, garantissant que les dépendances spécifiques d'un projet ne perturbent pas les autres.

Le choix de l'environnement pour coder en Python dépend principalement de vos besoins, de votre degré de confort et de la nature du projet. Dans le cadre de cours nous utiliserons **Visual Studio Code (VS Code)** car c'est l'un des environnements de développement les plus populaires pour coder en Python et il est également gratuit. Vous pouvez le télécharger sur le site officiel de Visual Studio Code à l'adresse suivante : <https://code.visualstudio.com/>



Attention : Pour utiliser l'extension **Python** dans **Visual Studio Code (VS Code)**, il est nécessaire d'avoir Python installé sur votre machine. L'extension **Python** dans **VS Code** sert de lien entre l'éditeur et l'**interpréteur Python** installé sur votre système. Sans Python, VS Code ne peut pas exécuter de scripts Python.

Étapes pour Installer Python et Configurer VS Code

1) Télécharger et Installer Python :

- Visitez le site officiel de Python: <https://www.python.org/downloads/>
- Téléchargez la dernière version stable pour votre système d'exploitation (Windows, macOS, Linux).
- Pendant l'installation, assurez-vous de cocher l'option « **Add Python to PATH** » pour faciliter l'accès à l'interpréteur Python à partir de la ligne de commande.

2) Vérifier l'Installation de Python :

- Ouvrez le terminal (ou l'invite de commandes sur Windows) et exécutez la commande : **python --version**
- Cela devrait afficher la version de Python installée

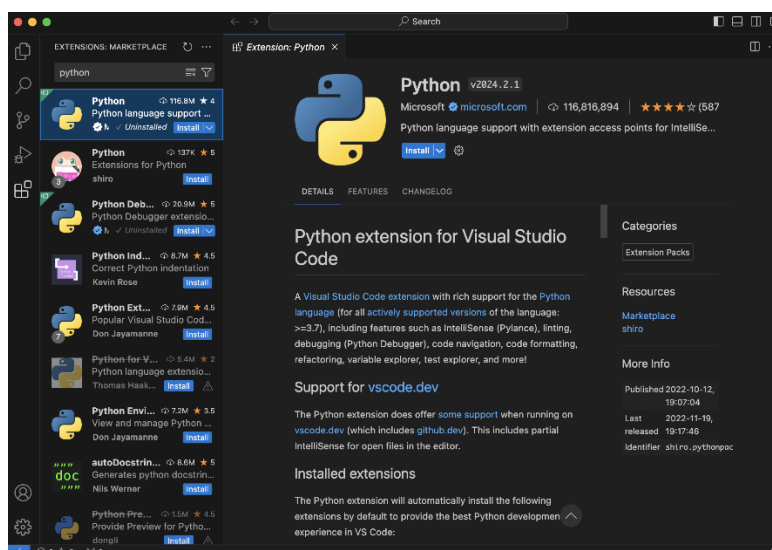
3) Installer Visual Studio Code : Si ce n'est pas déjà fait, téléchargez et installez VS Code depuis le site officiel.

4) Installer l'Extension Python dans VS Code :

- Ouvrez VS Code
- Accédez à l'onglet des extensions (ou appuyez sur **Ctrl+Shift+X**)
- Recherchez « **Python** » et installez l'extension officielle proposée par Microsoft.

5) Configurer l'Interpréteur Python dans VS Code :

- Ouvrez un fichier Python ou créez-en un nouveau.
- En bas à gauche de VS Code, cliquez sur la version de Python indiquée (ou le message « **Sélectionner l'interpréteur** ») pour choisir l'interpréteur que vous venez d'installer.
- Sélectionnez l'interpréteur Python correspondant dans la liste.



III. L'interpréteur python

III.1. Accéder à l'interpréteur python

L'interpréteur Python est un outil essentiel pour travailler avec le langage de programmation Python. Il vous permet d'exécuter directement des instructions Python, d'expérimenter avec des petits morceaux de code et de tester des fonctions, ce qui est particulièrement utile pour l'apprentissage et le développement.

L'interpréteur Python exécute le code Python ligne par ligne, ce qui signifie que vous pouvez entrer des commandes et voir les résultats immédiatement.

Il existe plusieurs façons d'accéder à l'interpréteur Python :

- 1) **Ligne de commande (Terminal ou Invite de commande) :**
 - a. Ouvrez votre terminal ou invite de commande.
 - b. Tapez python ou python3 (selon votre installation) et appuyez sur Entrée.
 - c. Vous verrez une invite interactive (constitué d'un « >>> ») indiquant que l'interpréteur est prêt à recevoir des commandes
- 2) **IDLE :**
 - a. **IDLE** est l'environnement de développement intégré (IDE) qui est inclus avec l'installation standard de Python. Vous pouvez l'ouvrir à partir de votre menu d'applications.
 - b. Une fois ouvert, il vous positionne automatiquement dans un interpréteur Python.
- 3) **Environnements de développement (comme PyCharm, VS Code, Jupyter Notebook) :**
 - a. Ces environnements proposent des interpréteurs intégrés où vous pouvez exécuter du code Python.

III.2. Utilisation de l'interpréteur python

Quelques conseils pour utiliser l'interpréteur :

1. **Entrée de code :**
 - Vous pouvez écrire du code Python ligne par ligne. Par exemple, vous pouvez écrire :
- ```
>>> x = 5
>>> y = 10
>>> z = x + y
>>> print(z)
15
```

## 2. Évaluation immédiate :

- Vous pouvez évaluer des expressions immédiatement :

```
>>> 3 + 4
7
```

## 3. Utilisation des fonctions intégrées :

- Vous pouvez appeler n'importe quelle fonction intégrée

```
>>> len("Bonjour")
7
```

## 4. Exécution de plusieurs lignes de code :

- Pour entrer des blocs de code (comme les définitions de fonctions ou les boucles), vous devez utiliser des indentations appropriées :

```
>>> def ma_fonction():
... print("Bonjour, monde!")
...
>>> ma_fonction()
Bonjour, monde!
```

## 5. Aide :

- Vous pouvez utiliser la fonction **help()** pour obtenir de l'aide sur les fonctions et les modules :

```
>>> help(str)
```

## 6. Quitter l'interpréteur :

- Pour quitter l'interpréteur, vous pouvez utiliser **exit()** ou **quit()**, ou simplement appuyer sur **Ctrl + Z** (Windows) ou **Ctrl + D** (Linux/Mac).

L'interpréteur Python est un excellent moyen d'expérimenter avec le code Python et d'apprendre rapidement. Il est simple d'accès et vous permet de tester immédiatement vos idées.

# Tests de connaissances

## Questions à choix multiples

- Pourquoi Python est-il souvent choisi pour l'ingénierie ?
  - Car il est le plus rapide des langages de programmation.
  - En raison de sa simplicité et de sa lisibilité.
  - Parce qu'il n'a pas besoin d'être installé.
  - Pour sa capacité à créer des jeux vidéo.
- Quel est l'un des principaux avantages d'utiliser Anaconda ?
  - C'est un langage de programmation.

- b. Il facilite la gestion des packages et des environnements.
  - c. Il n'a pas besoin d'installation.
  - d. C'est un système d'exploitation.
- 3. Quel outil permet d'exécuter du code Python dans un document interactif avec des graphiques ?
  - a. Sublime Text
  - b. Jupyter Notebook
  - c. Terminal
  - d. VS Code
- 4. Quelle commande est utilisée pour démarrer l'interpréteur Python dans le terminal ?
  - a. start python
  - b. python
  - c. run python
  - d. open python
- 5. Quelle est l'extension de fichier par défaut pour les notebooks Jupyter ?
  - a. .py
  - b. .ipynb
  - c. .txt
  - d. .jupyter

### Questions ouvertes

1. Expliquez pourquoi Python est particulièrement adapté pour les applications d'ingénierie.
2. Décrivez les étapes de l'installation de l'environnement de développement Python sur votre système d'exploitation.
3. Quelles sont les principales différences entre Jupyter Notebook et un éditeur de code comme VS Code ?



# CHAPITRE II : GEREZ VOS DONNEES AVEC LES VARIABLES

## Objectif du chapitre

Acquérir une compréhension fondamentale des concepts de base en Python, notamment les variables, les types de données, les listes et les dictionnaires, afin de pouvoir manipuler et structurer des données efficacement dans des programmes.

## Objectifs spécifiques

Comprendre et utiliser les concepts fondamentaux de Python :

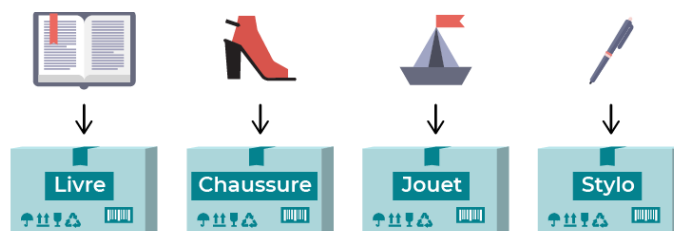
- Comprendre et utiliser les variables ;
- Identifier et manipuler les types de données ;
- Utiliser la fonction « type() » pour identifier les types de données ;
- Utiliser les listes pour stocker des groupes de données ;
- Comprendre les tuples et leur utilisation pour stocker des données ;
- Travailler avec des dictionnaires pour des données complexes.

## I. Enregistrez vos données avec des variables

### I.1 Qu'est ce qu'une variable ?

Imaginons qu'on s'apprête à déménager et qu'il faille ranger nos affaires dans des cartons. Il nous faut un carton pour la vaisselle, un pour les vêtements et un autre pour la télévision. Quand vous faites les cartons, la bonne chose à faire est de leur donner un nom. Ainsi vous saurez ce qui se trouve à l'intérieur.

Une **variable**, c'est comme un carton. C'est un moyen d'enregistrer vos données. En tant que développeur, il vous faudra enregistrer certaines informations et leur donner un nom afin de pouvoir les récupérer plus tard. Un nom d'utilisateur, des tickets d'avion encore disponibles, le jour de la semaine, un inventaire... Toutes ces données sont enregistrées dans les variables.



Il est très facile de déclarer une variable dans Python. Le code ci-dessous permet de déclarer une variable.

```
livre = "Gatsby le Magnifique"
```

**NB** : Quand on dit déclarer (ou initialiser) une variable, c'est juste une autre façon de dire « créer une variable ».

Une variable dans Python est constituée de trois éléments :

- **Son nom**
- **Son type**
- **Sa valeur**

Ici nous allons voir le nom et la valeur, puis nous verrons le type dans la suite.

Ici, **livre** est le nom de la variable (donc le nom du carton) et "**Gatsby le Magnifique**" est la valeur de la variable (ce qui se trouve dans le carton). Si vous souhaitez retrouver le titre du livre avec Python, vous pouvez écrire **livre** et vous obtiendrez "**Gatsby le Magnifique**" en retour.

Il est très simple de modifier la valeur d'une variable. Il suffit simplement d'assigner une nouvelle valeur à la variable.

Voici l'extrait de code qui déclare **livre** avec la valeur "**Gatsby le Magnifique**" puis qui la modifie avec "**Beloved**"

```
>>> livre = "Gatsby le Magnifique"
>>> print(livre)
Gatsby le Magnifique
>>> livre = "Beloved"
>>> print(livre)
Beloved
```

## 1.2 Afficher une ou plusieurs variables

Pour afficher une ou plusieurs variables en Python, vous pouvez utiliser la fonction intégrée **print()**. Comme suit :

```
Déclaration de plusieurs variables
nom = "Alice"
age = 25
prix = 19.99

Affichage de plusieurs variables
print(nom, age, prix)
```

## Formatage de la sortie

Pour un affichage plus structuré, vous pouvez utiliser les **f-strings** pour formater votre sortie, disponible depuis **Python 3.6** :

```
Déclaration des variables
nom = "Alice"
age = 25

Affichage formaté avec f-string
print(f"{nom} a {age} ans.")
```

## I.3. Nommez une variable

Un nom de variable doit refléter son contenu. Voici quelques recommandations générales pour choisir un nom :

- Utilisez des noms descriptifs dans votre code : Vous avez déjà retrouvé un vieux classeur dans un carton avec le nom « Trucs importants » sur la couverture ? C'était frustrant, n'est-ce pas ? Les noms de variables descriptifs et spécifiques simplifient la vie, et facilitent la lecture et la modification de votre code. Au lieu de **quantite** (ou pire, **qte**), ajoutez des détails : **quantite\_en\_stock**, **solde\_actuel**, etc.
- Utilisez des mots complets : Évitez d'abrégier ou de raccourcir les mots autant que possible, même si une alternative plus courte paraît évidente. Par exemple, **revenu\_annuel** est plus clair que **rev\_annuel**.
- **Suivez une convention d'appellation commune** : Il est recommandé de suivre la convention d'appellation du snake case en Python, conformément à la PEP8 qui est le standard de la syntaxe en Python : des noms composés de plusieurs mots séparés par des tirets bas ( `_` ) comme **nombre\_de\_chats**, **reponse\_finale**, **le\_meilleur\_developpeur\_python\_du\_monde**, etc.
- **Commencez avec une lettre ou le tiret bas** : Un nom de variable ne peut pas commencer par un nombre.
- **Utilisez uniquement les lettres, les chiffres et le tiret du bas... et surtout pas d'accents** : Par exemple, écrivez **bonjour\_1** mais pas **bonjour\_#1**.
- **N'oubliez pas que les noms de variables sont sensibles à la casse** : `age`, `Age` et `AGE` sont trois variables différentes.

## II. Classez vos données avec les types de données

### II.1. Qu'est ce qu'un type de données ?

Un **type de données** est une manière de classer les différentes variables. Prenons l'exemple d'un fruit que l'on peut classer selon son type, comme une pomme ou une orange, par exemple. Pour une variable, son type est déterminé par la nature de sa valeur. Par exemple, une variable qui

contient un nombre sera de type numérique, tandis qu'une variable qui contient une chaîne de caractères sera de type chaîne de caractères. Les types de données sont comparables aux types de données que l'on peut utiliser dans un tableur Excel, comme les nombres et les mots.

Les types de données les plus simples, ou primitifs, utilisés dans Python, sont :

- Les entiers (Integers, en anglais) ;
- Les virgules flottantes (Floats, en anglais) ;
- Les chaînes de caractères (Strings, en anglais) ;
- Les booléens.

## II.2. Les types de données numériques : entiers et virgules flottantes

Les types numériques se divisent en deux catégories dans Python : les entiers (Integers) et les virgules flottantes (Floats).

- Un entier est un nombre entier : 1, 4, 3 934, ...
- Une virgule flottante est un nombre décimal : 3.14, 563.2, 99.9, ...

**NB** : N'oubliez pas qu'en anglais, les virgules deviennent des points ! Par exemple, si l'on écrit 12,4 en français, dans Python on écrit 12.4.

Tout comme sur les nombres dans Excel ou avec une calculatrice, vous pouvez réaliser toutes sortes d'opérations arithmétiques avec des entiers et des virgules flottantes dans Python. Voici quelques exemples d'opérateurs arithmétiques que vous pouvez utiliser dans votre code :

- **x + y** : la somme de x et y (x plus y).
- **x - y** : la différence entre x et y (x moins y).
- **x \* y** : le produit de x et y (x fois y).
- **x / y** : le quotient de x et y (x divisé par y).
- **x % y** : le reste de x divisé par y.

## II.3. Les chaînes de caractères

**Chaîne de caractères**, c'est juste un mot sophistiqué pour ne pas dire... eh bien, « **mot** ». Une chaîne, c'est une information (généralement du texte) entourée de guillemets (soit simples soit doubles). Donc, **'bonjour'** avec des guillemets simples et **"bonjour"** avec des guillemets doubles sont tous les deux des chaînes. Et "au revoir" et "quoi de neuf" aussi.

Quand on utilise des nombres dans une variable, il faut juste se rappeler que '912' et "912" sont des chaînes simplement parce qu'ils sont entourés par des guillemets, alors que 912 est un nombre

entier. Concrètement, comme ce sont des informations dans votre code, '912' et "912" fonctionnent en tant que chaînes de caractères, alors que 912 fonctionne en tant que valeur numérique.

Voici quelques autres exemples de variables de type string (chaîne de caractères) :

- salutations = "bonjour !"
- le\_professeur = "Paul-Emile"
- prenom = "Sam"
- citation\_inspirante = "vous ratez 100 % des opportunités que vous ne saisissez pas"

## II.4. Les booléens

Ne soyez pas intimidé par ce nom étrange ! Un booléen, c'est assez simple. Il n'y a que deux options de valeur : **True** (vrai) ou **False** (faux).

Les booléens peuvent être très utiles pour déterminer si une action est réussie ou pas. Par exemple, si vous voulez avoir une variable qui enregistre le fait qu'il y a du soleil dehors ou pas, vous pouvez dire **temps\_ensoleille = True**. S'il n'y a pas de soleil, alors **temps\_ensoleille = False**.

## II.5. La fonction « type() »

La fonction « **type()** » en Python est utilisée pour déterminer le type de données d'une variable. Elle renvoie le type de l'objet en tant que sortie, tel que « **int** », « **str** », « **bool** », etc. Cela permet aux développeurs de vérifier rapidement et facilement le type de données avec lesquelles ils travaillent.

### Syntaxe de la fonction

```
type(objet)
```

- **paramètre** : objet - L'objet dont vous souhaitez connaître le type.
- **retourne** : Le type de l'objet.

### Exemples :

```
print(type(5)) # <class 'int'>
print(type(3.14)) # <class 'float'>
print(type("Hello")) # <class 'str'>
print(type([1, 2, 3])) # <class 'list'>
print(type((1, 2, 3))) # <class 'tuple'>
print(type({1: 'a'})) # <class 'dict'>
```

## Vérification du type

Pour vérifier si un objet est d'un certain type, vous pouvez comparer le résultat de « **type()** » avec le type souhaité.

```
x = 10
if type(x) is int:
 print("x est un entier")
```

# III. Enregistrez des groupes de données avec des listes

## III.1. Qu'est ce qu'une liste ?

Les pommes, les oranges, les poires. 🍏 🍊 🍐 Les chiens, les chats, les lapins. 🐶 🐱 🐰 Vous utilisez les listes tout le temps dans votre vie quotidienne, et c'est pareil dans Python. Vous devez avoir une liste pour y enregistrer une collection d'objets auxquels vous voulez accéder plus tard.

Dans Python, on utilise des crochets **[ ]** pour indiquer une liste. Le code suivant crée une liste de différentes plateformes de réseaux sociaux, et la sauvegarde dans une variable appelée **plateformes\_sociales**.

```
plateformes_sociales = ["Facebook", "Instagram", "Snapchat", "Twitter"]
```

Savez-vous quel type de données est utilisé dans cette liste ? Si vous avez vu les guillemets, vous avez peut-être immédiatement identifié ces valeurs comme des chaînes de caractères. Les listes acceptent n'importe quel type de données aussi, et même des combinaisons de types de données.

## III.2. Accédez aux éléments d'une liste

Pour accéder aux éléments d'une liste, on utilise un indice. Chaque élément a un indice qui lui correspond, selon sa position dans la liste. Vous obtenez la valeur de cet indice avec la syntaxe suivante : **liste[indice]**. Elle vous renverra la valeur de la liste qui est à la position de l'indice. L'important est de noter que dans la plupart des langages de programmation, y compris Python, l'indice commence à **0**, pas **1**. Donc si vous voulez accéder au premier élément dans la liste ci-dessus, vous devez taper :

```
>>> plateformes_sociales[0]
"Facebook"
```

Pour accéder au deuxième élément, tapez :

```
>>> plateformes_sociales[1]
"Instagram"
```

... et ainsi de suite.

En particulier dans Python, vous pouvez aussi accéder aux éléments en sens inverse, en utilisant des nombres négatifs. Pour accéder au dernier élément de la liste, utilisez l'indice **-1**.

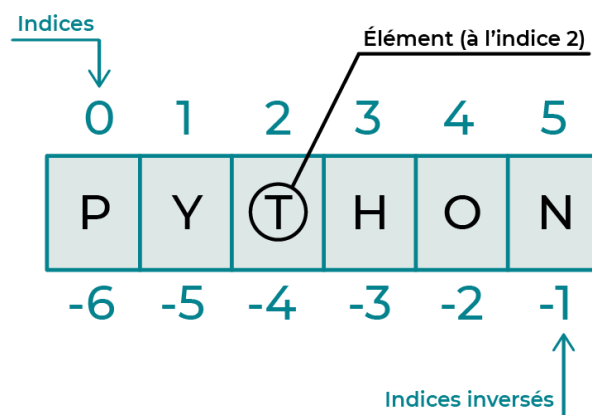
```
>>> plateformes_sociales[-1]
"Twitter"
```

## III.2. Accédez aux caractères d'une chaîne

Les indices fonctionnent aussi avec les chaînes de caractères ! En fait, les chaînes de caractères sont juste des listes de caractères. Chaque caractère correspond à un indice qui va de zéro à la longueur de la chaîne.

Par exemple, dans la chaîne **langage="PYTHON"**, **langage[2]** vous renverra « **T** ». Tout simplement parce que l'indice **2** dans le mot « **PYTHON** » est le « **T** ». Ou bien, avec l'indice de la position inverse, vous devez utiliser **langage[-4]** pour accéder à « **T** ».

```
>>> langage_de_programmation = "PYTHON"
>>> langage_de_programmation[2]
"T"
>>> langage_de_programmation[-4]
"T"
```



## III.3. Modifiez les éléments d'une liste

Maintenant que vous savez créer une liste et accéder à chaque case de cette liste, il est temps de passer à l'étape suivante : la modification des éléments de cette liste. C'est très simple, pour modifier une case d'une liste, il suffit d'utiliser l'indice de la case que l'on souhaite modifier, et d'y affecter la nouvelle valeur à l'aide de l'opérateur d'affectation (=), tout comme pour une variable.

```
>>> plateformes_sociales[2] = "LinkedIn"
>>> print(plateformes_sociales)
["Facebook", "Instagram", "LinkedIn", "Twitter"]
```

### III.4. Ajoutez, retirez et trieز les listes

Dans Python, c'est très simple de réaliser toutes sortes d'opérations avec les listes. Au lieu de créer une toute nouvelle liste à chaque fois que vous voulez ajouter, retirer ou trier des éléments, vous pouvez faire appel à une **méthode de liste**. Nous allons voir les méthodes plus tard, mais pour l'instant, vous devez juste savoir qu'une méthode est une façon de réaliser une opération spécifique sur un élément.

Par exemple, pour ajouter une plateforme de réseau social à la fin de la liste existante, vous pouvez utiliser la méthode **append()** :

```
>>> plateformes_sociales.append("TikTok")
>>> print(plateformes_sociales)
["Facebook", "Instagram", "Snapchat", "Twitter", "TikTok"]
```

Pour retirer un élément spécifique d'une liste, vous pouvez utiliser la méthode **remove()** :

```
>>> plateformes_sociales.remove("Snapchat")
>>> print(plateformes_sociales)
["Facebook", "Instagram", "Twitter", "TikTok"]
```

**remove()** retire uniquement la première instance du terme que vous saisissez.

Pour connaître la longueur de la liste, utilisez la méthode **len()** :

```
>>> len(plateformes_sociales)
4
```

**Attention** : N'essayez pas d'accéder à un indice supérieur ou égal à la taille de la liste, car cela provoquera une erreur d'index (IndexError). Ici, **plateformes\_sociales[4]** provoquerait une erreur d'index, car cette case n'existe pas.

La dernière méthode que nous allons voir est **sort()**. Elle trie les éléments de la liste. Le tri se fait alphabétiquement pour les listes de chaînes, et numériquement pour les listes de nombres.

```
>>> plateformes_sociales.sort()
```

Il y a beaucoup d'autres méthodes que vous pouvez utiliser avec les listes. Vous les trouverez dans la [documentation Python](#).

Voici un avant-goût de ces méthodes :

|                  |                                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>extend()</b>  | Ajoute plusieurs éléments à la fin de la liste.                                                                        |
| <b>insert()</b>  | Insère un élément à une position donnée dans la liste.                                                                 |
| <b>pop()</b>     | Supprime et renvoie l'élément à une position donnée de la liste, ou le dernier élément si aucun indice n'est spécifié. |
| <b>index()</b>   | Renvoie la première occurrence de l'élément spécifié dans la liste.                                                    |
| <b>count()</b>   | Renvoie le nombre d'occurrences de l'élément spécifié dans la liste.                                                   |
| <b>reverse()</b> | Inverse l'ordre des éléments de la liste.                                                                              |



## III.5. Les tuples

Les tuples sont une autre structure de Python qui sert à enregistrer des données. Au lieu des crochets `[]`, ils se caractérisent par les parenthèses `()`.

```
plateformes_sociales_tuple = ("Facebook", "Instagram", "TikTok", "Twitter")
```

Beaucoup des propriétés des tuples sont similaires à celles des listes normales. Par exemple, les listes et les tuples utilisent tous deux les indices. La principale différence est que les tuples sont **immuables**, alors que les listes sont **modifiables**.

```
plateformes_sociales_tuple[0]="Whatsapp" # Cette ligne provoquerait une erreur
```

Les tuples sont généralement plus rapides que les listes pour les opérations d'accès et d'itération, car leur taille est fixe. Les tuples ont moins de méthodes disponibles, car ils sont immuables. Les opérations sont limitées :

| Méthode        | Description                                                                    | Syntaxe                           | Exemple                                |
|----------------|--------------------------------------------------------------------------------|-----------------------------------|----------------------------------------|
| <b>count()</b> | Renvoie le nombre d'occurrences d'un élément dans le tuple.                    | <code>tuple.count(element)</code> | <code>mon_tuple.count(2)</code><br>→ 2 |
| <b>index()</b> | Renvoie l'index de la première occurrence d'un élément. Si non trouvé, erreur. | <code>tuple.index(element)</code> | <code>mon_tuple.index(3)</code><br>→ 2 |

### Autres opérations

| Opération     | Description                                               | Exemple                               |
|---------------|-----------------------------------------------------------|---------------------------------------|
| Concaténation | Combine deux tuples en un seul.                           | <code>tuple3 = tuple1 + tuple2</code> |
| Répétition    | Répète les éléments d'un tuple un certain nombre de fois. | <code>tuple4 = tuple1 * 3</code>      |

## III.6. Trouvez un élément

Pour savoir si un élément est présent dans une liste ou un tuple, on peut utiliser l'opérateur « **in** ». Cet opérateur retourne **True** si l'élément est présent dans la séquence, et **False** sinon.

```
>>> nombres = [1,2,3,4,5]
>>> 5 in nombres
True
>>> 8 in nombres
False
```

## IV. Enregistrez des données complexes avec des dictionnaires

### IV.1. Qu'est ce qu'un dictionnaire ?

Parfois vous avez besoin de représenter des données plus complexes que des nombres, des chaînes ou des listes.

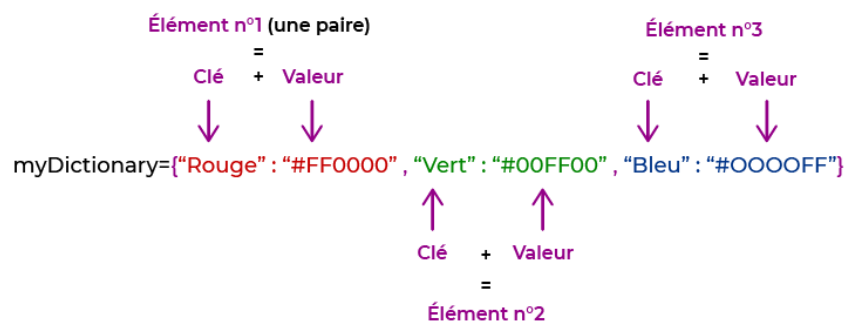
Un dictionnaire est une structure de données qui enregistre des données dans des paires clés-valeurs. Voici un exemple d'une clé et d'une valeur :

```
"responsable_de_campagne": "Jeanne d'Arc"
```

La **clé** est "responsable\_de\_campagne" et la **valeur** est "Jeanne d'Arc".

Les dictionnaires sont indiqués par des accolades **{ }** au début et à la fin. Chaque paire **clé-valeur** comprend un deux-points « : » placé entre la clé et la valeur, et une virgule « , » à la fin. Chaque dictionnaire doit être composé de clés uniques.

Dans le diagramme ci-dessous, le dictionnaire défini a trois éléments, et chaque élément est une paire clé-valeur.



### IV.2. Créez un dictionnaire

Imaginons que vous vouliez enregistrer des informations à propos d'une nouvelle campagne pour une entreprise de nourriture pour chien. Vous allez probablement devoir sauvegarder un nom de campagne, des dates de début et de fin, le nom d'un responsable de campagne, et des influenceurs importants. Vous pouvez sauvegarder toutes ces informations dans une variable avec un dictionnaire.

Pour enregistrer tout cela, vous pouvez sauvegarder un dictionnaire comme ça :

```
nouvelle_campagne = {
 "responsable_de_campagne": "Jeanne d'Arc",
 "nom_de_campagne": "Campagne nous aimons les chiens",
 "date_de_début": "01/01/2020",
 "influenceurs_importants": ["@MonAmourDeChien",
 "@MeilleuresFriandisesPourChiens"]
}
```

Vous pouvez aussi créer un nouveau dictionnaire avec des accolades vides `{ }` ou la fonction `dict()`, et avec des paires clés-valeurs comme indiqué ci-dessous :

```
taux_de_conversion = {}
taux_de_conversion['facebook'] = 3.4
taux_de_conversion['instagram'] = 1.2
taux_de_conversion = dict()
taux_de_conversion['facebook'] = 3.4
taux_de_conversion['instagram'] = 1.2
```

## IV.3. Accédez à une valeur d'un dictionnaire

Pour accéder aux différentes valeurs, vous pouvez utiliser la clé pour chacune des paires clés-valeurs.

```
>>> nouvelle_campagne['responsable_de_campagne']
"Jeanne d'Arc"
>>> taux_de_conversion['facebook']
3.4
```

## IV.4. Réalisez des opérations avec les dictionnaires

Comme pour les listes, plusieurs méthodes (ou opérations) intégrées à Python vous permettent de manipuler les données dans les dictionnaires.

### Ajoutez une paire clé-valeur

Pour ajouter une paire clé-valeur à un dictionnaire, ajoutez juste une nouvelle clé dans le dictionnaire existant. Si la clé existe déjà, vous l'écraserez en définissant une valeur. Le code suivant crée un dictionnaire appelé « **infos\_labradoodle** », et enregistre des informations à propos du poids et de l'origine des labradoodles, un croisement de chiens.

```
infos_labradoodle = {
 "poids": "13 à 16 kg",
 "origine": "États-Unis"
}
```

Pour ajouter une nouvelle paire clé-valeur, comme le nom scientifique du labradoodle, ajoutez simplement :

```
infos_labradoodle['nom_scientifique'] = "Canis lupus familiaris"
```

Maintenant, « **infos\_labradoodle** » renvoie :

```
>>> print(infos_labradoodle)
{'poids': '13 à 16 kg', 'origine': 'États-Unis', 'nom_scientifique': 'Canis lupus familiaris'}
```

Si vous écrivez « **infos\_labradoodle["poids"]="45 kg"** », la valeur existante sera écrasée.

## Supprimer une paire clé-valeur

Pour supprimer une paire clé-valeur, vous pouvez utiliser le mot-clé « **del** » et la clé que vous voulez supprimer, ou encore la méthode « **pop** ». Pour supprimer la paire clé-valeur "**origine**", écrivez :

```
>>> del infos_labradoodle["origine"]
>>> print(infos_labradoodle)
{"poids": "13 à 16 kg", "nom scientifique": "Canis lupus familiaris"}
```

Voici encore quelques méthodes couramment utilisées pour manipuler des dictionnaires :

| Méthodes        | Utilisation                                                                                                                                         | Syntaxe                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <b>keys()</b>   | Retourne une vue sur les clés du dictionnaire.                                                                                                      | <code>nom_du_dictionnaire.keys()</code>    |
| <b>values()</b> | Retourne une vue sur les valeurs du dictionnaire.                                                                                                   | <code>nom_du_dictionnaire.values()</code>  |
| <b>items()</b>  | Retourne une vue sur les couples (clé, valeur) du dictionnaire.                                                                                     | <code>nom_du_dictionnaire.items()</code>   |
| <b>get(clé)</b> | Retourne la valeur associée à la clé spécifiée. Si la clé n'est pas présente dans le dictionnaire, retourne la valeur <code>None</code> .           | <code>nom_du_dictionnaire.get(clés)</code> |
| <b>pop(clé)</b> | Supprime la clé spécifiée et retourne la valeur associée. Si la clé n'est pas présente dans le dictionnaire, retourne la valeur <code>None</code> . | <code>nom_du_dictionnaire.pop(clés)</code> |
| <b>clear()</b>  | Supprime tous les éléments du dictionnaire.                                                                                                         | <code>nom_du_dictionnaire.clear()</code>   |

## Vérifier l'existence d'une clé-valeur spécifique

Vous pouvez utiliser le mot-clé « **in** » pour vérifier si une clé spécifique existe dans un dictionnaire. Pour faire cela, spécifiez la clé que vous voulez rechercher, écrivez le mot-clé « **in** » et le nom de la variable du dictionnaire que vous examinez. Le résultat renvoie un booléen qui indique si la clé est dans ce dictionnaire. Par exemple, si vous voulez voir si la clé « **poids** » existe dans votre dictionnaire « **infos\_labradoodle** », écrivez le code qui suit :

```
>>> "poids" in infos_labradoodle
True
>>> "race" in infos_labradoodle
False
```

## Tests de connaissances



# CHAPITRE III : GEREZ LA LOGIQUE DE VOS PROGRAMMES AVEC PYTHON

## Objectif du chapitre

Maîtriser les concepts de contrôle de flux, de répétition, de modularité et de gestion des erreurs en Python, tout en intégrant des principes mathématiques pour concevoir des programmes robustes et efficaces.

## Objectifs spécifiques

- Comprendre et appliquer les structures conditionnelles (if, elif, else) pour contrôler le flux d'exécution d'un programme ;
- Identifier et utiliser les différentes structures de boucle (for, while) pour répéter des tâches, en se basant sur des algorithmes mathématiques pour optimiser les processus.
- Définir et créer des fonctions pour regrouper des tâches répétitives et améliorer la lisibilité du code, en appliquant des concepts d'ingénierie logicielle pour une meilleure organisation du projet.
- Utiliser des techniques de débogage pour détecter et corriger les erreurs dans le code, en appliquant des concepts analytiques issus des mathématiques pour diagnostiquer les problèmes.

## I. Contrôlez le déroulement du programme avec des conditions

### I.1. Définition

Le **déroulement du programme** est l'ordre dans lequel les lignes de code sont exécutées. Certaines lignes seront lues une fois seulement, d'autres plusieurs fois. D'autres encore pourraient être complètement ignorées, tout dépend de la façon dont vous les avez codées. Dans ce chapitre, nous allons regarder comment programmer votre code avec des instructions conditionnelles.

Les **instructions conditionnelles** sont un moyen de contrôler la logique et le déroulement de votre code avec des conditions.

Vous prenez tout le temps des décisions d'après des conditions dans votre vie quotidienne. Par exemple : *S'il fait beau, je vais à la plage*. Dans ce scénario, votre décision d'aller à la plage dépend du temps qu'il fait. Le temps qu'il fait est donc la condition.

## 1.2. Les conditions avec les instructions if/else

L'un des blocs essentiels à la structure d'un déroulement conditionnel est l'instruction **if**. Avec une instruction **if**, vous pouvez exécuter certaines lignes de code uniquement si une certaine condition est **vraie (True)**. Si cette condition est **fausse (False)**, le code ne s'exécutera pas.

Par exemple, pour notre exemple sur le temps qu'il fait :

```
if ensoleille:
 print("on va à la plage !")
```

La ligne 2 s'exécute uniquement si la condition « **ensoleille** » est True.

Gardons l'exemple du temps qu'il fait, et allons plus loin pour y ajouter une instruction « **else** » :

*S'il fait beau dehors, je vais à la plage.*

*Sinon (par exemple s'il pleut), je reste à la maison !*

Avec les instructions if/else dans Python, la syntaxe ressemble à ça :

```
if mon_booleen:
 # exécuter le code quand mon_booleen est vrai
else:
 # exécuter le code quand mon_booleen est faux
```

Donc pour afficher une instruction sous une certaine condition, vous pouvez suivre les étapes suivantes :

```
ensoleille = True
if ensoleille:
 print("on va à la plage !")
else:
 print("on reste à la maison !")
```

Dans cet extrait de code, puisque « **ensoleille** » est vrai, vous verrez « on va à la plage ! » s'afficher dans votre terminal. Si sa valeur était fausse, le code afficherait « on reste à la maison ! »

## 1.3. Les conditions alternatives avec la clause elif

Les instructions **if/elif/else** vous permettent de définir des conditions multiples. Le mot-clé **elif** vous permet d'ajouter autant de conditions que vous voulez. Vous devez ensuite terminer avec une instruction **else**.

Admettons que vous vouliez aller à la plage s'il fait chaud dehors, et que vous vouliez faire un bonhomme de neige s'il neige. Et sinon, vous restez à la maison. Vous pouvez coder tout ça avec la syntaxe ci-dessous :

```
enseleille = False
neige = True
if enseleille:
 print("on va à la plage !")
elif neige:
 print("on fait un bonhomme de neige")
else:
 print("on reste à la maison !")
```

## I.4. Les conditions multiples avec des opérateurs logiques

Si vous voulez vérifier plusieurs conditions pour un seul résultat dans la même instruction « **if** », vous pouvez utiliser les opérateurs logiques :

- **and** : vérifie si deux conditions sont toutes les deux vraies.
- **or** : vérifie si au moins une condition est vraie.
- **not** : vérifie si une condition n'est pas vraie (c'est-à-dire fausse).

Ces opérateurs peuvent être combinés et mélangés selon vos besoins. Admettons que vous vouliez aller à la plage seulement s'il y a du soleil ET que c'est le weekend. Mais s'il y a du soleil et que nous sommes au milieu de la semaine, vous devez être au travail.

```
avec_soleil = True
en_semaine = False
if avec_soleil and not en_semaine:
 print("on va à la plage !")
elif avec_soleil and en_semaine:
 print("on va au travail !")
else:
 print("on reste à la maison !")
```

**Attention :** Il est important de garder en tête l'ordre d'évaluation des conditions dans une expression conditionnelle, surtout lorsque des opérateurs "or" et "and" sont utilisés. L'opérateur "or" s'arrêtera dès qu'il trouvera une condition vraie, tandis que l'opérateur "and" s'arrêtera dès qu'il rencontrera une condition fausse. Il est donc conseillé de bien réfléchir à l'ordre des conditions afin d'optimiser la performance du programme, et d'éviter des évaluations inutiles.

## I.5. Les conditions complexes avec des expressions comparatives

Les expressions comparatives vous permettent de comparer différentes expressions entre elles, et d'évaluer si une expression est vraie ou fausse. Si vous avez deux valeurs **a** et **b**, vous pouvez utiliser les opérateurs de comparaison suivants dans Python :

- Égal à : **a == b**
- Non égal à : **a != b**



- Moins que : **a < b**
- Moins que ou égal à : **a <= b**
- Plus que : **a > b**
- Plus que ou égal à : **a >= b**

### Par exemple :

```
nombre_de_sieges = 30
nombre_dinvites = 25

if nombre_dinvites < nombre_de_sieges:
 # autoriser plus d'invités
else:
 # ne pas autoriser plus d'invités
```

**Attention :** A ne pas vous tromper avec le comparateur « égal à » `==`. Quand vous testez si une variable est égale à une autre, vous devez utiliser `==` au lieu de `=`. Quand vous voulez assigner une valeur à une variable, vous devez utiliser un « égal à » `=`.

## I.5. Simplifiez le code avec les matchs cases

Le `match` est une fonctionnalité pour faciliter la comparaison de valeurs à l'aide de motifs. Le but est de simplifier la syntaxe de certaines structures courantes qui utilisent des blocs `if`, `elif` et `else`. Le **`match`** offre une alternative plus concise et plus lisible.

Découvrons les mots-clés utilisés dans cette nouvelle structure conditionnelle. Le mot-clé **`match`** est utilisé pour indiquer le début d'un bloc de `match case`, suivi de la variable à évaluer. Le mot-clé **`case`** est utilisé pour vérifier si une valeur donnée correspond à une condition spécifique dans ce bloc. Le symbole `_` est utilisé pour définir une action à effectuer si aucune condition ne correspond.

Par exemple, si vous voulez tester la valeur d'une variable et exécuter une action différente pour chaque cas, vous pouvez utiliser le `match`. Imaginons que vous vouliez exécuter une action différente selon la valeur de la variable `fruit` :

```
fruit = "pomme"
match fruit:
 case "pomme":
 print("J'aime les pommes !")
 case "banane":
 print("Je n'aime pas les bananes.")
 case "orange":
 print("Les oranges sont bonnes pour la santé.")
 case _:
 print("Je ne connais pas ce fruit.")
```

Dans cet exemple, on teste la valeur de la variable `fruit` et on exécute une action différente pour chaque cas possible. Si `fruit` est égal à `"pomme"`, on affiche `"J'aime les pommes !"`. Si `fruit` est égal à `"banane"`, on affiche `"Je n'aime pas les bananes."` et ainsi de suite. Si la variable `fruit` n'est égale à

aucun des cas testés, on exécute la dernière action. C'est le cas par défaut, où le `_` représente la valeur par défaut. Il est utilisé lorsque toutes les autres options de cas ne correspondent pas à la valeur testée.

**Remarque :** Vous avez vu un format spécifique d'indentation dans les exemples de code précédents. Après une instruction `if`, tout le code qui suit doit être indenté, c'est-à-dire qu'il doit être décalé par rapport à la marge. Sinon Python ne fait pas la différence entre l'instruction et le code à exécuter.

## II. Répétez des tâches facilement à l'aide de boucles

### II.1. Définition

En programmation, il y a des ensembles d'instructions qu'il faut répéter plusieurs fois. Par exemple, si vous voulez réaliser une même tâche sur tous les éléments d'une liste. Et si vous avez une liste de campagnes et que vous voulez les afficher toutes ?

Quand vous avez besoin de répéter un ensemble d'instructions, parfois vous savez combien de fois vous allez le répéter, d'autres fois vous ne savez pas. Parfois, le nombre de répétitions n'est pas important, et vous voulez répéter le code jusqu'à ce qu'une certaine condition soit remplie. Pour tous ces usages, vous allez utiliser des boucles.

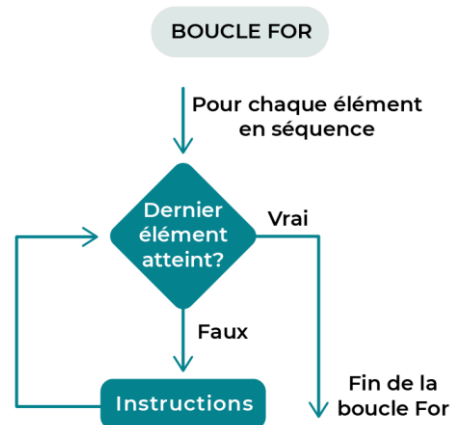
### II.2. Utilisez la boucle for

La boucle **for** est le type de boucle centrale dans Python. Une boucle **for** est utilisée pour itérer sur une séquence. Ça peut être une liste, un tuple, un dictionnaire ou même une chaîne de caractères. Avec une boucle **for**, vous pouvez exécuter le même code pour chaque élément dans cette séquence.

Avec Python, c'est très facile de créer des boucles. Si vous voulez afficher tous les éléments dans une liste, le code ressemblera à ça :

```
racas_de_chien = ["golden retriever", "chihuahua", "terrier", "carlin"]
for chien in racas_de_chien:
 print(chien)
```

Dans ce code, chaque élément dans **racas\_de\_chien** sera affiché dans le terminal. **chien** est un nom de variable qui se met à jour pour être l'élément suivant à chaque fois que la boucle se répète. Vous pouvez modifier **chien** en **cocorico** et il s'exécutera toujours de la même façon, mais cela n'a pas beaucoup d'intérêt !



Vous pouvez faire le même genre de boucle **for** si vous voulez passer en boucle chaque caractère d'une chaîne. Pour boucler un certain nombre de fois, vous pouvez utiliser la fonction « **range()** ». Elle renverra une séquence de nombres qui vont de 0 à un nombre de fin déterminé.

Vous n'avez pas encore appris les fonctions, mais ça ne va pas tarder ! Pour l'instant, vous devez juste savoir qu'on peut saisir un nombre dans la fonction « **range()** », et qu'elle renvoie une séquence de nombres allant de 0 à ce nombre moins un.

```
for x in range(5):
 print(x)
```

Ce code affichera 0, 1, 2, 3, 4 en séquence.

La fonction « **range()** » est réglée sur **0** par défaut pour la valeur de début. Vous pouvez la modifier en ajoutant un autre nombre entier comme ça : **range(4, 10)** ! Cette plage renvoie les valeurs de **4** à **9** (sans inclure **10**).

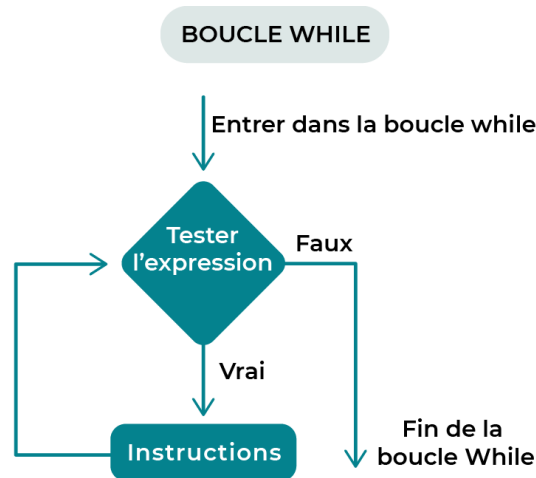
## II.3. Utilisez la boucle while

La boucle **for** vous permet d'exécuter du code un nombre spécifique de fois, alors que la boucle **while** continue de s'exécuter jusqu'à ce qu'une certaine condition soit remplie. Précédemment, vous avez découvert des conditions, ou instructions, qui évaluent si une expression est vraie ou fausse. C'est la même chose ici : le code dans l'instruction **while** s'exécute jusqu'à ce que la condition devienne fausse.

L'extrait de code ci-dessous vérifie la capacité actuelle, et l'augmente d'une unité jusqu'à ce que la capacité maximale soit atteinte (**+= 1** augmente la valeur actuelle de 1).

```
capacite_maximale = 10
capacite_actuelle = 3
while capacite_actuelle < capacite_maximale:
 capacite_actuelle += 1
```

Comme cette **capacite\_actuelle** commence à 3, ce code s'exécute 7 fois jusqu'à ce que **capacite\_actuelle** atteigne 10.



Il est également important de connaître les **boucles infinies**. Si la condition que vous avez réglée est toujours vraie, la boucle va s'exécuter pour toujours !

Par exemple :

```
x = 0
while x != 5:
 x += 2
```

Dans cette situation, x n'atteindra jamais 5 !

On utilise généralement une boucle **for** lorsqu'on connaît à l'avance le nombre d'itérations à effectuer, et une boucle **while** lorsqu'on ne connaît pas à l'avance le nombre d'itérations à effectuer, et que la boucle doit s'arrêter lorsque la condition spécifiée est satisfaite.

## II.4. Utilisez les mot-clés **break** et **continue**

Il est courant d'utiliser des boucles pour répéter une série d'instructions plusieurs fois. Parfois, il peut être utile d'interrompre ou de sauter une itération dans la boucle. C'est là que les instructions **break** et **continue** entrent en jeu.

L'instruction **break** permet de sortir d'une boucle prématurément. Elle est souvent utilisée lorsqu'une condition est rencontrée, et que l'on souhaite arrêter la boucle avant qu'elle ne se termine normalement. Voici un exemple :

```
for i in range(10):
 if i == 5:
 break
 print(i)
```

Dans cet exemple, la boucle **for** va itérer de 0 à 9, mais lorsque la valeur de **i** atteint 5, l'instruction **break** est exécutée et la boucle est interrompue. Le programme affichera les valeurs de 0 à 4, mais pas 5, car la boucle a été interrompue grâce au mot-clé **break**.

L'instruction **continue** permet de passer à la prochaine itération de la boucle, sans exécuter le reste du code présent dans la boucle pour l'itération en cours. Elle est souvent utilisée lorsqu'une condition est rencontrée, mais que l'on souhaite continuer la boucle sans exécuter le reste du code.

```
liste = [1, 2, 3, 4, 5]
Boucle for sur la liste
for element in liste:
 if element == 3:
 # Si l'élément vaut 3, on passe à l'itération suivante sans exécuter le
 # reste du code
 continue
 # Dans tous les autres cas, on exécute le reste du code
 print(element)
```

Si un élément vaut **3**, l'instruction `continue` est exécutée, ce qui fait passer directement à l'itération suivante sans exécuter le reste du code présent dans la boucle pour cet élément. Ainsi, la sortie de ce code nous donnera :

```
1
2
4
5
```

## III. Regroupez des tâches en utilisant des fonctions

### III.1. Qu'est ce qu'une fonction ?

Une **fonction** est un bloc de code avec un but spécifique, auquel vous pouvez donner un nom. Quand vous appelez cette fonction, vous exécutez le code qu'elle contient. Les fonctions vous laissent saisir des paramètres pour exécuter le même code sur différentes valeurs.

Il y a différents types de fonctions dans Python :

- Les fonctions intégrées fournies avec Python.
- Les fonctions définies par l'utilisateur que les développeurs (vous !) créent.

Les fonctions sont des blocs de code réutilisables qui permettent d'organiser et de structurer le code, ainsi que de faciliter sa maintenance. En Python, la création d'une fonction se fait à l'aide du mot-clé « **def** », suivi du nom de la fonction et des éventuels paramètres entre parenthèses.



Nous allons maintenant explorer trois types de fonctions différents pour découvrir toutes les possibilités de cette fonctionnalité :

- Les fonctions sans paramètres.
- Les fonctions avec paramètres.
- Les fonctions avec une valeur de retour.

## III.2. Utilisez une fonction sans paramètres

Une **fonction sans paramètres et sans valeur de retour** est la fonction la plus basique que l'on puisse créer en programmation. Elle est assimilable à un bloc de code à lancer quand cela est nécessaire. Elle est utile pour encapsuler un bloc de code répétitif, et faciliter sa réutilisation dans différents endroits du programme.

```
def afficher_message():
 print("Bonjour, comment ça va ?")
```

Cette fonction s'appelle « **afficher\_message** » et ne prend aucun paramètre en entrée, car elle est définie avec des parenthèses vides. Elle n'a pas de valeur de retour car elle se contente d'afficher un message à l'écran en utilisant la fonction « **print()** ». Pour appeler cette fonction, il suffit d'utiliser son nom et les parenthèses vides, car il n'y a pas de paramètres :

```
afficher_message()
```

## III.3. Utilisez une fonction avec paramètres

On peut également créer une **fonction avec des paramètres**, qui permettent de transmettre des valeurs à la fonction. Les paramètres sont simplement listés entre parenthèses, séparés par des virgules. Voici un exemple d'une fonction qui prend deux paramètres, un nom et un prénom, et qui les affiche ensuite.

```
def afficher_nom_prenom(nom, prenom):
 print("Nom :", nom)
 print("Prénom :", prenom)
```

Pour appeler cette fonction, il faut préciser les valeurs à transmettre aux paramètres, soit deux chaînes de caractères.

```
afficher_nom_prenom("TEJIOGNI", "Marc")
```

La fonction affichera :

```
Nom : TEJIOGNI
Prénom : Marc
```

### III.4. Utilisez une fonction avec une valeur de retour

Une **fonction avec une valeur de retour** est une fonction qui peut prendre des paramètres et effectuer des opérations, mais qui renvoie également une valeur à la fin. Cette valeur peut être utilisée à d'autres endroits du programme. Par exemple, si nous avons une fonction qui calcule la somme de deux nombres, nous pouvons stocker le résultat dans une variable pour l'utiliser plus tard dans notre programme.



La fonction "somme" additionne les deux paramètres en entrée

```
def calculer_somme(a, b):
 resultat = a + b
 return resultat
```

Dans cet exemple, la fonction prend deux nombres en paramètres, les additionne, puis **renvoie** le résultat de l'addition. Le résultat est stocké dans la variable **resultat** et est renvoyé à l'aide du mot-clé « **return** ».

Pour utiliser la valeur de retour de la fonction, nous pouvons stocker le résultat dans une variable, comme ceci :

```
somme = calculer_somme(2, 3)
print(somme) #Ce print affichera 5
```

Dans ce cas, la variable « **somme** » contiendra la valeur renvoyée par la fonction, qui est le résultat de l'addition de 2 et 3, soit 5. Cette valeur peut ensuite être utilisée à d'autres endroits du programme, si besoin.

**Remarque :** Il est possible en python de retourner **plusieurs** valeurs dans une fonction en les séparant par **des virgules** dans l'instruction de retour de la fonction. Les valeurs retournées seront automatiquement regroupées dans un **tuple**.

**Exemple :**

```
def calculer_stats(lst):
 somme = sum(lst)
 moyenne = somme / len(lst)
 maximum = max(lst)
 return somme, moyenne, maximum

Exemple d'utilisation
nombres = [10, 20, 30, 40, 50]
somme, moyenne, max_val = calculer_stats(nombres)

print("Somme:", somme)
print("Moyenne:", moyenne)
print("Maximum:", max_val)
```

## IV. Écrivez du code en évitant les erreurs courantes

### IV.1. Pourquoi écrire du code de façon structurée ?

Avant de plonger dans les méthodes de programmation, voyons pourquoi il faut garder le code structuré autant que possible.

Pensez à l'endroit chez vous le moins bien rangé et moins bien organisé. Un tiroir, un placard ou même une pièce entière. C'est difficile de trouver quelque chose au milieu de tout ça, n'est-ce pas ? Si vous venez de vous en servir, ça peut encore aller, mais quand ça fait un moment, ça peut devenir compliqué.

Avec le code, c'est pareil... ou pire. Si vous n'y avez pas touché depuis plusieurs mois et que vous ne l'avez pas écrit avec rigueur, vous allez perdre du temps à vous rappeler les différentes fonctions et leur structure.

Maintenant, imaginez que vous avez hérité du tiroir/du placard/de la pièce d'une personne désorganisée et qui ne vit plus ici. C'est à ça que ça ressemble, de travailler avec le code de quelqu'un d'autre qui n'a pas été rigoureux !

Vous devez avoir une définition partagée avec d'autres développeurs sur ce que signifie « avoir un code structuré ». De cette façon, vous comprendrez mieux le code, sans avoir à connaître la personne qui l'a écrit.

D'autre part, il faut aussi prendre en compte toutes les erreurs qui peuvent faire planter le code, et savoir les prévoir pour éviter une mauvaise surprise.

Vous comprenez maintenant pourquoi vous devez coder correctement. Voyons comment faire.

### IV.2. Comment écrire du code de façon structurée

#### Ne vous répétez pas

Le premier commandement de la programmation est le **DRY** : ne vous répétez pas (Don't Repeat Yourself, en anglais). En tant que développeur, vous voulez que tout soit aussi rapide et efficace que possible, et automatiser autant que vous pouvez. Si vous vous retrouvez à copier et coller souvent des morceaux de code, c'est signe que vous pouvez isoler ce à quoi servent ces lignes de code.

Admettons que vous vouliez calculer le taux de conversion et l'afficher dans plusieurs campagnes. Vous pouvez le copier et coller pour chaque campagne, comme dans le code ci-dessous :

```
taux_de_conversion1 = premiere_campagne['visiteurs'] / premiere_campagne['conversions']
taux_de_conversion2 = seconde_campagne['visiteurs'] / seconde_campagne['conversions']
taux_de_conversion3 = troisieme_campagne['visiteurs'] / troisieme_campagne['conversions']
```

Si vous commencez à écrire du code comme ça, la première chose à faire est de le transformer en fonction. Vous pouvez écrire une fonction appelée **calculer\_taux\_de\_conversion**, et tout faire en une fois.



```
def calculer_taux_de_conversion(campagne):
 taux de conversion = campagne['visiteurs'] / campagne['conversions']
 return taux de conversion
taux de conversion1 = calculer_taux_de_conversion(premiere_campagne)
taux de conversion2 = calculer_taux_de_conversion(seconde_campagne)
taux de conversion3 = calculer_taux_de_conversion(troisieme_campagne)
```

Les deux extraits de code ci-dessus font la même chose, mais il y en a un mieux structuré que l'autre, et qui suit le principe **DRY** !

### Respectez le principe de responsabilité unique

Quand vous organisez quelque chose, il vaut mieux avoir un endroit où ranger chaque objet, car vous savez qu'il sera plus facile à retrouver. C'est pareil pour écrire des fonctions. Chaque fonction doit être responsable d'une seule et unique fonctionnalité, et rien de plus.

Admettons que vous vouliez calculer les taux de conversion pour plusieurs campagnes, et sauvegarder les données dans un fichier **CSV**.

**Remarque :** CSV signifie **comma-separated values (valeurs séparées par des virgules)**. Le format de fichier **CSV** est utilisé pour enregistrer des données tabulaires (c'est-à-dire des informations structurées dans un tableau), comme un tableur ou une base de données.

Au lieu d'appeler une fonction « **calculer\_taux\_de\_conversion\_sauvegarder\_csv()** », nous allons séparer deux tâches en fonctions séparées : « **calculer\_taux\_de\_conversion()** » et « **sauvegarder\_csv()** ». Chaque fonction doit être responsable d'une seule tâche. Dès que vous essayez d'accomplir plus d'une chose dans une seule fonction, vous risquez de ne plus comprendre ce que vous vouliez faire à l'origine. Le fait de séparer les responsabilités vous permet de définir un nom plus descriptif pour la fonction, de déboguer le code plus facilement, et de mieux comprendre le code.

### Commentez votre code

Les commentaires sont un moyen très pratique de rappeler, à vous et aux autres développeurs, ce qu'un morceau de code est censé faire. Il y a différents standards pour les commentaires. Votre code doit être suffisamment lisible pour ne pas avoir besoin de trop de commentaires pour que quelqu'un d'autre le comprenne.

Dans Python, on trouve des **commentaires à ligne unique et multiple**.

- Les **commentaires à ligne unique** sont indiqués par « # ». Exemple : La phrase suivante est un commentaire à ligne unique à propos d'une méthode :

```
#ceci est un commentaire
```

- Les **commentaires sur plusieurs lignes ou multiple**, on utilise généralement des chaînes de caractères multilignes (triple guillemets ''' ou ''') qui ne sont pas affectées dans l'exécution si elles ne sont pas attribuées à une variable.

```
"""
Ceci est un commentaire multilignes.
Il peut couvrir plusieurs lignes.
Utilisé pour documenter ou commenter de longues sections.
"""
```

**Remarque :** Techniquement, ces chaînes multilignes sont appelées des **docstrings**. Si elles ne sont pas utilisées comme **docstring** (c'est-à-dire, si elles ne suivent pas une définition de fonction) alors elles produisent des commentaires multilignes sinon elles servent à **documenter ou à décrire**.

## Documentez votre code

Les **docstrings** sont des chaînes de documentation qui sont utilisées pour documenter les fonctions, les modules, les classes et les méthodes dans le code. Elles sont généralement placées directement après la déclaration de la fonction, de la classe, du module ou de la méthode, entre **des guillemets triples**.

L'une des principales utilisations des docstrings est de fournir une description claire et concise de ce que fait une fonction, de ses paramètres et de sa valeur de retour. Cela aide les autres développeurs à comprendre rapidement comment utiliser la fonction sans avoir à examiner le code lui-même.

Voici un exemple de docstring pour une fonction de calcul de la somme de deux nombres :

```
def somme(a, b):
 """
 Cette fonction calcule la somme de deux nombres et retourne le résultat.

 Parameters:
 a (int): le premier nombre
 b (int): le deuxième nombre

 Returns:
 int: la somme de a et b
 """
 return a + b
```

Lorsque vous appelez cette fonction dans l'interpréteur Python, vous pouvez utiliser la fonction « **help()** » pour afficher la **docstring**, voici le résultat :

```
>>> help(somme)
somme(a, b)
 Cette fonction calcule la somme de deux nombres et retourne le
 résultat.
 Parameters:
 a (int): le premier nombre
 b (int): le deuxième nombre
 Returns:
 int: la somme de a et b
```

## Gérez les erreurs et les exceptions

Lorsqu'un programme rencontre une erreur, cela peut le faire planter complètement, ce qui n'est pas souhaitable. Pour éviter cela, il est possible d'utiliser les exceptions pour gérer les erreurs. Les exceptions sont des erreurs qui peuvent être détectées et gérées par le programme. Si une exception est levée, le programme peut afficher un message d'erreur, puis continuer à s'exécuter normalement.

Par exemple, si on demande à l'utilisateur de saisir un nombre, mais qu'il saisit un texte au lieu d'un nombre, cela peut causer une erreur. Pour éviter cela, on peut utiliser un bloc **try/except** pour gérer l'erreur, et afficher un message d'erreur personnalisé à l'utilisateur.

```
while True:
 try:
 x = int(input("Entrez un nombre entier : "))
 break
 except ValueError:
 print("Oops ! Ce n'est pas un nombre entier. Essayez encore...")
```

Dans ce code, le bloc **try** contient le code qui peut causer une erreur, c'est-à-dire la conversion de la saisie utilisateur en un entier. Si une erreur se produit, le bloc **except** est exécuté et affiche un message d'erreur personnalisé. Le programme revient ensuite au début de la boucle **while** et demande à l'utilisateur de saisir un nouveau nombre.

Si vous souhaitez approfondir vos connaissances sur la gestion des exceptions en Python, vous pouvez consulter la [page officielle de la documentation Python](#) dédiée à cette fonctionnalité.

## Respectez les standards du code

En général, vous lisez du code bien plus souvent que vous n'en écrivez. C'est pour ça que c'est important d'avoir des lignes directrices pour améliorer sa lisibilité et le rendre aussi cohérent que possible.

Le guide de style officiel pour le code Python est un document appelé [PEP 8](#) (ressource en anglais). Il contient des règles et des bonnes pratiques qui aident à standardiser l'écriture en Python par les développeurs. Globalement, les philosophies derrière l'écriture correcte en Python sont répertoriées dans un document appelé [PEP 20](#) (ressource en anglais) :

Il est vital de connaître ces préceptes du « **Zen de Python** », comme on les appelle. Je vous recommande aussi de consulter le **PEP** quand vous avez des questions qui surviennent dans vos projets de programmation en Python.

Même si les PEP sont des ressources en anglais, il existe aussi des ressources en français basées sur les PEP, comme la traduction française de [The Hitchhiker's Guide to Python](#).

# CHAPITRE IV : CALCUL NUMERIQUE ET VISUALISATION STATIQUE DES DONNEES

## Objectif du chapitre

Fournir une compréhension des bibliothèques Python essentielles pour les calculs numériques et la visualisation de données, en utilisant des applications pratiques pour illustrer les concepts fondamentaux.

## Objectifs spécifiques

- Comprendre les structures de données (tableaux et matrices) offertes par Numpy et appliquer des opérations mathématiques et statistiques sur des tableaux multidimensionnels.
- Apprendre à créer des graphiques et des visualisations de données en utilisant Matplotlib et tracer des fonctions mathématiques et des données expérimentales pour une analyse visuelle.
- Utiliser Numpy et Matplotlib pour modéliser et visualiser des concepts physiques, comme la loi d'Ohm et les trajectoires balistiques.
- Comprendre les méthodes numériques de résolution d'équations différentielles, telles que les méthodes d'Euler et de Runge-Kutta.
- Créer des programmes simulant des phénomènes physiques, tels que le mouvement harmonique simple et la chute d'un objet avec résistance de l'air.

## I. Introduction à NumPy

### I.1. Présentation de la bibliothèque

Les Data Analysts ou Data Scientists ont au quotidien à manipuler des données en grande quantité et de sources très variées. Celles-ci sont généralement représentées sous la forme de listes ou tableaux de nombres. Cependant, les objets natifs de Python sont assez limités pour les manipuler correctement. Considérons par exemple la liste suivante, représentant le revenu mensuel de cinq clients de notre banque :

```
revenus = [1800, 1500, 2200, 3000, 2172]
```

Si on souhaite calculer la moyenne, il n'y a pas de fonction existante par défaut permettant de le faire : il faudrait la recoder soi-même ! C'est encore plus compliqué si on souhaite avoir une information plus spécifique, comme la médiane. C'est là que la librairie **NumPy** intervient !

Diminutif de **Numerical Python**, elle fournit un ensemble de fonctions pour effectuer efficacement des opérations sur les données, mais propose également un objet qui est au cœur de presque tout l'écosystème data en Python : l'**array**. Je vous propose de découvrir un peu plus en détail cette librairie, et pourquoi elle est aussi réputée dans l'environnement data Python aujourd'hui !

## Installation

Pour utiliser NumPy dans Visual Studio Code (VS Code), il suffit d'ouvrir un terminal dans VS Code et exécutez la commande suivante :

```
pip install numpy
```

## 1.2. Différenciez les arrays des listes classiques

Le tableau **NumPy**, ou **array**, en anglais, est d'une certaine manière similaire à une liste Python. **Son avantage ?** Il permet d'effectuer simplement et rapidement de nombreuses opérations complexes, en plus de faciliter le stockage et la manipulation des données. Reprenons notre exemple de calcul de la moyenne présenté ci-dessus. Dans les faits, la fonction n'est pas très complexe. Elle pourrait s'écrire :

```
def moyenne(liste):
 return sum(liste)/len(liste)
print(moyenne(revenus)) # => 2134.4
```

Mais **NumPy** donne directement accès à une fonction existante. Dans un premier temps, importons la librairie **NumPy** :

```
import numpy as np
```

Calculons à présent la moyenne :

```
print(np.mean(revenus))
```

En effet, ici la révolution vient avec les **arrays NumPy** ! On estime qu'à mesure que la taille du tableau augmente, des opérations via **array NumPy** deviennent environ 30 fois plus rapides que via une liste Python classique. Cette différence aussi importante s'explique par le fait que les **arrays NumPy** ne peuvent contenir qu'un seul et même type. En effet, contrairement à une liste classique où l'on peut stocker tous types d'objets, **NumPy** n'acceptera qu'un seul type en entrée. De plus, **NumPy** donne également accès à de nombreuses autres fonctions mathématiques indispensables, applicables à des arrays ou même à des listes :

```
x = [-2, -1, 1, 2]
print("La valeur absolue: ", np.abs(x))
print("Exponentielle: ", np.exp(x))
print("Logarithme: ", np.log(np.abs(x)))
```

Il en existe encore de nombreuses autres, que je vous invite à découvrir dans la [documentation de NumPy](#). Je vous propose à présent de créer notre premier array.

## 1.2. Créez un array NumPy

Vous pouvez faire cela de différentes façons, mais la plus simple est de le faire... à partir d'une liste Python classique :

```
revenus_array = np.array([1800, 1500, 2200, 3000, 2172])
print(revenus_array) # [1800 1500 2200 3000 2172]
```

**Remarque :** Si dans la liste de départ il y a des données de types différents, **NumPy** essaiera de tous les convertir au type le plus général. Par exemple, si un tableau contient des entiers (int) et des nombres décimaux (float), tous ses éléments seront convertis en nombres décimaux (float).

Il existe également des fonctions NumPy permettant de créer des arrays selon un certain pattern ou une spécification particulière. Les plus couramment utilisées sont :

- **np.zeros(n)** : permet de créer un array rempli de 0, de n éléments ;
- **np.ones(n)** : similaire en tous points à la fonction présentée ci-dessus, à la différence que l'array sera rempli cette fois-ci de 1 ;
- **np.arange(i, j, p)** : permet de créer un array rempli avec une séquence linéaire, qui ira de i à j, par pas de p ;
- **np.linspace(i, j, n)** : permet de créer un array de n valeurs espacées uniformément entre i et j.

### Créez un array monotype

Comme nous avons pu le voir précédemment, un array ne peut contenir qu'un seul et unique type. Vous pouvez accéder au type des éléments d'un array via la méthode **.dtype** :

```
print(revenus_array.dtype) # int64
```

**Remarque :** Les types en NumPy seront toujours présentés via deux informations :

- le type (ici **int**) ;
- la précision (ici **64**).

Cette dernière correspond au nombre de bits sur lesquels sont codés les nombres, définissant ainsi les différentes valeurs possibles. Par exemple, un **int8** (entier codé sur 8 bits) pourra prendre des valeurs de -128 à 127 !

Pour plus d'informations sur les types NumPy, consultez la [page Data types de la documentation officielle](#).

Voyons à présent comment sélectionner des éléments au sein d'un **array NumPy**.

## 1.3. Sélectionnez des éléments au sein d'un array

Qu'en est-il de l'accès aux différents éléments d'un array ? Tout comme avec une liste classique, vous pouvez faire cela via l'indice de ces derniers.

### Accédez à un seul élément

L'accès à un élément spécifique d'un array se fait via la syntaxe : **nom\_array[indice]** :

```
pour accéder au 5ème élément
print(revenus_array[4])

pour accéder au dernier élément
print(revenus_array[-1])

On peut aussi modifier les valeurs
revenus_array[1] = 1900
```

### Accédez à plusieurs éléments contigus

Nous pouvons accéder à un ensemble d'éléments contigus en combinant **[ ]** et **:**. La syntaxe suit une règle simple : **nom\_array[i:j:p]**, avec :

- **i** : l'indice du début ;
- **j** : l'indice de fin ;
- **p** : le pas.

Permettant de sélectionner tous les éléments compris entre l'indice **i** inclus, jusqu'à **j-1**. Ainsi, l'élément d'indice **j** est forcément exclu. La ligne :

```
print(revenus_array[0:3]) # [1800 1500 2200]
```

Permet de sélectionner, au sein de notre array **revenus\_array**, l'ensemble des éléments de l'indice 0 (soit le premier élément) jusqu'à celui d'indice 2 (soit le troisième). L'élément d'indice 3 n'est pas inclus ! Si vous souhaitez également le sélectionner, il faudra ajouter 1 à l'indice de fin : **revenus\_array[0:4]**.

**Remarque :** Le début peut être omis si on veut commencer au début de la liste (c'est-à-dire si **début = 0**). La fin peut être omise si on veut aller jusqu'au bout de la liste (c'est-à-dire **fin = -1** ou **fin = len(liste)**). Le pas, ainsi que le dernier **:**, peuvent être omis si le pas est de **1** (**-1** si la fin est inférieure au début).

```
Les 3 premiers éléments
print(revenus_array[:3])

Les éléments à partir de l'indice 2
print(revenus_array[2:])

Un élément sur deux
print(revenus_array[::2])
```

Si le pas est négatif, le début et la fin sont inversés. On peut utiliser cette propriété pour inverser un tableau.

```
print(revenus_array[::-1]) # [2172 3000 2200 1500 1800]
```

### Accédez à plusieurs éléments selon une condition

On peut cependant souhaiter aller plus loin dans l'accès aux différents éléments. En effet, lors d'une étude de données on est régulièrement amené à devoir sélectionner une partie de nos données, selon un critère (ex. un genre spécifique) ou une condition (ex. toutes les personnes en dessous d'un certain âge). Les **arrays NumPy** permettent de faire cela ! En effet, de la même façon qu'il est possible de sélectionner des éléments via leur indice, il est possible avec les arrays NumPy de renseigner la condition selon laquelle on souhaite sélectionner les éléments du tableau, avec l'écriture **nom\_array[condition de sélection]**.

Voilà par exemple comment sélectionner uniquement les valeurs supérieures à 2 000 :

```
print(revenus_array[revenus_array > 2000]) # [2200 3000 2172]
```

Détaillons le fonctionnement de ce processus de sélection :

- Tout d'abord, on cite explicitement le nom de l'array avant les crochets, pour spécifier dans quel array on souhaite effectuer la sélection ;
- Ensuite, on spécifie à l'intérieur des crochets la condition. Le résultat de ce qui est à l'intérieur des crochets doit forcément être un array ou une liste de booléens ! Ici, si on exécute uniquement la condition **revenus\_array > 2000**, vous verrez que vous aurez simplement un array contenant uniquement des booléens, avec True lorsque la valeur satisfait la condition, False sinon.

**Attention :** On pourrait naïvement exécuter **revenus\_array[ > 2000]** en pensant que Python comprendra qu'on souhaite appliquer la condition à l'array qui est cité juste avant. Le résultat sera une **erreur**. Il faut bien considérer que l'array dans lequel on sélectionne, et la condition à appliquer, sont deux choses différentes. Même si cela concerne le même array, il est nécessaire de le spécifier également dans la condition !

Il est naturellement possible de complexifier cela, avec plusieurs conditions :

```
print(revenus_array[(revenus_array > 2000) & (revenus_array < 3000)])
[2200 2172]
```

**Remarque :** Le et logique (respectivement ou logique) n'est pas matérialisé par le mot clé "and" (respectivement "or") comme dans une structure conditionnelle if classique, mais par le symbole & (respectivement "|"). De plus, chaque condition doit être absolument délimitée par des parenthèses, sous peine de rencontrer une erreur. Cette écriture s'explique par le fait qu'on ne compare pas des booléens, mais des arrays de booléens, terme à terme.



Il est à noter que cette syntaxe ne permet pas uniquement de sélectionner des éléments, mais qu'elle peut également être utilisée pour modifier des éléments au sein d'un array. En effet, si vous exécutez :

- **revenus\_array[3] = 1790**, cela signifie que vous remplacez le 4<sup>e</sup> élément de notre array par 1 790 ;
- **revenus\_array[revenus\_array > 2000] = 0**, vous remplacerez tous les éléments supérieurs à 2 000 au sein de notre array par 0. Il faut donc être vigilant en utilisant cette syntaxe, pour ne pas remplacer des valeurs qu'il n'aurait pas été souhaitable de remplacer.

Je vous propose à présent de découvrir quelques-unes des nombreuses méthodes applicables à des arrays.

## I.4. Utilisez les méthodes d'array

Nous avons d'ores et déjà vu comment obtenir le type d'un array à partir de la méthode **.dtype**. De la même façon, on peut accéder facilement aux dimensions de notre array via la méthode **.shape** :

```
revenus_array.shape
(5,) on a array contenant 5 éléments
```

Le résultat nous indique qu'on a un array contenant **5 éléments**. La virgule « , » après le **5** indique que c'est un **tableau à une seule dimension**. En Python, les tuples sont utilisés pour représenter les dimensions d'un tableau. Un **tuple avec une seule valeur** nécessite une **virgule** pour être reconnu comme tel.

On peut également appliquer de nombreuses opérations mathématiques très simplement :

```
calculer la moyenne
revenus_array.mean()

calculer le maximum (ou le minimum) :
revenus_array.max()
revenus_array.min()

accéder à l'indice de l'élément minimum (ou maximum) :
revenus_array.argmin()
revenus_array.argmax()

ordonner par ordre croissant :
revenus_array.sort()
print(revenus_array)

calculer la somme :
revenus_array.sum()
```

Cette liste est naturellement non exhaustive ! Vous pouvez trouver la liste de l'ensemble des méthodes applicables à un array directement dans la [documentation officielle](#).

## I.5. Créez un tableau avec NumPy

Jusque-là, nous n'avons vu que des tableaux/arrays à une dimension, mais on ne travaille que très rarement avec une seule colonne.

Prenons un exemple : vous travaillez dans le milieu bancaire et vous avez besoin de créer un tableau où vous retrouveriez, en plus des revenus de vos clients, leurs mensualités de remboursement de prêt, le nombre d'enfants à charge, et toute autre information susceptible de vous être utile.

Considérons les clients suivants :

- Hugo, un jeune homme de 21 ans, gagnant 140 000 par mois et n'ayant aucun enfant ;
- Richard, un homme de 54 ans, gagnant 280 000 par mois et ayant 2 enfants ;
- Émilie, une femme de 27 ans gagnant 370 000 et ayant 3 enfants.

Nos données peuvent être représentées via ce tableau :

|         | Âge | Revenus (FCFA) | Nombre d'enfants |
|---------|-----|----------------|------------------|
| Hugo    | 21  | 140 000        | 0                |
| Richard | 54  | 280 000        | 2                |
| Émilie  | 27  | 370 000        | 3                |

Et voici comment pourrait se matérialiser ce tableau en Python, en utilisant des listes :

```
hugo = [21, 140000, 0]
richard = [54, 280000, 2]
emilie = [27, 370000, 3]
tableau = [hugo, richard, emilie]
```

Décortiquons cela ensemble :

- Dans un premier temps, nous créons une liste pour chaque personne dans notre banque, contenant l'ensemble des informations à disposition sur cette personne ;
- Enfin, nous créons une liste `tableau` dans laquelle nous stockons l'ensemble des différentes listes créées précédemment.

La liste **tableau** contient donc 3 éléments, qui sont eux-mêmes des listes de 3 éléments : c'est donc une **liste de listes**. Nous avons néanmoins toujours l'ensemble des limitations liées aux listes, avec cette façon de faire.

**Comment pourrions-nous faire alors ?** Vous vous en doutez très certainement, mais oui, encore une fois la réponse se trouve du côté de NumPy et des arrays ! En effet, un array est un objet **multidimensionnel**, c'est-à-dire qu'il est possible de créer des arrays de toutes dimensions, et que l'ensemble des méthodes d'array prennent en compte ce côté multidimensionnel. Génial, non ? Testons tout cela ensemble !

La façon la plus simple de créer un tableau est de le faire à partir d'une liste de listes Python, comme avec une liste classique. Il suffira d'exécuter **np.array(tableau)** pour transformer notre liste de listes en array NumPy de 3 lignes et 3 colonnes.

Voyons quelques autres exemples ensemble pour bien comprendre cette notion :

```
tab1 = np.array([[1, 2], [3, 4], [5, 6]])
tab2 = np.array([[1, 2, 3], [4, 5, 6]])
```

Ces quelques lignes de code permettent de créer :

- Un array **tab1** de 3 lignes et 2 colonnes ;
- Et un array **tab2** de 2 lignes et 3 colonnes.

Il est possible de créer des tableaux en bien plus que 2 dimensions. Par exemple, pour un tableau en 3D il suffira d'utiliser une liste de listes de listes.

Comme précédemment, nous pouvons également utiliser des fonctions NumPy pour créer des tableaux plus ou moins complexes. Voici quelques exemples de fonctions couramment utilisées :

```
un tableau de 3x5 rempli de 1
np.ones((3, 5))

un tableau de 4 lignes et de 4 colonnes contenant que des 0
np.zeros((4, 4))

un tableau de 6x3 rempli de valeurs aléatoires comprises entre 0 et 1
np.random.random((6, 3))

un tableau de 3x3 rempli de valeurs aléatoires entières, comprises entre 1 et 10
np.random.randint(1, 10, size=(3, 3))
```

N'hésitez pas à reprendre ces fonctions et à les exécuter dans votre propre environnement en changeant certaines valeurs, pour bien appréhender leur fonctionnement.

## I.6. Découvrez l'analogie entre les arrays et les matrices

Vous vous souvenez que j'ai évoqué le fait que NumPy était une librairie Python dessinée pour les calculs scientifiques ? Les tableaux que nous avons vus ensemble jusqu'ici, matérialisés par des arrays NumPy, portent le nom de matrices, en mathématiques. Le but premier de NumPy, avant même toutes ses applications dans le domaine de la data, est de proposer des outils pour manipuler ces matrices et effectuer ce qu'on appelle des calculs matriciels.

Une matrice est un tableau d'éléments, comme nous avons pu en voir jusqu'ici. On a l'habitude de la représenter sous la forme suivante :

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Les matrices sont une composante essentielle de l'**algèbre linéaire**. Vous allez forcément être amené à croiser à nouveau leur chemin si vous poursuivez dans l'analyse de données, car elles sont la base même de nombreuses méthodes statistiques de modélisation !!

Nous avons régulièrement à effectuer différentes opérations avec ces matrices. Considérons les 3 matrices suivantes, A, B et C :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad B = \begin{pmatrix} 5 & 10 \\ 15 & 20 \end{pmatrix} \quad C = \begin{pmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{pmatrix}$$

Comme elles sont de même dimension (même nombre de lignes et de colonnes), on peut calculer la somme (ou la différence) de la matrice A et de la matrice B, en additionnant terme à terme chaque élément :

$$A + B = \begin{pmatrix} 1+5 & 2+10 \\ 3+15 & 4+20 \end{pmatrix} = \begin{pmatrix} 6 & 12 \\ 18 & 24 \end{pmatrix}$$

La même chose avec NumPy :

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 10], [15, 20]])
C = np.array([[2, 4, 6], [8, 10, 12]])
print(A+B)
```

Il est également possible de faire une multiplication terme à terme via l'opérateur \*, comme avec une multiplication normale : **A \* B**. Cependant, lorsqu'on effectue des calculs matriciels, on utilise beaucoup plus ce qu'on appelle le **produit matriciel**. C'est un calcul assez fastidieux à faire à la main – d'autant plus avec des matrices de grande dimension – mais qu'un ordinateur n'a en revanche aucun problème à réaliser.

Avec NumPy, le produit matriciel se fait via l'opérateur @ : **A @ C**. Cependant, cet opérateur n'est disponible que depuis **Python 3.5**. Avant cela, le produit matriciel se faisait via la fonction **dot** de NumPy : **AC = np.dot(A, C)**. Cette dernière fonctionne toujours aujourd'hui, les deux syntaxes sont donc possibles !

## II. Introduction à SciPy

### II.1. Présentation de la bibliothèque

**SciPy** est une bibliothèque open-source pour le langage de programmation Python, utilisée principalement pour le calcul scientifique et technique. Elle s'appuie sur **NumPy** et fournit un ensemble de fonctions et d'outils qui facilitent le travail avec des données numériques.

**SciPy** offre généralement plus de fonctionnalités que **NumPy**, mais dans un sens spécifique.

- **NumPy** se concentre principalement sur la manipulation efficace de tableaux multidimensionnels et fournit des fonctions fondamentales pour le calcul numérique (opérations mathématiques, transformations, etc.).
- **SciPy**, en revanche, construit par-dessus NumPy et propose une large gamme de modules spécialisés pour le calcul scientifique, comme l'optimisation, l'intégration, la résolution d'équations différentielles, la statistique avancée, l'interpolation, et bien d'autres.

Ainsi, **SciPy** enrichit **NumPy** en fournissant des outils plus spécialisés et avancés pour des applications scientifiques et techniques.

### Installation

Pour utiliser **SciPy** dans Visual Studio Code (VS Code), il suffit d'ouvrir un terminal dans VS Code et exécutez la commande suivante :

```
pip install scipy
```

**NB** : **pip** est un gestionnaire de paquets pour Python. Il permet d'installer, de mettre à jour et de gérer des bibliothèques et des dépendances Python facilement.

### Utilisation

Pour importer la librairie **SciPy** dans votre environnement devez utiliser ceci :

```
import scipy
```

Nous verrons par la suite que la bibliothèque **SciPy** est composée d'un grand nombre de **sous-bibliothèques** spécifiques à certains problèmes.

De plus, **SciPy** se base sur l'utilisation de vecteurs et de matrices que l'on peut utiliser à l'aide de la bibliothèque **NumPy**. Il est donc intéressant d'importer également cette bibliothèque :

```
import numpy as np
```

## II.2. Algèbre linéaire avec SciPy

Il existe par exemple toute une partie liée à l'algèbre linéaire et à la résolution matricielle de problèmes mathématiques. Cette sous-bibliothèque s'appelle **linalg**. Pour l'importer, il suffit d'ajouter la ligne suivante :

```
from scipy import linalg
```

### Inverse d'une matrice

Mathématiquement, l'inverse d'une matrice  $A$  est la matrice  $B$  telle que  $\mathbf{AB} = \mathbf{I}$  où  $\mathbf{I}$  est la matrice d'identité. L'inverse est noté  $\mathbf{B} = \mathbf{A}^{-1}$ . Dans SciPy, cet inverse peut être obtenu en utilisant la méthode **inv** de la sous-bibliothèque **linalg** :

```
A = np.array([[2,0], [1,5]])
B = linalg.inv(A)
print(B)
```

## Déterminant d'une matrice

Pour calculer le déterminant d'une matrice carrée, on utilise la fonction **det()** comme l'exemple suivant :

```
A = np.array([[3,5], [2,4]])
d = linalg.det(A)
print(d) # affiche: 2.00000000000000018 ou 1.9999999999999996
```

## Autres fonctions mathématiques autour des matrices

Il existe d'autres opérations liées aux matrices, dans la sous-bibliothèque **linalg** de **SciPy** :

- **expm** : Exponentielle d'une matrice
- **logm** : Logarithme d'une matrice
- **sinm**, **cosm**, **tanm** : Fonctions trigonométriques sur des matrices
- **sqrtn** : Racine carrée

Il est à noter que NumPy est également une bibliothèque puissante pour les calculs numériques adapté pour l'algèbre lineaire mais il n'est pas spécifiquement conçu pour résoudre des **équations différentielles**.

## II.3. Equations différentielles avec SciPy

Une **équation différentielle** est une équation qui implique une ou plusieurs dérivées d'une fonction inconnue. En d'autres termes, elle relie une fonction à ses taux de changement.

$$\text{Exemple simple : } \frac{dy}{dt} = ky$$

C'est une équation différentielle du premier ordre, où  $y(t)$  est la fonction inconnue, et  $\frac{dy}{dt}$  est sa dérivée par rapport à  $t$ .

Elle permet ainsi, de modéliser des phénomènes qui évoluent dans le temps ou dans l'espace, comme la croissance des populations, la diffusion de la chaleur, ou la dynamique des systèmes mécaniques.

**SciPy** offre des outils puissants pour résoudre des équations différentielles ordinaires à l'aide de la fonction **solve\_ivp**. Pour l'importer, il suffit d'ajouter la ligne suivante :

```
from scipy.integrate import solve_ivp
```

**Exercice** : Voici un exemple pour résoudre l'équation différentielle  $\frac{dy}{dt} = -2y$  avec une condition initiale  $y(0) = 1$

```
import numpy as np
from scipy.integrate import solve_ivp

Définir l'équation différentielle
def equation(t, y):
 return -2 * y

Conditions initiales
y0 = [1] # Valeur initiale
t_span = (0, 5) # Intervalle de temps

Résoudre l'équation
solution = solve_ivp(equation, t_span, y0, t_eval=np.linspace(0, 5, 100))

Afficher la solution
for t, y in zip(solution.t, solution.y[0]):
 print(f"t = {t:.2f}, y = {y:.4f}")
```

Décortiquons un peu tout ça :

- **Définition de l'Équation Différentielle :**
  - **def equation(t, y) :** C'est une fonction qui représente l'équation différentielle.
  - **t :** Variable indépendante, généralement le temps.
  - **y :** Variable dépendante, représentant la solution de l'équation à un instant donné.
  - **return -2 \* y :** C'est l'expression de l'équation différentielle. Cela signifie que la dérivée de **y** par rapport à **t** est proportionnelle à **-2y**.
- **Conditions Initiales :**
  - **y0 = [1] :** C'est la condition initiale, indiquant que **y(0) = 1**. La solution commence à 1 lorsque **t = 0**.
  - **t\_span = (0, 5) :** C'est l'intervalle de temps sur lequel nous voulons résoudre l'équation, de **t = 0** à **t = 5**.
- **Résolution de l'Équation :**
  - **solution = solve\_ivp(...)** : Appelle la fonction de SciPy pour résoudre l'équation différentielle.
  - **equation** : La fonction définie précédemment.
  - **t\_span** : L'intervalle de temps.
  - **y0** : La condition initiale.
  - **t\_eval=np.linspace(0, 5, 100)** : Cette option génère 100 points de temps uniformément répartis entre 0 et 5, où la solution sera évaluée. Cela permet d'obtenir une courbe lisse lors du traçage.
- **Affichage de la Solution :**
  - **for t, y in zip(solution.t, solution.y[0])** : Boucle à travers les temps et les valeurs de la solution.
  - **solution.t** : Contient les points de temps où la solution a été évaluée.
  - **solution.y[0]** : Contient les valeurs de **y** correspondantes à chaque point de temps. **solution.y** est une matrice où chaque ligne correspond à une variable (ici, une seule variable).

- `print(f't = {t:.2f}, y = {y:.4f}')` : Imprime les valeurs de `t` et `y` avec un formatage précis. `:.2f` pour deux décimales et `:.4f` pour quatre décimales.

**Remarque :** La bibliothèque `matplotlib.pyplot` aurait pu éventuellement être utilisée pour tracer la courbe de cette équation.

### La fonction `zip`

La fonction `zip` en Python sert à combiner plusieurs itérables (comme des listes ou des tuples) en un seul itérable de tuples. Chaque tuple contient un élément de chaque itérable correspondant à la même position.

#### Exemple :

```
liste1 = [1, 2, 3]
liste2 = ['a', 'b', 'c']

resultat = zip(liste1, liste2)

Convertir en liste pour afficher
print(list(resultat)) # [(1, 'a'), (2, 'b'), (3, 'c')]
```

## III. Construisez des data visualisations avec Matplotlib

### III.1. Comprenez l'importance de la représentation graphique

Aujourd'hui, les entreprises du monde entier génèrent un volume de données extrêmement important : plusieurs dizaines de zettaoctets (milliards de téraoctets) selon les dernières estimations. Et ce chiffre ne fait qu'augmenter exponentiellement au fur et à mesure que les technologies de stockage évoluent.

Il devient donc indispensable pour une entreprise d'avoir des outils pour analyser ces données, afin d'en tirer des enseignements exploitables au niveau business (des tendances d'achat selon les périodes, par exemple) et plus globalement, de pouvoir **surveiller les différentes activités de l'entreprise** afin d'identifier d'éventuels problèmes.

Toutes les librairies vues jusqu'à présent font partie de ces outils, mais demandent une certaine technicité, au moins en programmation. Or, la plupart des personnes en charge de la stratégie au sein d'une entreprise (ceux qu'on appelle les **décideurs**) n'ont généralement pas ce bagage.

Ainsi, il faut donc un outil permettant de rendre la donnée accessible à ces personnes. C'est ce qu'on appelle la **data visualisation**, ou **dataviz** pour les intimes. La **data visualisation** est un ensemble de techniques utilisées pour communiquer des informations clés sur un gros volume de données, afin d'en faciliter la compréhension et la lecture pour tous. Ce domaine s'est vraiment



démocratisé en entreprise dans les années 80, lorsque les entreprises ont pris conscience que l'analyse de leurs données pourrait aider grandement à la prise de décision !

Lorsqu'il est question de représentation ou de graphique, de nombreux choix s'offrent à nous. On peut citer de façon non exhaustive :

- Les courbes pour représenter une évolution d'une caractéristique (comme la démographie d'un pays) dans le temps ;
- Les graphiques circulaires (appelés plus vulgairement des graphiques camemberts) pour représenter une répartition ;
- Ou encore des cartes lorsqu'on souhaite afficher une information géographique.

Il en existe bien d'autres...

## III.2. Présentation de la bibliothèque

À ce stade, nous avons réuni de nombreuses informations sur nos clients, que nous pourrions mixer et représenter au sein de différents graphiques.

Par exemple, plutôt que d'avoir l'information sous forme de tableau, il serait intéressant de représenter le chiffre d'affaires total par agence. On peut aussi regarder le taux d'endettement de nos clients en fonction de leur revenu, pour voir si une tendance se dégage.

Il nous faut donc une librairie pour réaliser ces différents graphiques avec Python. Il en existe une multitude, et c'est parfois un peu compliqué de comprendre l'intérêt et la plus-value de l'une par rapport à une autre. C'est pourquoi je vous propose de regarder en détail la librairie principalement utilisée pour réaliser des visualisations : **Matplotlib**. Enfin, nous allons plutôt utiliser **pyplot** qui est inclus dans **Matplotlib**.

Voilà comment installer cette dernière dans Visual Studio Code (VS Code) depuis le terminal :

```
pip install matplotlib
```

Et voilà comment importer cette dernière :

```
import matplotlib.pyplot as plt
```

Chaque représentation graphique a une fonction correspondante avec Matplotlib :

- Nuage de points ou scatter plot, en anglais : **scatter()** ;
- Diagrammes en ligne ou en courbes : **plot()** ;
- Diagrammes en barres : **bar()** ;
- Histogrammes : **hist()** ;
- Diagrammes circulaires : **pie()**.

## III.3. Tracez vos premiers graphiques

### III.3.1. Le nuage de points

Cherchons à représenter un graphique présentant le taux d'endettement en fonction du revenu. Ces deux variables sont numériques, sans évolution dans le temps : nous allons donc tracer un nuage de points, via la fonction **scatter**. Cette fonction nécessite de définir **x** et **y**, qui sont les valeurs à placer en abscisse et en ordonnée :

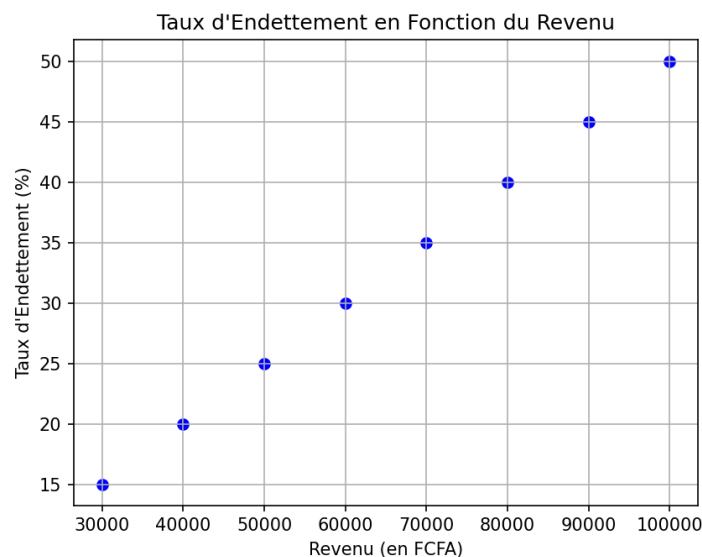
```
import matplotlib.pyplot as plt

Étape 1 : Préparer les Données
revenu = np.array([30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000])
taux_endettement = np.array([15, 20, 25, 30, 35, 40, 45, 50])

Étape 2 : Tracer le Nuage de Points
plt.scatter(revenu, taux_endettement, color='blue', marker='o')

Étape 3 : Ajouter des Étiquettes et un Titre
plt.title("Taux d'Endettement en Fonction du Revenu")
plt.xlabel("Revenu (en FCFA)")
plt.ylabel("Taux d'Endettement (%)")

Étape 4 : Afficher le graphique
plt.grid()
plt.show()
```



Il existe de nombreuses options pour personnaliser un nuage de points. On peut modifier :

- La couleur des points, en utilisant l'argument **color** ou **c** ;
- La taille des points, via l'argument **size** ou **s** ;
- Le type de marqueur via l'argument **marker**. Voici quelques valeurs courantes que vous pouvez utiliser :
  - 'o' : Cercle (par défaut)

- 's' : Carré
  - '^' : Triangle (pointé vers le haut)
  - 'v' : Triangle (pointé vers le bas)
  - '<' : Triangle (pointé vers la gauche)
  - '>' : Triangle (pointé vers la droite)
  - 'D' : Losange
  - 'd' : Losange plus petit
  - 'x' : Croix
  - '+' : Plus
  - '|' : Barre verticale
  - '\_' : Barre horizontale
- La transparence des points via l'argument **alpha**. Il prend une valeur entre 0 et 1.
  - La fonction **grid()** est utilisée dans Matplotlib pour afficher une grille sur le graphique. La grille aide à visualiser les valeurs des points sur les axes. Elle permet de mieux interpréter les données en facilitant la comparaison des valeurs.

Cette liste n'est qu'une infime partie de ce que vous pouvez modifier... Vous trouverez la liste complète des arguments sur la [documentation officielle de la fonction](#).

On souhaite maintenant **représenter le chiffre d'affaires total par agence**. Il existe deux possibilités pour cela : soit un **diagramme à barres**, soit un **diagramme circulaire**. Nous allons réaliser ces deux visualisations.

### III.3.2. Le diagramme circulaire

Là où nous autres francophones et amateurs de fromages voyons dans le diagramme circulaire un camembert, les anglo-saxons y voient plutôt une tarte (pie, en anglais) à découper en plusieurs parts ! C'est de là que provient le nom de la fonction permettant de tracer un diagramme circulaire : **pie**. Son utilisation est très similaire à celle de **scatter**. Il y a deux arguments à préciser :

- **x** : Cet argument représente les valeurs qui déterminent la taille de chaque secteur dans le diagramme circulaire. Il s'agit d'une liste ou d'un tableau contenant des nombres, où chaque nombre correspond à une portion du total. Il détermine la proportion de chaque secteur par rapport au total. Par exemple, si vous avez des chiffres d'affaires par agence à représenter, chaque valeur de **x** influencera la taille de l'agence correspondant dans le diagramme.
- **labels** : Cet argument fournit les étiquettes qui seront affichées pour chaque secteur du diagramme circulaire. Il s'agit d'une liste de chaînes de caractères, où chaque chaîne correspond à **une catégorie associée** à une valeur dans **x**. Il permet d'identifier chaque secteur avec un nom ou une description, facilitant ainsi la compréhension des données représentées.

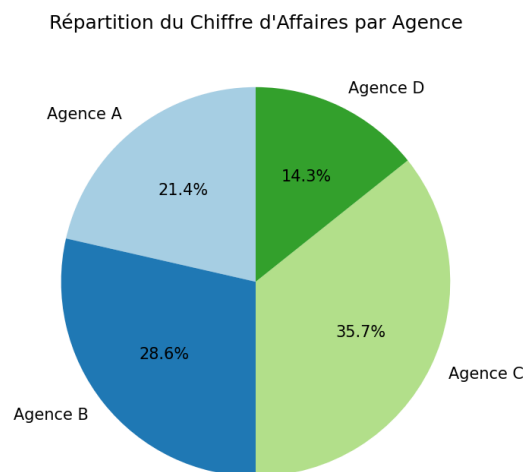
Représentation du chiffre d'affaires total par agence à l'aide du diagramme circulaire :

```
import matplotlib.pyplot as plt

Étape 1 : Préparer les Données
agences = ['Agence A', 'Agence B', 'Agence C', 'Agence D']
chiffre_affaires = [150000, 200000, 250000, 100000]

Étape 2 : Créer le Diagramme Circulaire
plt.pie(x=chiffre_affaires, labels=agences, autopct='%.1f%%', startangle=90,
 colors=plt.cm.Paired.colors)
plt.title("Répartition du Chiffre d'Affaires par Agence")

Étape 3 : Afficher les graphiques
plt.axis('equal') # Assure que le diagramme est un cercle
plt.tight_layout() # Ajuste la mise en page pour éviter le chevauchement
plt.show()
```



Il existe de nombreuses options pour personnaliser ce diagramme :

- L'argument **autopct** : permet d'afficher textuellement le pourcentage associé à chaque « part ». (dans notre cas un chiffre après la virgule)
- L'argument **startangle** : Cet argument détermine l'angle de départ pour le premier secteur du diagramme circulaire. Il peut être spécifié en degrés. En modifiant cet angle, vous pouvez faire pivoter le diagramme circulaire. Par défaut, le premier secteur commence à 0 degré (en haut).
- L'argument **colors** : Cet argument permet de spécifier les couleurs utilisées pour chaque secteur du diagramme circulaire. Vous pouvez passer une liste de couleurs, où chaque couleur correspond à un secteur. Cela permet de personnaliser l'apparence du diagramme pour le rendre plus attrayant ou pour mieux distinguer les différentes catégories. Par exemple, vous pouvez utiliser **colors=['red', 'blue', 'green']** pour définir les couleurs des secteurs.
- La fonction **axis()** dans Matplotlib permet de manipuler les axes d'un graphique. Voici ses principales utilisations :
  - Définir les Limites des Axes : **plt.axis([xmin, xmax, ymin, ymax])** : permet de spécifier explicitement les limites des axes x et y. Il est utile pour zoomer sur une région d'intérêt ou pour standardiser les échelles entre plusieurs graphiques.

- Ajustement Automatique des Limites : **`plt.axis('auto')`** : permet à Matplotlib de déterminer automatiquement les limites des axes en fonction des données. Il est pratique pour des visualisations rapides sans avoir à définir manuellement les limites.
  - Égalité des Axes : **`plt.axis('equal')`** : assure que les unités sont égales sur les axes x et y. Il évite la distorsion des formes géométriques, comme les cercles. par exemple, dans un cercle qui apparaîtrait comme une ellipse si les axes ne sont pas égaux
  - Mode de Visualisation : **`plt.axis('off')`** ou **`plt.axis('on')`** : permet de masquer ou d'afficher les axes. Il est utile pour des graphiques esthétiques ou des présentations où les axes ne sont pas nécessaires.
- La fonction **`tight_layout()`** dans Matplotlib est utilisée pour optimiser la disposition des sous-graphiques dans une figure. Elle ajuste automatiquement les paramètres de la disposition pour minimiser les chevauchements d'éléments, comme les axes, les titres et les étiquettes. Cela garantit que tous les éléments du graphique sont clairement visibles et bien espacés.

### III.3.3. Le diagramme à barres

L'alternative au diagramme circulaire est un diagramme à barres ! On peut représenter exactement la même information, mais avec une perspective différente. Pour utiliser la fonction **`bar`**, qui est l'implémentation de Matplotlib des diagrammes à barres, il faut préciser deux arguments :

- **`x`** : Cet argument spécifie les positions des barres sur l'axe des x. Il peut s'agir d'une liste ou d'un tableau contenant les catégories (par exemple, des noms d'agences). Détermine où chaque barre sera placée sur l'axe des x. Par exemple, si vous passez une liste de noms d'agences, chaque barre sera alignée sous le nom correspondant.
- **`height`** : Cet argument détermine la hauteur des barres. Il s'agit d'une liste ou d'un tableau contenant des valeurs numériques (par exemple, des chiffres d'affaires). Définit à quelle hauteur chaque barre s'étendra sur l'axe des y. Plus la valeur est élevée, plus la barre sera haute.

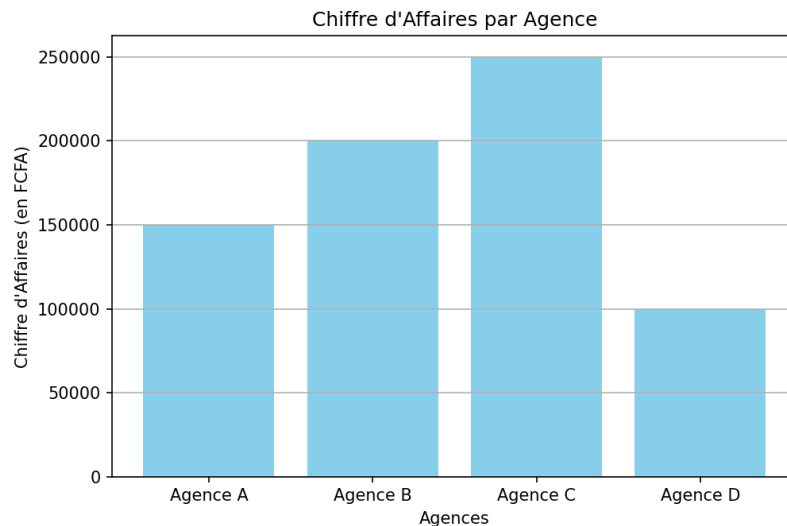
Représentation du chiffre d'affaires total par agence à l'aide du diagramme à barres :

```
import matplotlib.pyplot as plt

Étape 1 : Préparer les Données
agences = ['Agence A', 'Agence B', 'Agence C', 'Agence D']
chiffre_affaires = [150000, 200000, 250000, 100000]

Étape 2 : Créer le Diagramme à Barres
plt.bar(x=agences, height=chiffre_affaires, color='skyblue')
plt.title("Chiffre d'Affaires par Agence")
plt.xlabel("Agences")
plt.ylabel("Chiffre d'Affaires (en FCFA)")
plt.grid(axis='y')

Étape 3 : Afficher les graphiques
plt.show()
```



### III.3.4. L'histogramme

Lorsqu'il s'agit de connaissance client dans le milieu bancaire, on arrive assez rapidement à la question : Comment se répartissent nos clients en termes de revenus ? A-t-on majoritairement des classes moyennes ? ou des revenus plus modestes ?

L'histogramme est particulièrement utile lorsqu'on souhaite avoir une idée de la distribution d'une variable, et donc particulièrement adéquat pour répondre à cette problématique. La fonction Matplotlib correspondante est **hist**.

Représentation de la répartition des clients d'une banque en termes de revenus :

```
import matplotlib.pyplot as plt
import numpy as np

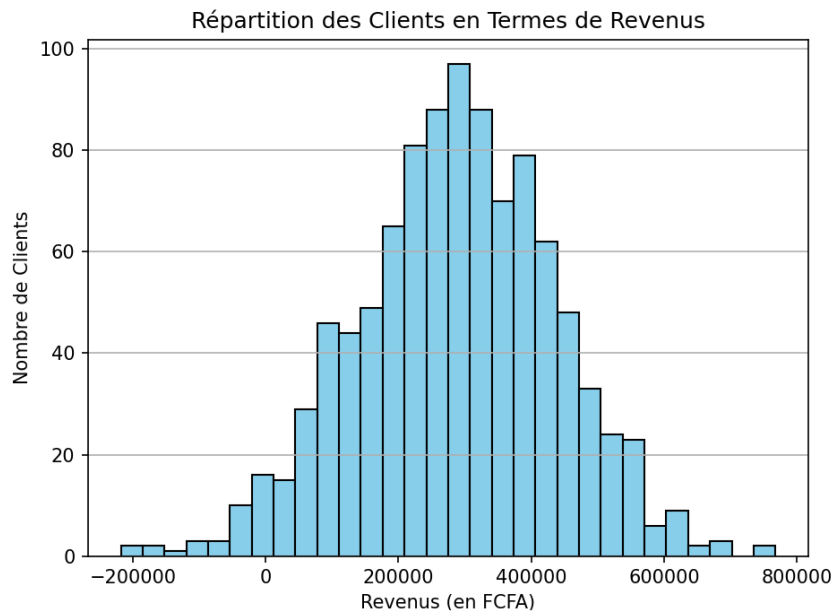
Générer des revenus aléatoires pour 1000 clients
avec une moyenne de 300000 FCFA et un écart-type de 150000 FCFA
revenus = np.random.normal(loc=300000, scale=150000, size=1000)

Créer l'histogramme
plt.hist(revenus, bins=30, color='skyblue', edgecolor='black')

Ajouter des titres et des étiquettes
plt.title("Répartition des Clients en Termes de Revenus")
plt.xlabel("Revenus (en FCFA)")
plt.ylabel("Nombre de Clients")

Afficher la grille
plt.grid(axis='y')

Afficher le graphique
plt.tight_layout()
plt.show()
```



- La fonction **np.random.normal()** de NumPy est utilisée pour générer des nombres aléatoires suivant une distribution normale (ou gaussienne) :
  - **Moyenne (loc)** : La valeur autour de laquelle les données sont centrées.
  - **Écart-type (scale)** : Mesure de la dispersion des données autour de la moyenne.
  - **Taille (size)** : Nombre de valeurs à générer.

Cette fonction est utile pour créer des ensembles de données réalistes dans des scénarios de simulation.

- Le paramètre **bins** dans la fonction **plt.hist()** détermine le nombre de classes ou d'intervalles dans lesquels les données seront regroupées. Il spécifie combien de segments (ou classes) vous souhaitez créer pour répartir vos données. Une valeur de **bins** plus élevée signifie plus de détails, tandis qu'une valeur plus basse entraîne une vue plus générale.
- Le paramètre **edgecolor** dans Matplotlib est utilisé pour définir la couleur des bordures des éléments graphiques, tels que les barres d'un histogramme ou les points d'un nuage de points.

### III.3.5. Les courbes

On souhaite suivre l'évolution du chiffre d'affaires sur les 6 premiers mois, pour voir la façon dont il varie, et éventuellement anticiper la suite. Nous avons pour cela à notre disposition le chiffre d'affaires réalisé par notre banque de Janvier à Juin 2025.

On a ici, une évolution du chiffre d'affaires dans le temps, donc le choix de représentation le plus pertinent serait de tracer une **courbe**. Pour ce faire, on utilise la fonction **plot** de Matplotlib. Celle-ci prend deux arguments en entrée : les informations à mettre en abscisse, et celles à mettre en ordonnée.

Voilà comment tracer l'évolution de notre chiffre d'affaires :

```
import matplotlib.pyplot as plt

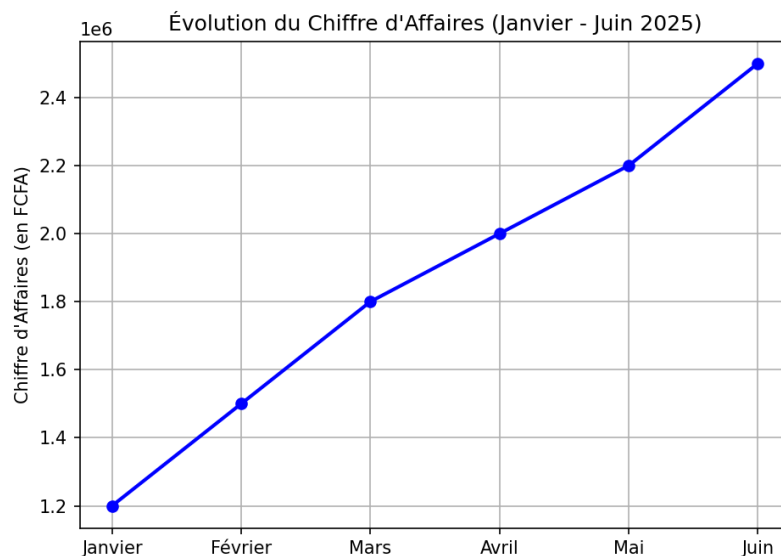
Données de chiffre d'affaires (en FCFA)
mois = ['Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin']
chiffre_affaires = [1200000, 1500000, 1800000, 2000000, 2200000, 2500000]

Créer la courbe
plt.plot(mois, chiffre_affaires, marker='o', color='blue', linestyle='-',
 linewidth=2)

Ajouter des titres et des étiquettes
plt.title("Évolution du Chiffre d'Affaires (Janvier - Juin 2025)")
plt.xlabel("Mois")
plt.ylabel("Chiffre d'Affaires (en FCFA)")

Afficher la grille
plt.grid()

Afficher le graphique
plt.tight_layout()
plt.show()
```



- Le paramètre **linestyle** ou **ls** : permet de définir le style de la ligne tracée sur le graphique. Il permet de choisir un style de ligne approprié peut aider à distinguer différents ensembles de données ou à améliorer la clarté du graphique. Options :
  - '-' : Ligne continue.
  - '--' : Ligne pointillée.
  - '-.' : Ligne mixte (pointillé et continu).
  - ':' : Ligne en pointillés.
- Le paramètre **linewidth** ou **lw** : permet de définir l'épaisseur de la ligne tracée. Il prend généralement un nombre positif comme valeur (par exemple, 1, 2, etc.).



### III.3.6. Créez plusieurs graphiques sur une même fenêtre

Maintenant, le directeur général de l'entreprise souhaite suivre à travers un graphique unique les statistiques suivantes :

- Le taux d'endettement en fonction du revenu (nuage de points)
- Le chiffre d'affaires total par agence (diagramme à barres)
- La répartition des clients en termes de revenus (histogrammes)
- L'évolution du chiffre d'affaires sur les 6 premiers mois (courbes)

Pour créer ce graphique unique qui intègre toutes ces statistiques, vous pouvez utiliser la notion des sous-graphiques (subplots) depuis Matplotlib. Voici comment structurer cela :

```
import matplotlib.pyplot as plt
import numpy as np

Données fictives
1. Taux d'endettement en fonction du revenu
revenu = np.array([30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000])
taux_endettement = np.array([15, 20, 25, 30, 35, 40, 45, 50])

2. Chiffre d'affaires total par agence
agences = ['Agence A', 'Agence B', 'Agence C']
chiffre_affaires_agences = [1500000, 2000000, 1800000]

3. Répartition des clients en termes de revenus
labels = ['<20k', '20k-50k', '50k-80k', '>80k']
sizes = [15, 45, 25, 15]

4. Évolution du chiffre d'affaires sur les 6 premiers mois
mois = ['Jan', 'Fév', 'Mar', 'Avr', 'Mai', 'Juin']
chiffre_affaires = [1200000, 1500000, 1800000, 2000000, 2200000, 2500000]

Création de la figure et des sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

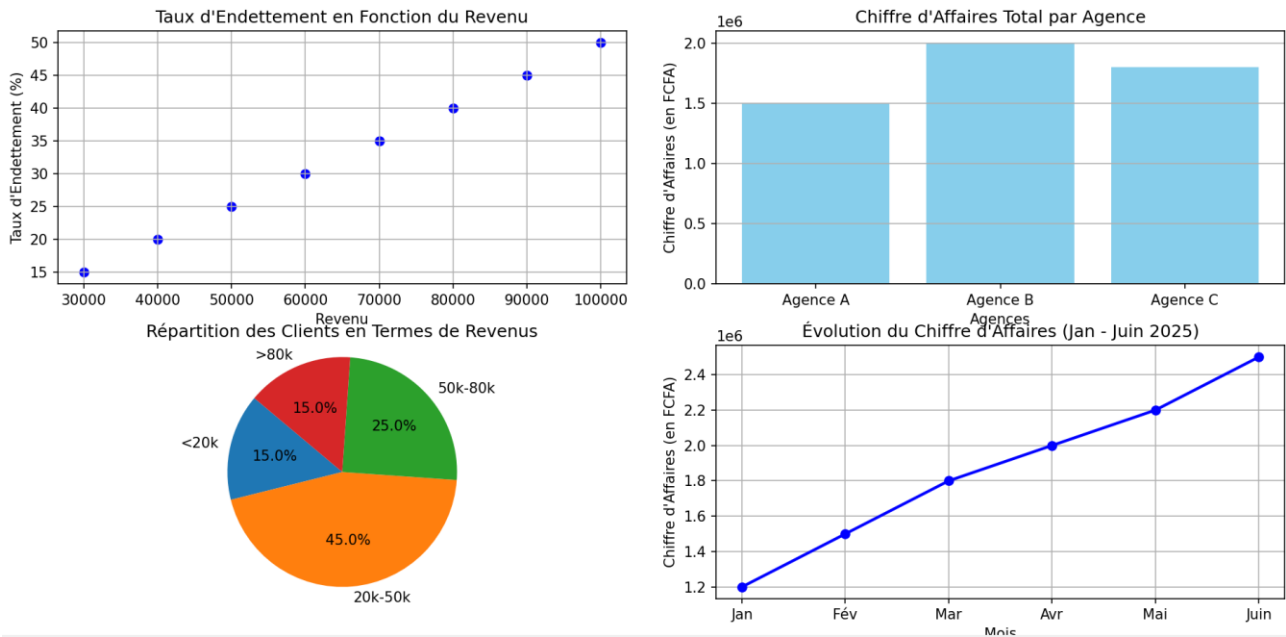
1. Nuage de points
axs[0, 0].scatter(revenu, taux_endettement, color='blue')
axs[0, 0].set_title("Taux d'Endettement en Fonction du Revenu")
axs[0, 0].set_xlabel("Revenu")
axs[0, 0].set_ylabel("Taux d'Endettement (%)")
axs[0, 0].grid()

2. Diagramme à barres
axs[0, 1].bar(agences, chiffre_affaires_agences, color='skyblue')
axs[0, 1].set_title("Chiffre d'Affaires Total par Agence")
axs[0, 1].set_xlabel("Agences")
axs[0, 1].set_ylabel("Chiffre d'Affaires (en FCFA)")
axs[0, 1].grid(axis='y')

3. Histogrammes
axs[1, 0].pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
axs[1, 0].set_title("Répartition des Clients en Termes de Revenus")
axs[1, 0].axis('equal') # Pour un cercle

4. Évolution du chiffre d'affaires
axs[1, 1].plot(mois, chiffre_affaires, marker='o', color='blue', linestyle='-', linewidth=2)
axs[1, 1].set_title("Évolution du Chiffre d'Affaires (Jan - Juin 2025)")
axs[1, 1].set_xlabel("Mois")
axs[1, 1].set_ylabel("Chiffre d'Affaires (en FCFA)")
axs[1, 1].grid()

Ajustement des espaces
plt.tight_layout()
plt.show()
```



Décortiquons un peu cette ligne : **fig, axs = plt.subplots(2, 2, figsize=(12, 10))**

La fonction **plt.subplots()** de Matplotlib est utilisée pour créer une figure et un ensemble de sous-graphiques (subplots) dans une seule commande. Voici une décomposition de ses composants :

- **fig** : C'est l'objet de la figure principale qui contient tous les sous-graphiques. Vous pouvez utiliser **fig** pour ajuster des propriétés générales de la figure, comme les titres, les tailles, ou pour enregistrer l'image.
- **axs** : C'est un tableau d'axes (subplots) qui permet d'accéder à chaque sous-graphe individuellement. Vous pouvez utiliser **axs[i, j]** pour cibler un sous-graphe spécifique, où **i** est l'indice de la ligne et **j** celui de la colonne.
- **(2, 2)** : Indique que vous souhaitez créer une grille de **2 lignes** et **2 colonnes** de sous-graphiques. Cela permet d'organiser plusieurs visualisations dans un espace compact et lisible.
- **figsize=(12, 10)** : Définit la taille de la figure en pouces, avec une largeur de **12 pouces** et une hauteur de **10 pouces**. Une taille appropriée garantit que les éléments graphiques sont bien espacés et lisibles.

# CHAPITRE V : ANALYSE ET VISUALISATION DYNAMIQUES DES DONNEES

## Objectif du chapitre

Développer des compétences en analyse de données physiques et chimiques à l'aide de Python, en intégrant des outils de visualisation avancée pour modéliser et interpréter des phénomènes scientifiques.

## Objectifs spécifiques

- Apprendre à utiliser la bibliothèque Pandas pour la gestion et l'analyse de données expérimentales : Introduction à la manipulation de données, importation, nettoyage et prétraitement des données.
- Explorer des outils avancés pour visualiser des données complexes en 3D : Introduction à Plotly et Mayavi pour la création de visualisations 3D interactives.
- Utilisation de la modélisation 3D pour représenter des structures et des systèmes complexes : Créer une visualisation 3D d'un problème d'ingénierie, mettant en avant l'application des concepts appris.

## I. Analyse des données avec Pandas

### I.1. Présentation de la bibliothèque

**Pandas** est une bibliothèque open-source en Python, principalement utilisée pour la manipulation et l'analyse de données. Elle fournit des structures de données flexibles et faciles à utiliser, permettant de travailler avec des données structurées (tableaux) de manière intuitive.

#### Caractéristiques Principales

- **Structures de Données :**
  - **DataFrame** : Une structure de données bidimensionnelle, semblable à une table SQL ou une feuille de calcul Excel, avec des lignes et des colonnes étiquetées.
  - **Series** : Une structure de données unidimensionnelle, équivalente à une colonne d'un DataFrame.
- **Manipulation de Données :**
  - **Chargement de Données** : Importation de données à partir de diverses sources (CSV, Excel, SQL, etc.).
  - **Indexation et Sélection** : Accès facile aux données via des étiquettes ou des positions.

- **Nettoyage des Données** : Traitement des valeurs manquantes, suppression des doublons, etc.
- **Analyse de Données** :
  - **Statistiques** : Calcul des moyennes, médianes, écarts-types et autres statistiques descriptives.
  - **Groupeement et Agrégation** : Regroupement de données pour effectuer des calculs sur des sous-ensembles.
  - **Fusion et Jointure** : Combinaison de différents DataFrames pour enrichir les données.
- **Visualisation** :
  - Intégration avec des bibliothèques de visualisation comme Matplotlib pour tracer des graphiques directement à partir des DataFrames.

## Installation

Pour utiliser **Pandas** dans Visual Studio Code (VS Code), il suffit d'ouvrir un terminal dans VS Code et exécutez la commande suivante :

```
pip install pandas
```

## Utilisation

Voici un exemple simple de création et de manipulation d'un DataFrame :

```
import pandas as pd

Création d'un DataFrame
data = {
 'Nom': ['TEJIOGNI', 'TOUKO', 'KAMGA'],
 'Âge': [25, 30, 35],
 'Ville': ['Douala', 'Yaoundé', 'Bafoussam']
}
df = pd.DataFrame(data)

Affichage du DataFrame
print(df)

Calcul de la moyenne d'âge
moyenne_age = df['Âge'].mean()
print(f"Moyenne d'âge : {moyenne_age}")
```

Résultat :

|                      | Nom      | Âge | Ville     |
|----------------------|----------|-----|-----------|
| 0                    | TEJIOGNI | 25  | Douala    |
| 1                    | TOUKO    | 30  | Yaoundé   |
| 2                    | KAMGA    | 35  | Bafoussam |
| Moyenne d'âge : 30.0 |          |     |           |

## I.2. Créez votre premier data frame avec Pandas

### I.2.1. Préparez votre jeu de données

Comme nous avons pu le voir, les arrays NumPy sont particulièrement efficaces pour traiter des valeurs numériques. Mais les données, dans la réalité, ne sont pas composées uniquement de chiffres et de nombres. En effet, on retrouve aussi :

- Des catégories ;
- Des labels ;
- Des dates ;
- Du texte brut.

De plus, ces données ont généralement un format prédéfini en analyse de données, où chaque ligne va correspondre à un individu (au sens statistique du terme), et chaque colonne va être une caractéristique spécifique des individus. C'est ce que l'on appelle une **variable**.

Voici quelques exemples pour illustrer cela :

- Dans le milieu automobile, chaque individu sera une voiture, et on pourra avoir comme caractéristiques la puissance du moteur, les dimensions du véhicule, la marque, le modèle, la couleur, etc. ;
- Dans une étude de grande distribution, chaque individu pourra être un produit sur lequel on aurait plusieurs informations (le prix, la catégorie, etc.) ;
- Dans le milieu bancaire enfin, chaque individu sera une personne, sur laquelle on aurait enregistré le salaire moyen, le genre, ses mensualités de remboursement de prêt, etc. ;

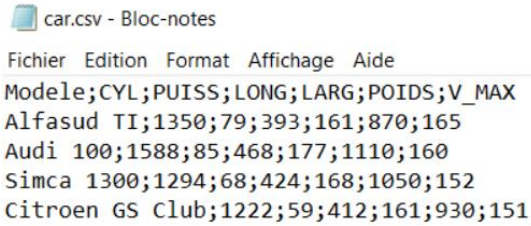
Voilà par exemple à quoi pourrait ressembler ce dernier cas :

|   | identifiant | ville     | CP    | revenu | remboursement | duree | type       | taux  | email                     | nom              | genre |
|---|-------------|-----------|-------|--------|---------------|-------|------------|-------|---------------------------|------------------|-------|
| 0 | 0           | TOULOUSE  | 31100 | 3669.0 | 1130.05       | 240   | immobilier | 1.168 | LaurentDagenais@rhyta.com | Laurent Dagenais | M     |
| 1 | 1           | PARIS     | 75009 | 5310.0 | 240.00        | 64    | automobile | 3.701 | GuyMarois@fleckens.hu     | Guy Marois       | M     |
| 2 | 1           | PARIS     | 75009 | 5310.0 | 1247.85       | 300   | immobilier | 1.173 | GuyMarois@fleckens.hu     | Guy Marois       | M     |
| 3 | 2           | MARSEILLE | 13010 | 1873.0 | 552.54        | 240   | immobilier | 0.972 | BeaufortLesage@einrot.com | Beaufort Lesage  | M     |
| 4 | 3           | MARSEILLE | 13010 | 1684.0 | 586.03        | 180   | immobilier | 1.014 | RussellDurand@armyspy.com | Russell Durand   | M     |

En explicitant le format des données, vous avez peut-être eu l'image d'un fichier **Excel**, et c'est une bonne idée ! Excel est encore pour beaucoup d'entreprises un format très utilisé pour stocker et déplacer des données. Mais ce n'est pas le seul, les données peuvent être stockées sous bien des formats différents.

Par exemple, on retrouve régulièrement des **fichiers texte** ou des **fichiers CSV** (pour **comma-separated values**). Ce sont simplement des fichiers contenant l'ensemble des données brutes, séparées par un **délimiteur**.

Voici un exemple avec un jeu de données automobiles, dont le délimiteur est le **point-virgule** :



```
car.csv - Bloc-notes
Fichier Edition Format Affichage Aide
Modele;CYL;PUISS;LONG;LARG;POIDS;V_MAX
Alfasud TI;1350;79;393;161;870;165
Audi 100;1588;85;468;177;1110;160
Simca 1300;1294;68;424;168;1050;152
Citroen GS Club;1222;59;412;161;930;151
```

Cela peut également être sous la forme de **fichier JSON**. Le **JavaScript Object Notation (JSON)** est un format standard utilisé pour représenter des données structurées. Cela ressemble à un gros dictionnaire Python pouvant contenir lui-même d'autres dictionnaires et/ou listes. C'est une description un peu "réductrice", mais une image vaut mieux que mille mots :



```
indiv.json - Bloc-notes
Fichier Edition Format Affichage Aide
{
 "prenom": "Benjamin",
 "nom": "Marlé",
 "adresse": {
 "rue": "quai neuf trois quart",
 "ville": "Paris",
 "code postal": "75001",
 "pays": "France"
 },
 "mails": [
 "benjamin.marle@openclassrooms.fr",
 "benjamin.marle@yahoo.fr"
]
}
```

Nous n'avons ici vu que les formats texte, JSON et CSV, mais la liste n'est naturellement pas exhaustive. De plus, plutôt que de stocker les données dans des fichiers, on a plutôt l'habitude de le faire dans des **bases de données SQL**.

Nous cherchons donc un outil qui nous permette de représenter les données avec le format souhaité (individus/variables), de manipuler différents types de données, et de lire les données provenant de différentes sources. Cet outil n'est autre que la librairie **Pandas**, et plus particulièrement les objets **data frame**.

Le **data frame** est un objet Python permettant de représenter les données sous forme de tableau, où chaque colonne est explicitement nommée. Il reprend les mêmes paradigmes que l'array NumPy : chaque colonne peut naturellement être d'un type différent, mais une colonne ne peut contenir qu'un seul type ! Cette organisation simplifie l'accès aux variables, et permet de nombreuses manipulations de données plus ou moins complexes.

On pourrait se poser la question : *quel est l'intérêt de la librairie NumPy si Pandas permet de faire tout cela ?* Excellente question ! Vous verrez au fur et à mesure de ce cours, et plus globalement de vos futures analyses de données, que vous serez régulièrement amené à jongler entre les deux. Il est assez courant que certaines méthodes appliquées à des data frames retournent des arrays, et qu'on ait besoin de retransformer ces arrays en data frames à posteriori. C'est pourquoi il est nécessaire de maîtriser les deux librairies, pour être parfaitement armé avant de plonger dans une analyse de données.

## 1.2.2. Générez votre premier data frame depuis des jeux de données au format .csv, .json, .xlsx

Nous allons créer notre premier data frame à partir des formats cités ci-dessus ; je vous propose donc ainsi d'importer le même jeu de données... sous 3 formats différents : format Excel, format CSV et format JSON. Vous trouverez les trois fichiers dans ce [dossier compressé](https://drive.google.com/file/d/1KSRbOuseKBq4GZ69P9f35ZFavdiohniH/view?usp=sharing) :

<https://drive.google.com/file/d/1KSRbOuseKBq4GZ69P9f35ZFavdiohniH/view?usp=sharing>

Une fois télécharger et décompresser, déplacer les fichiers vers le répertoire où se trouve votre fichier Python « **analyse.py** » (fichier où sera écrit le code permettant de lire les données contenus dans les différents formats). Voici le code du fichier Python :

```
import pandas as pd

data_csv= pd.read_csv('clients.csv')
print(data_csv.head()) # Afficher les 5 premières lignes
data_json= pd.read_json('clients.json')
print(data_json.head()) # Afficher les 5 premières lignes
data_excel= pd.read_excel('clients.xlsx')
print(data_excel.head()) # Afficher les 5 premières lignes
```

|   | identifiant | email                     | nom              | genre |
|---|-------------|---------------------------|------------------|-------|
| 0 | 0           | LaurentDagenais@rhyta.com | Laurent Dagenais | M     |
| 1 | 1           | GuyMarois@fleckens.hu     | Guy Marois       | M     |
| 2 | 2           | BeaufortLesage@einrot.com | Beaufort Lesage  | M     |
| 3 | 3           | RussellDurand@armyspy.com | Russell Durand   | M     |
| 4 | 4           | AlexisRiel@rhyta.com      | Alexis Riel      | M     |

**Remarque :** Si vous rencontrez l'erreur `ModuleNotFoundError: No module named 'openpyxl'` lors de l'exécution de `pd.read_excel('clients.xlsx')`, cela signifie que la bibliothèque **openpyxl**, qui est nécessaire pour lire des fichiers Excel au format **.xlsx**, n'est pas installée. Pour résoudre ce problème, vous devez installer **openpyxl** en utilisant **pip**. Ouvrez votre terminal ou votre invite de commande et exécutez la commande suivante : **pip install openpyxl**

Et pour lire un fichier Excel au format **.xls**, vous devez également utiliser la bibliothèque **xlrd**, qui est souvent nécessaire pour ce type de fichier. Ouvrez votre terminal ou votre invite de commande et exécutez la commande suivante : **pip install xlrd**

Dans la suite, nous travaillerons beaucoup plus avec les fichiers d'extension **.csv**, car le **format CSV** est de loin le plus couramment utilisé pour enregistrer et échanger des DataFrames en raison de sa simplicité, sa compatibilité étendue et sa légèreté. On n'aura donc :

```
import pandas as pd

clients= pd.read_csv('clients.csv')
print(clients.head()) # Afficher les 5 premières lignes
```

## I.3. Manipuler le data frame

### I.3.1. Naviguez dans le data frame

Dans un data frame, chaque colonne est explicitement nommée, rendant la compréhension de cette dernière plus claire, et permettant d'accéder à une colonne spécifique à partir de son nom !

Pour accéder à une colonne d'un data frame, il suffit d'utiliser la syntaxe **nom\_dataframe[nom\_colonne]**.

Si on voulait accéder à **la liste des e-mails** à partir du data frame **clients** importé précédemment, on écrira :

```
import pandas as pd

clients= pd.read_csv('clients.csv')
Recupérer la colonne email
email = clients['email']
print(email)
```

Ainsi, on accède à la variable **email** de notre data frame **clients**. La syntaxe permet une lecture assez claire de ce à quoi on essaie d'accéder.

*Et si on souhaite accéder à plusieurs colonnes ?* Il suffit de stocker l'ensemble des noms des colonnes auxquelles vous souhaitez accéder dans une liste. Voilà par exemple comment sélectionner **le nom** et **l'e-mail** de nos clients :

```
import pandas as pd

clients = pd.read_csv('clients.csv')
Recupérer les colonnes nom et email
colonnes = ['nom', 'email']
data= clients[colonnes]
Afficher les 5 premières lignes des colonnes nom et email
print(data.head())
```

|   | nom              | email                     |
|---|------------------|---------------------------|
| 0 | Laurent Dagenais | LaurentDagenais@rhyta.com |
| 1 | Guy Marois       | GuyMarois@fleckens.hu     |
| 2 | Beaufort Lesage  | BeaufortLesage@einrot.com |
| 3 | Russell Durand   | RussellDurand@armyspy.com |
| 4 | Alexis Riel      | AlexisRiel@rhyta.com      |

Vous noterez que les colonnes s'affichent dans l'ordre dans lequel nous l'avons spécifié (ici : le nom avant l'e-mail) et non comme elles sont disposées initialement (l'e-mail est avant le nom dans notre data frame). Et si vous vérifiez le data frame initial, vous verrez que l'ordre initial est bien conservé. Ainsi, le processus de sélection nous permet de réorganiser l'ordre des colonnes comme on le souhaite, sans modifier le data frame initial.



### 1.3.2. Découvrez l'objet Series de Pandas

Avant d'aller plus loin, j'aimerais vous poser une simple question : à votre avis, *quels objets manipulons-nous depuis le début de cette section ?* Vous commencez à bien connaître l'objet data frame, mais une colonne d'un data frame... n'est plus un data frame ! En effet, il est temps de vous présenter le second objet de Pandas que vous allez énormément utiliser, et qui est intimement lié au data frame : la **Series**. En effet, chaque colonne de votre data frame est de type **Series**.

Il est important de différencier les deux objets. Même s'ils partagent de nombreuses méthodes, certaines sont néanmoins exclusives à l'un ou l'autre, et cela peut être une source d'erreur lors de l'implémentation d'analyse de données avec Python.

Une différence évidente, mais que je me permets tout de même de noter : un data frame a 2 dimensions avec plusieurs colonnes, alors que la Series n'a qu'une seule dimension. Cela peut fortement vous aider lorsque vous utiliserez certaines méthodes, car vous pourrez savoir à quel type d'objet vous faites face, en vous basant sur l'affichage que vous avez de ce dernier.

Une **Series** ne peut contenir qu'un seul type, alors qu'un **data frame**, qui est finalement **une collection de Series**, peut contenir des colonnes de types différents : une colonne d'entiers, une colonne de nombres décimaux, etc.

Il est important de garder ces notions en tête lors de vos futures analyses de données. De nombreuses méthodes sont communes à ces deux objets, alors que d'autres sont spécifiques. Par exemple, l'ensemble des attributs : **shape**, **head**, **dtypes** existent pour des **Series**. Mais certaines méthodes, notamment certaines que nous verrons un peu plus tard dans ce cours, sont exclusives aux data frames, à cause de leur aspect multidimensionnel.

Mais sinon, vous pouvez accéder à l'ensemble des méthodes et attributs des **Series** directement via la [documentation de Pandas](https://pandas.pydata.org/docs/reference/api/pandas.Series.html) : <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>

### 1.3.3. Manipulez les colonnes

Dans cette section, je vous propose de voir quelques manipulations indispensables sur les data frames :

- Créer ou supprimer une colonne ;
- Renommer une colonne ;
- Changer le type d'une colonne ;
- Trier un data frame selon une ou plusieurs colonnes.

#### Modifiez une colonne existante

Avant de voir comment créer une colonne, voyons comment en modifier une déjà existante. Reprenons la syntaxe vue précédemment, **nom\_dataframe[nom\_colonne]**. Cette dernière

permet d'accéder à une colonne d'un data frame sans modifier ce dernier. Ce que vous ne savez pas encore, c'est qu'elle permet également de modifier un data frame existant.

Considérons une variable informatique **a** déjà existante :

- On peut y accéder en l'appelant, lui appliquer plusieurs fonctions différentes (comme **print(a)** pour l'afficher) sans modifier sa valeur ;
- Mais on peut également définir ou changer sa valeur (**a=4** qui stockera l'entier 4 dans notre variable **a**).

C'est exactement la même chose avec une colonne d'un data frame :

- On peut y accéder et appliquer plusieurs fonctions et/ou méthodes : c'est ce que nous avons fait jusque-là ;
- On peut également changer les valeurs qui se trouvent à l'intérieur.

Par exemple, si j'exécute :

```
clients['nom'] = 1
```

Cela aura deux effets : cela va modifier la variable **nom** existante en remplaçant toutes les valeurs par **1**, et cela transforme également le type de la variable (**print(clients['nom'].dtypes)**).

**nom** était de type **object** jusque-là ; elle est à présent de type **integer** car 1 est un entier. Ainsi, si on le remplace par une valeur fixe (comme c'est le cas ici), cela aura pour effet de remplacer l'ensemble des valeurs de notre colonne par cette valeur fixe, et changera le type de la colonne dans le type de la valeur fixe spécifiée, qu'elle soit numérique ou non.

On peut également remplacer une colonne par un objet de même dimension. Comprenez par là une liste, un array ou une series comprenant l'exact même nombre d'éléments. Par exemple, si je souhaite modifier la colonne **identifiant** par elle-même multipliée par 100, je peux le faire de la façon suivante :

```
clients['identifiant'] = clients['identifiant']*100
```

Je peux aussi décider de la remplacer par des valeurs aléatoires entre 1 et 1 000 :

```
clients['identifiant'] = np.random.randint(1, 1000, clients.shape[0])
```

**np.random.randint(1, 1000, clients.shape[0])** : génère un tableau de nombres entiers aléatoires compris entre 1 et 999 (puisque 1000 n'est pas inclus). Le nombre de valeurs générées correspond au nombre de lignes dans clients, grâce à **clients.shape[0]**.

Vous noterez que l'array généré pour remplacer les valeurs doit avoir strictement la même dimension que la colonne qu'il remplace. Vous aurez une erreur si ce n'est pas le cas, donc soyez vigilant sur cet aspect ! Et aussi, en pratique, on ne modifiera une colonne que lorsqu'on est vraiment sûr de nous, afin justement de ne pas supprimer par mégarde des données dont on aurait besoin.

## Créez une colonne

Maintenant, comment créer une colonne ? De la même façon qu'on en modifie une. On appelle la colonne, comme si elle existait déjà, et on lui attribue une valeur. Par exemple :

```
clients['id'] = clients['identifiant'] + 1000
print(clients.head())
```

Si vous exécutez ces lignes de votre côté, vous verrez que même si la colonne **id** n'existe pas dans notre data frame, Pandas va la créer et lui attribuer la valeur définie (ici, la valeur se trouvant dans **identifiant**, plus 1 000).

Pour résumer, que ce soit pour modifier ou créer une colonne **col**, la syntaxe sera : **mon\_dataframe['col'] = x** où **x** représente soit une valeur fixe, soit un objet de même dimension que la colonne qu'on souhaite modifier/créer.

## Supprimez une colonne

Il existe officiellement 3 façons. D'abord la méthode **.drop** des data frames :

```
clients= clients.drop(columns='id')
```

Contrairement aux deux options qui suivront, la méthode **.drop** ne modifie pas le data frame existant, elle renvoie juste une sorte de copie du data frame en y ayant appliqué les modifications – ici supprimer la colonne **id**. Vous aurez donc ainsi besoin de remplacer votre data frame.

Ensuite, on peut aussi effectuer cela via le mot clé **del** :

```
del clients['id']
print(clients.head())
```

Ou finalement via la méthode **.pop** :

```
clients.pop('id')
print(clients.head())
```

Ces 3 méthodes sont équivalentes : c'est selon votre préférence !

## Renommez une colonne

La méthode pour renommer une colonne est **.rename**. On peut ainsi renommer une ou plusieurs colonnes via la syntaxe : **mon\_dataframe.rename(columns={'ancien nom': 'nouveau nom'})**. Voilà par exemple comment renommer la colonne **identifiant** en **ide** :

```
clients= clients.rename(columns={'identifiant': 'ide'})
print(clients.head())
```

On peut naturellement renommer plusieurs colonnes en une fois. Par exemple en renommant en plus **email** en **mail** :

```
clients= clients.rename(columns={'identifiant': 'ide', 'email': 'mail'})
```

À noter que la méthode **rename** ne modifie pas le data frame existant. Il existe cependant un argument pour cette méthode (et pour toutes les méthodes similaires) nommé **inplace**, qu'il suffit de fixer à Vrai (**True**) pour pallier cela. Ainsi, **clients.rename(columns={'identifiant': 'ide'}, inplace=True)** est strictement équivalent à **clients= clients.rename(columns={'identifiant': 'ide'})**.

### Changez le type d'une colonne

La méthode **.astype** permet de changer le type d'une colonne. Par exemple, si on souhaite transformer la colonne **identifiant**, initialement composée d'entiers, en nombres décimaux, on peut le faire de la façon suivante :

```
clients['identifiant']= clients['identifiant'].astype(float)
print(clients.head())
```

Il suffira de préciser le type entre parenthèses !

### Triez un data frame

En analyse de données, on a régulièrement besoin de trier des données selon une ou plusieurs colonnes. Pandas met à disposition la méthode **.sort\_values** pour faire cela très aisément. Il suffit de préciser entre parenthèses la ou les colonnes selon lesquelles il faut trier. Voici quelques exemples :

```
trier selon l'identifiant, par ordre croissant :
clients= clients.sort_values('identifiant')

#trier selon l'identifiant par ordre décroissant :
clients= clients.sort_values('identifiant', ascending = False)

trier selon le genre puis le nom, par ordre croissant :
clients= clients.sort_values(['genre', 'nom'])
```

Le paramètre **inplace** est également utilisable ici pour le DataFrame directement :

**clients.sort\_values('identifiant', inplace=True)**

Le mot clé **ascending** permet de définir si on souhaite trier par ordre croissant ou décroissant. Ainsi, si on souhaite trier une colonne par ordre croissant et une autre par ordre décroissant ? Il faudra dans ce cas donner en paramètre à **ascending** une liste de booléens. Voici par exemple comment trier par genre en ordre croissant et par nom en ordre décroissant :

```
Clients= clients.sort_values(['genre', 'nom'], ascending=[True, False])
```

## I.4. Filtrez les données du data frame

Imaginons les situations suivantes :

- Pour une offre commerciale spécifique, on cherche à ne sélectionner que les hommes de notre data frame ;
- On souhaite identifier les hauts revenus parmi notre clientèle. Pour cela, on souhaite sélectionner tout ceux gagnant mensuellement plus de **x** FCFA ;
- On souhaite identifier les clients dits « risqués », c'est-à-dire ceux qui ont un taux d'endettement supérieur aux 35 % légaux.

Jusqu'à présent, nous avons été capables de sélectionner des colonnes spécifiques et de les manipuler à notre guise. Mais les exemples précédents ne peuvent être satisfaits avec de simples manipulations de colonnes : il faut appliquer une condition spécifique pour ne sélectionner que les lignes pertinentes.

Cette opération est ce qu'on appelle une **restriction**, ou un **filtrage**. Naturellement, Pandas permet de faire cela, pour avoir un contrôle total sur nos données. Voyons cela ensemble !

### I.4.1. Appliquez la sélection via des indices

Le premier type de sélection est réalisé via des **indices**. Il se fait via la méthode **.iloc**.

Pour rappel, un indice est la position à laquelle se trouve un élément au sein d'un tableau, que ce soit une liste, une Series ou un array. En Python, les indices commencent à 0 : ainsi, le premier élément aura pour indice 0, le second 1, etc.

La méthode **.iloc** suit la syntaxe suivante :

**mon\_dataframe.iloc[indice\_ligne, indice\_colonne]**

Prenons un exemple concret : on souhaite sélectionner le nom du premier client de notre base de données. Comme c'est le premier client, **son indice de ligne sera 0**. Nous souhaitons sélectionner son nom ; la variable nom est la 3<sup>e</sup> variable de notre data frame, l'**indice de sa colonne sera donc... 2**. Eh oui ! Pour les colonnes aussi, les indices commencent à 0. Le code de sélection serait donc :

```
clients.iloc[0,2]
```

```
'Laurent Dagenais'
```

Cela correspond bien au nom de notre premier client.

Maintenant, admettons que je souhaite sélectionner le nom des 10 premiers clients, comment procéder ?

Comme avec les arrays, nous pouvons utiliser l'opérateur « : » pour sélectionner la plage souhaitée ! Voici quelques exemples :

```
sélectionner le nom des 10 premiers clients :
clients.iloc[:10, 2]

sélectionner le genre des 10 premiers clients :
clients.iloc[:10, 3]

sélectionner toutes les colonnes des 10 premiers clients :
clients.iloc[:10, :]

sélectionner l'email et le nom des 10 clients suivants :
clients.iloc[10:20, 1:3]

sélectionner l'e-mail et le genre des 10 derniers clients :
clients.iloc[-10:, [1, 3]]
```

La méthode **iloc** peut être utile dans certains cas, mais elle est finalement assez limitée. En effet, elle n'est pas adaptée si on souhaite sélectionner selon une certaine condition (par exemple tous les revenus supérieurs à x FCFA), et il faut absolument connaître l'indice des différentes colonnes. Cela reste jouable avec 5-6 colonnes, mais devient beaucoup plus compliqué lorsqu'on est en présence de dizaines de colonnes.

Il existe une autre méthode, qui est similaire à **iloc** mais qui permet de pallier l'ensemble de ces problématiques : la méthode **.loc**.

## 1.4.2. Utilisez la sélection via des conditions

Pour être tout à fait honnête avec vous, les data frames possèdent de nombreuses méthodes, comme vous avez pu commencer à vous en rendre compte. Mais de toutes, il y en a certainement une que vous allez utiliser et réutiliser à outrance : c'est la méthode **.loc**

Celle-ci suit la syntaxe suivante :

**mon\_dataframe.loc[ condition sur les lignes, colonne(s) ]**

C'est à première vue très simple, mais cette syntaxe peut en pratique devenir vite complexe, et peut permettre en même temps une grande flexibilité. C'est LA méthode qui pourra répondre à près de 90 % de vos besoins en termes de sélection d'une partie de vos données.

Voici quelques exemples d'utilisation de la méthode **.loc** pour filtrer les données d'un DataFrame à partir du fichier **clients.csv** :

```
import pandas as pd

Charger le fichier CSV
clients = pd.read_csv('clients.csv')

Sélectionner tous les clients de genre masculin :
hommes = clients.loc[clients['genre'] == 'M']
print(hommes)

Sélectionner les clients avec un identifiant supérieur à 100 :
clients_sup_100 = clients.loc[clients['identifiant'] > 100]
print(clients_sup_100)

Sélectionner les clients féminins dont le nom contient "Anne" :
femmes_anne = clients.loc[(clients['genre'] == 'F') & (clients['nom'].str.contains('Anne'))]
print(femmes_anne)

Sélectionner les clients dont l'identifiant est pair et qui sont de genre masculin :
hommes_pairs = clients.loc[(clients['genre'] == 'M') & (clients['identifiant'] % 2 == 0)]
print(hommes_pairs)

Sélectionner uniquement les colonnes email et nom pour les clients de genre féminin :
femmes_email_nom = clients.loc[clients['genre'] == 'F', ['email', 'nom']]
print(femmes_email_nom)

Sélectionner les clients masculins avec un identifiant entre 50 et 150, et afficher uniquement l'email et le nom :
hommes_id_50_150 = clients.loc[
 (clients['genre'] == 'M') &
 (clients['identifiant'].between(50, 150)),
 ['email', 'nom']
]
print(hommes_id_50_150)
```

Une autre option de la méthode **.loc** est la sélection par **index**. Avant d'aller plus loin sur cet aspect, laissez-moi redéfinir la différence entre des **index** et des **indices** ! Les **indices**, comme nous l'avons vu plus haut, sont la position intrinsèque d'un élément au sein d'un tableau. Les **index**, en revanche, correspondent à une valeur qui est associée à chaque ligne. C'est ce que vous voyez sur la gauche de votre data frame :

clients.head()

|   | identifiant | email                     | nom              | genre |
|---|-------------|---------------------------|------------------|-------|
| 0 | 0           | LaurentDagenais@rhyta.com | Laurent Dagenais | M     |
| 1 | 1           | GuyMarois@fleckens.hu     | Guy Marois       | M     |
| 2 | 2           | BeaufortLesage@einrot.com | Beaufort Lesage  | M     |
| 3 | 3           | RussellDurand@armyspy.com | Russell Durand   | M     |
| 4 | 4           | AlexisRiel@rhyta.com      | Alexis Riel      | M     |

Index

Par défaut, ils correspondent... aux indices ! Mais ils peuvent ne pas être numériques ; on pourrait par exemple fixer le nom de la personne comme index, si on est sûr qu'il n'y aura pas de doublons.

Faisons un simple tri pour bien comprendre la différence entre l'indice et l'index :

clients.sort\_values('nom').head()

|     | identifiant | email                         | nom              | genre |
|-----|-------------|-------------------------------|------------------|-------|
| 13  | 13          | AcelineHughes@jourrapide.com  | Aceline Hughes   | F     |
| 135 | 135         | AdelaideBrousse@gustr.com     | Adelaide Brousse | F     |
| 15  | 15          | AgateGrandbois@gustr.com      | Agate Grandbois  | F     |
| 7   | 7           | AgramantPepin@dayrep.com      | Agramant Pepin   | M     |
| 16  | 16          | AgricanFoucalt@jourrapide.com | Agrican Foucalt  | M     |

Cette situation est intéressante, car si on regarde notre première ligne, son index est **13**. Pourtant, c'est bel et bien la première ligne, donc son indice est **0** !!!

Je pense que vous commencez à comprendre la différence : un indice est relatif aux opérations réalisées. Un individu peut avoir le premier indice à un moment, et un tout autre indice en fonction du tri effectué. En revanche, l'index est intrinsèque aux lignes au sein d'un data frame. Peu importe les opérations effectuées, que ce soit un tri, une suppression, etc., l'index sera toujours le même ! La seule condition, c'est que chaque ligne doit avoir un index unique : c'est-à-dire qu'il ne doit pas y avoir deux fois la même valeur au sein d'un index ; on ne pourra par exemple pas avoir 2 fois l'index 0, etc.

**Pourquoi je vous parle de tout cela ?** Parce qu'il est possible de faire une sélection au sein d'un data frame selon les index, avec la méthode **.loc**. Faites le test avec les lignes suivantes :

```
import pandas as pd
clients = pd.read_csv('clients.csv')
print(clients.loc[0:10, :])
print(clients.iloc[0:10, :])
```

Il est d'ailleurs possible d'accéder à la liste des index d'un data frame via l'attribut **.index**. Par exemple, **clients.index** nous renverra l'ensemble des index de notre data frame.

Il est parfois intéressant, après avoir effectué certaines opérations, de réinitialiser ces index. C'est possible via la méthode **.reset\_index()** :

```
import pandas as pd
clients = pd.read_csv('clients.csv')

création d'un data frame trié pour 'désordonner' les Index
clients_temp = clients.sort_values('nom')
print(clients_temp)

reset_index (cette méthode garde les anciens index dans une nouvelle colonne) :
print(clients_temp.reset_index())

reset_index sans garder les anciens Index :
print(clients_temp.reset_index(drop=True))
```

### 1.4.3. Modifiez une sélection

Comme avec les colonnes, on peut utiliser les méthodes **.iloc** ou **.loc** pour modifier un data frame existant. C'est même encore plus intéressant que ce que nous avons vu jusque-là, car il est possible de modifier précisément une partie d'un data frame.



Voici quelques exemples :

```
import pandas as pd

Charger le fichier CSV
clients = pd.read_csv('clients.csv')

Modifications avec .loc

Changer le genre d'un client spécifique (identifiant 0) :
clients.loc[clients['identifiant'] == 0, 'genre'] = 'F'
print(clients.loc[clients['identifiant'] == 0])

Mettre à jour l'email d'un client (identifiant 1) :
clients.loc[clients['identifiant'] == 1, 'email'] = 'nouveau_email@example.com'
print(clients.loc[clients['identifiant'] == 1])

Changer le genre de tous les clients masculins en féminin :
clients.loc[clients['genre'] == 'M', 'genre'] = 'F'
print(clients[clients['genre'] == 'F'])

Mettre à jour les emails de tous les clients dont le nom contient "Laurent" :
clients.loc[clients['nom'].str.contains('Laurent'), 'email'] = 'laurent@example.com'
print(clients[clients['nom'].str.contains('Laurent')])

Modifications avec .iloc

Changer le nom du client à la position 5 (6ème ligne) :
clients.iloc[5, 2] = 'Nouveau Nom'
print(clients.iloc[5])

Modifier le genre des clients de la position 10 à 15 :
clients.iloc[10:15, 3] = 'F'
print(clients.iloc[10:15])
```

## I.5. Agrégez des données avec Pandas

Maintenant que nous sommes capables d'accéder à tout ce que nous souhaitons au sein de notre data frame, je vous propose de voir comment agréger les différentes informations.

On suppose un **jeu de données (300 échantillons)** constitué de l'ensemble des prêts immobiliers et automobiles contractés par des clients auprès d'une banque au Cameroun. Cette banque opère dans 6 villes différentes : Douala, Yaoundé, Bamenda, Kribi, Ebolowa et Bafoussam. Le jeu de données comprend les colonnes suivantes :

- Identifiant : Un numéro unique attribué à chaque prêt ou client, permettant de distinguer chaque enregistrement de manière univoque.
- Code : Qui représente le code du client ayant effectué le prêt (ce code est de la forme CL-X où X est un nombre)
- Ville : Ville où se trouve l'agence
- BP : Boîte Postale du client permettant de localiser précisément la zone géographique.
- Revenu : Revenu mensuel du client

- Remboursement : Le montant total que le client doit rembourser
- Duree : La durée du prêt exprimée en mois
- Type : Le type de prêt ("immobilier" ou "automobile")
- Taux : Le taux d'intérêt annuel appliqué au prêt

Chaque prêt contracté par un des clients est forcément relié à l'une des agences. Ainsi, le directeur de la banque pourrait se poser cette question : *Admettons que je veuille calculer le chiffre d'affaires mensuel réalisé par chaque agence. Comment puis-je faire ?* Bonne question !

Il va donc falloir :

- Sélectionner l'ensemble des clients de l'agence de Douala et calculer la somme de leurs mensualités ;
- Sélectionner l'ensemble des clients de l'agence de Yaoundé et calculer la somme de leurs mensualités ;
- etc.

Le résultat final serait ainsi une ligne par agence/ville, avec la somme des mensualités perçues par chacune d'elles.

Cette opération est ce qu'on appelle en algèbre relationnelle **une agrégation**. C'est une opération très courante sur des data frames, soit pour analyser les données sous un certain angle, soit pour recalculer certaines variables, comme la moyenne des mensualités par agence.

Je vous propose de voir comment réaliser cela avec Pandas.

### 1.5.1. Créez un jeu de données fictif

Un **jeu de données fictif** est un ensemble de données généré artificiellement, qui imite la structure et les caractéristiques d'un vrai jeu de données, mais sans contenir d'informations réelles ou sensibles.

Il permet de faire des tests, des simulations ou des exercices d'analyse sans compromettre la confidentialité ou la sécurité des données réelles. Utile pour apprendre, développer ou tester des algorithmes, en créant des exemples contrôlés.

Créons un DataFrame fictif pour le jeu de données constitué de l'ensemble des prêts immobiliers et automobiles :

```
import pandas as pd
import numpy as np

Définir les paramètres
villes = ['Douala', 'Yaoundé', 'Bamenda', 'Kribi', 'Ebolowa', 'Bafoussam']
types = ['immobilier', 'automobile']
data = []

Générer 300 échantillons
for i in range(300):
```

```
identifiant = i + 1
code = f"CL-{identifiant}"
ville = np.random.choice(villes)
BP = np.random.randint(1000, 9999)
revenu = np.random.randint(200000, 600000)
remboursement = np.round(revenu * np.random.uniform(0.2, 0.5), 2) #Montant à rembourser mensuel
duree = np.random.choice([36, 48, 60, 72]) # Durées en mois
type_pret = np.random.choice(types)
taux = np.round(np.random.uniform(5.0, 8.5), 2) # Taux d'intérêt

data.append([identifiant, code, ville, BP, revenu, remboursement, duree, type_pret, taux])

Créer le DataFrame
df = pd.DataFrame(data, columns=['identifiant', 'code', 'ville', 'bp', 'revenu', 'remboursement',
 'duree', 'type', 'taux'])
print(df.head())

Enregistrer dans un fichier CSV
df.to_csv('prets.csv', index=False)
```

La méthode **df.to\_csv('prets.csv', index=False)** est utilisée dans Pandas pour sauvegarder un DataFrame sous forme de fichier CSV. Voici une explication détaillée de chaque élément:

- **df** : C'est l'objet DataFrame que vous souhaitez sauvegarder. Il contient vos données sous forme tabulaire.
- **to\_csv** : C'est une méthode de Pandas qui permet de convertir le DataFrame en un fichier CSV (Comma-Separated Values).
- **'prets.csv'** : C'est le nom du fichier dans lequel vous souhaitez enregistrer les données. Si le fichier n'existe pas, il sera créé. Si un fichier avec ce nom existe déjà, il sera écrasé.
- **index=False** : Ce paramètre indique que vous ne souhaitez pas inclure l'index du DataFrame dans le fichier CSV. Par défaut, Pandas inclut l'index, qui est une colonne supplémentaire numérotant chaque ligne. En mettant **index=False**, vous évitez d'avoir cette colonne d'index dans le fichier final.

## 1.5.2. Agrégez plusieurs lignes

La première méthode pour faire une agrégation avec Pandas est **.group\_by**. Pour l'utiliser, vous aurez besoin de vous **fixer** sur une ou plusieurs colonnes, qui seront ce qu'on appelle les **index** de votre résultat agrégé.

Comme nous l'avons dit précédemment, l'index n'est pas forcément l'indice, ça peut être également des chaînes de caractères. Dans notre exemple, notre index serait la variable **ville**. L'index va permettre de créer plusieurs groupes : un pour chaque valeur unique de l'index.

Après avoir choisi les colonnes sur lesquelles vous allez vous fixer, vous aurez à choisir une **fonction d'agrégation**. La fonction d'agrégation va prendre en entrée un groupe de plusieurs lignes, pour effectuer un calcul sur celles-ci dans l'optique de retourner une unique valeur pour chacun des groupes. Ici, la fonction d'agrégation serait une somme.

Si vous avez déjà fait un peu de SQL, vous aurez très certainement fait le lien entre cette opération et l'opération de **GROUP BY** réalisable en SQL. Les deux fonctionnent en effet de façon très similaire !

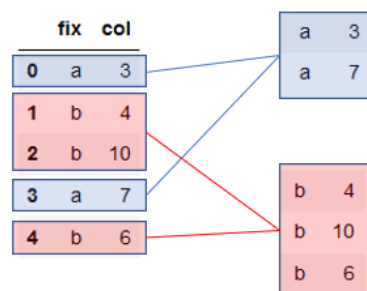
Considérons le data frame **test** suivant :

| test |     |     |
|------|-----|-----|
|      | fix | col |
| 0    | a   | 3   |
| 1    | b   | 4   |
| 2    | b   | 10  |
| 3    | a   | 7   |
| 4    | b   | 6   |

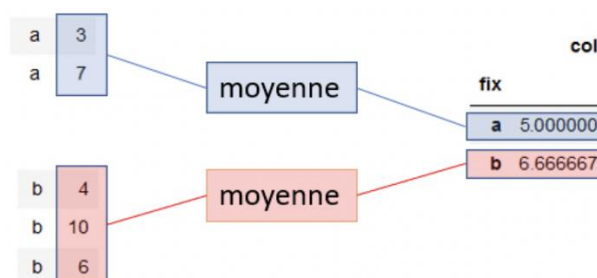
On souhaite calculer la moyenne de la variable **col** pour chaque valeur de la variable **fix**. La ligne en Python pour faire cela est :

```
test.groupby('fix')['col'].mean()
```

Décomposons ce qu'il se passe lors de l'exécution de cette ligne. Dans un premier temps, notre index sera la variable **fix** ; il va donc y avoir 2 groupes créés, un pour chaque valeur dans notre variable **fix**.



Sur chacun de ces groupes, on va appliquer la fonction d'agrégation choisie (ici, la moyenne) afin d'avoir le résultat souhaité :



À présent, comment calculer l'information recherchée initialement, soit le chiffre d'affaires mensuel de chacune de nos agences ?

```
import pandas as pd

prets = pd.read_csv('prets.csv')

Chiffre d'affaires mensuel par agence
chiffre_affaires = prets.groupby('ville')['remboursement'].sum()
print("Chiffre d'affaires mensuel par agence :")
print(chiffre_affaires)
```

Maintenant, imaginons que nous ne souhaitons la même information que par agence **ET** par type de prêt. Vous vous en doutez très probablement, il suffira de transmettre la liste des variables à placer en index :

```
Chiffre d'affaires par agence et par type de prêt
chiffre_affaires_par_type = prets.groupby(['ville', 'type'])['remboursement'].sum()
print("\nChiffre d'affaires par agence et par type de prêt :")
print(chiffre_affaires_par_type)
```

On peut même appliquer des fonctions d'agrégation différentes en fonction de la colonne, voire appliquer plusieurs fonctions d'agrégation sur une même colonne. Ici, je calcule la moyenne et la somme de **remboursement** par agence, ainsi que le maximum de **revenu** :

```
Agrégation avec plusieurs fonctions
aggregation_result = prets.groupby('ville').agg({'remboursement': ['sum', 'mean'], 'revenu': 'max'})
print("\nAgrégation des remboursements et revenus par agence :")
print(aggregation_result)
```

**Attention :** Lorsqu'on utilise **group\_by** dans pandas, les variables fixées en index (celles sur lesquelles vous faites le regroupement, par exemple **ville**) deviennent l'index du DataFrame résultant. Cela signifie qu'elles ne sont plus dans les colonnes classiques du DataFrame, mais dans l'index. C'est-à-dire que si vous essayez d'accéder à la variable **ville** du résultat du **group\_by**, vous aurez simplement une erreur ! Vous serez obligé d'effectuer un **reset\_index** pour la replacer en **colonne**.

Reprenons sur l'avant-dernière ligne de code présentée (portant sur le *Chiffre d'affaires par agence et par type de prêt* : **prets.groupby(['ville', 'type'])['remboursement'].sum()**). Imaginons à présent que nous souhaitons avoir ce même résultat, mais sous forme d'un tableau à double entrée

## 1.5.2. Agrégez des lignes et des colonnes

Nous souhaitons donc avoir la même agrégation, mais avec cette fois-ci, en lignes nos agences, et en colonnes les différents types de prêts. La méthode qui vous permettra de faire cela est la méthode **.pivot\_table**. Celle-ci prend **4 arguments** en paramètres :

- **index** : variable(s) placée(s) en ligne ;
- **columns** : variable(s) placée en colonne(s) ;
- **values** : variable sur laquelle on va appliquer la fonction d'agrégation ;
- **aggfunc** : fonction d'agrégation.

```
Pivot table pour obtenir un tableau à double entrée
pivot_table=prets.pivot_table(index='ville',columns='type',values='remboursement',aggfunc='sum', fill_value=0)
print("\nTableau à double entrée :")
print(pivot_table)
```

Si vous exécutez vous aurez le résultat suivant :

Tableau à double entrée :

| type      | automobile | immobilier |
|-----------|------------|------------|
| ville     |            |            |
| Bafoussam | 3273398.43 | 3786891.78 |
| Bamenda   | 3283168.32 | 3338940.99 |
| Douala    | 2875068.97 | 3383263.04 |
| Ebolowa   | 4611925.70 | 3981805.16 |
| Kribi     | 3632496.54 | 2422262.33 |
| Yaoundé   | 3183559.33 | 5769069.06 |

**Remarque :** L'attribut **fill\_value** dans une **table pivot** (pivot table) est utilisé pour remplacer les valeurs manquantes (**NaN** - **Not a Number**) dans le tableau résultant. Lorsqu'aucune donnée ne correspond à une combinaison spécifique des index et des colonnes, **fill\_value** permet de spécifier une valeur par défaut à afficher à la place. Éviter les **NaN** permet d'avoir un tableau plus propre et plus facile à lire, sans valeurs manquantes. Vous pouvez définir une valeur significative (comme 0) pour indiquer qu'il n'y a pas de remboursement pour cette combinaison.

### La fonction melt

La fonction **melt** dans pandas est utilisée pour transformer un DataFrame de **format large** (wide format) en **format long** (long format). Elle permet de "fondre" (melt) plusieurs colonnes en une seule, en conservant les identifiants, ce qui est utile pour la manipulation, l'analyse ou la visualisation de données. Cela permet de réorganiser les données pour faciliter leur analyse ou leur visualisation.

Ainsi, partant du table pivot, nous souhaiterions avoir un résultat semble à ceci **prets.groupby(['ville', 'type'])['remboursement'].sum()** avec les index villes comme colonne. On aura :

```
Utilisation de melt pour transformer le tableau
melted = pivot_table.reset_index().melt(id_vars='ville', value_vars=pivot_table.columns,
var_name='type', value_name='remboursement')
print("\nTableau transformé avec melt :")
print(melted)
```

Tableau transformé avec melt :

|    | ville     | type       | remboursement |
|----|-----------|------------|---------------|
| 0  | Bafoussam | automobile | 3273398.43    |
| 1  | Bamenda   | automobile | 3283168.32    |
| 2  | Douala    | automobile | 2875068.97    |
| 3  | Ebolowa   | automobile | 4611925.70    |
| 4  | Kribi     | automobile | 3632496.54    |
| 5  | Yaoundé   | automobile | 3183559.33    |
| 6  | Bafoussam | immobilier | 3786891.78    |
| 7  | Bamenda   | immobilier | 3338940.99    |
| 8  | Douala    | immobilier | 3383263.04    |
| 9  | Ebolowa   | immobilier | 3981805.16    |
| 10 | Kribi     | immobilier | 2422262.33    |
| 11 | Yaoundé   | immobilier | 5769069.06    |

La méthode **melt** possède majoritairement **4 arguments** en paramètres :

- **id\_vars** : Les variables sur lesquelles on va agréger les données (ici, *ville*).
- **value\_vars** : Les colonnes à transformer en lignes (ici, *automobile* et *immobilier*).
- **var\_name** : Nom de la nouvelle colonne créée pour les noms des colonnes d'origine (ici, *type*).
- **value\_name** : Nom de la colonne pour les valeurs correspondantes (ici, *remboursement*).

## I.6. Fusionnez des données avec Pandas

Vous êtes à présent à même d'agréger vos données. Mais que faire lorsque ces dernières sont partagées entre plusieurs fichiers de données et donc, plusieurs data frames ? C'est ce que je vous propose de découvrir !

### I.6.1. Comprenez l'intérêt de la fusion de données

Dans une banque, les sources de données peuvent être multiples. Nous avons déjà notre liste de clients d'une part, et la liste des prêts fournie par le service adéquat au sein de notre établissement. Mais il peut y avoir bien d'autres choses ! On peut imaginer notamment :

- Les informations issues de la navigation des clients sur le site web ;
- Les données récoltées via notre application mobile ;
- Les données issues des différents services (Professionnels, Investissement) ;
- etc.

Chaque source peut être matérialisée par un ou plusieurs fichiers de données et donc, un ou plusieurs data frames. On peut donc être amenés à travailler avec de nombreuses sources différentes, qui ont pourtant des informations complémentaires.

Prenons par exemple nos deux fichiers de données, **clients** et **prêts** :

- D'un côté, nous avons des informations générales sur nos clients, comme leur e-mail ou leur genre ;
- De l'autre côté, nous avons des informations sur les prêts qui ont été contractés par ces clients.

Ces informations sont complémentaires car elles concernent les **mêmes clients**. Il serait donc intéressant de pouvoir **recouper** ces deux jeux de données pour mener des analyses pertinentes, comme par exemple vérifier si les montants d'emprunt ou les taux attribués par nos conseillers varient en fonction du genre de notre client. Le fait de rassembler comme cela plusieurs jeux de données différents est appelé une **jointure**, en algèbre relationnelle.

Tout comme l'agrégation, c'est une opération très courante lorsqu'on manipule des données, car il est rare que l'ensemble des données à utiliser proviennent d'une seule et unique source !

Pour effectuer une **jointure**, nous aurons absolument besoin d'une information qui soit en commun dans chaque jeu de données : c'est que nous appelons une **clé**. La **clé** à utiliser pour faire une jointure peut être formée d'une (comme c'est très souvent le cas) ou plusieurs colonnes : il n'y a pas de limite à ce niveau ! La seule condition est que la clé choisie soit présente dans chaque jeu de données qu'on essaie de joindre aux autres.

Dans l'exemple ci-dessus, l'opération de jointure a un sens car il est possible de faire le lien entre les informations générales de clientèle et les informations de prêt, à partir de l'identifiant client : cette information est commune aux deux data frames. C'est donc la clé que nous allons utiliser pour effectuer notre jointure.

Comme pour l'agrégation, l'opération est quasi équivalente à une jointure qu'on pourrait réaliser en SQL, à la différence près que les clés primaires et étrangères ne sont pas explicitement identifiées dans un data frame Python.

Je vous propose de voir comment réaliser cela avec Python.

### 1.6.1. Fusionnez des données avec merge

La **fonction** ou **méthode** prévue par Pandas pour faire une jointure se nomme **merge**. Eh oui ! Vous avez bien lu ! J'ai bien dit fonction ou méthode. Car il existe deux façons de faire une jointure entre 2 data frames **A** et **B** avec Pandas :

- Via la fonction Pandas. Dans ce cas-là, l'écriture sera **pd.merge(A, B)**
- Via la méthode de data frame. Dans ce cas-là, l'écriture sera **A.merge(B)**

**Quelles sont donc les différences entre ces deux écritures ?** Absolument aucune, en dehors de la syntaxe présentée ci-dessus. Que vous utilisiez la fonction ou la méthode, vous aurez à paramétrer les mêmes arguments et vous obtiendrez le même résultat ! Disons que vous pouvez choisir l'écriture qui vous semble la plus logique et la plus simple à comprendre.

Dans ce que je vous ai présenté jusque-là, j'ai totalement occulté la **clé** utilisée pour faire la jointure. Par défaut, si on ne lui indique rien de spécifique, Pandas va chercher les colonnes en commun dans les différents data frames (celles qui portent le même nom) et va les sélectionner comme clés : c'est ce qu'on appelle une **jointure naturelle**.

Mais nous pouvons naturellement spécifier la clé à utiliser ! Il va donc y avoir deux cas possibles :

- Lorsque la clé porte le même nom dans chaque data frame, on peut utiliser l'argument **on**
- Lorsque la clé n'a pas le même nom, il est nécessaire de préciser quelle clé on va utiliser dans chaque data frame. On aura alors besoin des arguments **right\_on** et **left\_on** qui correspondront à la clé dans le data frame à droite et celle dans le data frame à gauche.



À ce propos, vous verrez que cette notion de **right** (droite) et **left** (gauche) revient régulièrement lorsqu'on parle de jointure. Laissez-moi expliquer ça : c'est simplement l'ordre dans lequel nous plaçons nos data frames au sein du **merge**.

Voici 3 exemples de jointures :

```
jointure entre 2 dataframes A et B
pd.merge(A, B, on='id')

jointure entre 2 dataframes A et C
C.merge(A, left_on='identifiant', right_on='id')

jointure entre 2 dataframes A et B
pd.merge(A, B, left_on='id', right_on='identifiant')
```

Décomposons chaque cas :

- Dans le premier cas, c'est une jointure simple sur une clé **id**. Cela suppose donc qu'il y a dans le data frame A et dans le data frame B, une colonne qui porte ce nom. Cela renverra une erreur dans le cas contraire ;
- Dans le cas suivant, le data frame considéré comme **left** est le data frame sur lequel on effectue la jointure : ici, il s'agit donc du data frame C. Le data frame A est considéré comme étant à droite. La jointure est réalisée sur la clé **identifiant** pour le data frame de gauche (ici, C), et **id** pour l'autre ;
- Dans le dernier cas, le data frame considéré comme **left** est le premier renseigné, donc ici A. B est le data frame **right**. La jointure est réalisée avec la clé **id** du data frame de gauche (ici, A), et **identifiant** du data frame de droite.

Je vous propose à présent de joindre nos deux data frames **prêts** et **clients** dans un nouveau data frame nommé très originalement... **df**. La clé commune à chaque data frame est l'**identifiant**, c'est donc ce que nous utiliserons comme clé pour effectuer notre jointure :

```
import pandas as pd

Jointure entre 2 dataframes clients et prêts
clients= pd.read_csv('clients.csv')
prêts= pd.read_csv('prêts.csv')

df= pd.merge(clients, prêts, on='identifiant', how='inner')
print("\nTableau fusionné clients et prêts :")
print(df)
```

Si vous regardez bien le code, vous verrez qu'un argument a été ajouté à la fonction **merge** : **how='inner'**.

L'argument **how** dans la fonction **pd.merge()** de pandas détermine la manière dont les DataFrames sont combinés en fonction des clés spécifiées.

Voici les différentes options disponibles pour **how** :

- **inner** (jointure interne – inner join) : C’est le type de jointure par défaut. Il ne conserve que les lignes ayant des clés communes dans les deux DataFrames. *Cette technique sera appliquée par défaut si **how** n'est pas spécifié.*
- **left** (jointure à gauche – left join) : Conserve toutes les lignes du DataFrame de gauche (premier DataFrame). Les lignes du DataFrame de droite (deuxième DataFrame) sont ajoutées lorsque les clés correspondent. Les valeurs manquantes sont remplies avec **NaN**.
- **right** (jointure à droite – right join) : Conserve toutes les lignes du DataFrame de droite. Les lignes du DataFrame de gauche sont ajoutées lorsque les clés correspondent, avec des valeurs manquantes remplies avec **NaN**.
- **outer** (jointure externe – outer join) : C’est une sorte de conjonction des jointures à gauche et à droite. C’est-à-dire qu’il conserve toutes les lignes des deux DataFrames. Les valeurs manquantes pour les clés non correspondantes sont remplies avec **NaN**.

## 1.6.2. Concaténez des données avec Pandas

La **concaténation** intervient lorsqu’on souhaite assembler plusieurs data frames qui ont la même structure, les mêmes colonnes, les mêmes types, mais des informations différentes. Le cas typique correspond à des exports de données mensuelles, où on aurait les mêmes variables d’un mois à l’autre.

L’idée ici ne serait donc pas de joindre les deux fichiers, mais simplement de les mettre l’un à la suite des autres, comme lors d’un ajout d’élément à une liste, via la méthode **append**. La jointure a une dimension plus horizontale, alors que la concaténation est plus verticale.

La fonction Pandas permettant de faire une concaténation est la fonction **concat**. Pour concaténer plusieurs data frames, il suffit de placer l’ensemble de ceux-ci dans une liste, et d’utiliser la fonction **concat** sur cette liste.

Par exemple, voilà comment procéder avec deux data frames, **df1** et **df2** :

```
liste_concat = [df1, df2]
pd.concat(liste_concat)
```

Le souci d’une concaténation, c’est qu’elle ne gère pas du tout les index par défaut. L’opération met juste le second data frame à la suite du premier, sans appliquer la moindre modification. Ainsi, vous aurez des index en doublon, ce qui peut être problématique ! Pour éviter cela, on peut utiliser l’argument **ignore\_index**, qui équivaut à appliquer une méthode **reset\_index** :

**pd.concat([df1, df2], ignore\_index=True)**

## II. Introduction à Plotly

### II.1. Présentation de la bibliothèque

Plotly est une bibliothèque de visualisation de données interactive en Python qui permet de créer des graphiques dynamiques et attrayants. Elle est particulièrement utile pour les analyses de données et la présentation visuelle, offrant une multitude de types de graphiques.

#### Caractéristiques principales

- **Graphiques interactifs** : Les graphiques peuvent être zoomés, déplacés et explorés par les utilisateurs.
- **Large éventail de types de graphiques** : Graphiques en ligne, en barres, en nuages de points, en surface, en histogrammes, etc.
- **Facilité d'utilisation** : Il s'intègre facilement avec des bibliothèques comme Pandas et NumPy.
- **Support pour les tableaux et images** : Il donne la possibilité de créer des tableaux interactifs et d'ajouter des images aux visualisations.
- **Personnalisation** : Il dispose d'options pour personnaliser les couleurs, les étiquettes, les axes, et plus encore.

#### Installation

Pour utiliser **Plotly** dans Visual Studio Code (VS Code), il suffit d'ouvrir un terminal dans VS Code et exécutez la commande suivante :

```
pip install plotly
```

### II.2. Implémentez un graphique avec Plotly

Voici un aperçu des différents types de graphiques que vous pouvez créer avec Plotly :

#### Graphique en ligne

Le graphique en ligne est idéal pour visualiser des données continues sur une période ou un axe ordinal. Il connecte des points de données avec des lignes, permettant d'observer facilement les tendances, les fluctuations et les variations au fil du temps. Ce type de graphique est souvent utilisé pour représenter des séries temporelles, comme les ventes mensuelles ou les températures quotidiennes. Les marqueurs sur les points de données aident à identifier des valeurs spécifiques.

#### Utilisations :

- Suivi des performances financières au fil des mois.

- Visualisation des changements climatiques sur une période donnée.
- Comparaison de plusieurs séries de données pour observer les différences de tendance.

**Exemple :**

```
import plotly.graph_objects as go

Données
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]

Création d'un graphique
fig = go.Figure(data=go.Scatter(x=x, y=y, mode='lines+markers'))

Ajout de titres
fig.update_layout(title='Exemple de graphique en ligne',
 xaxis_title='X Axis',
 yaxis_title='Y Axis')

Affichage du graphique
fig.show()
```

**Graphique en barres**

Le **graphique en barres** est utilisé pour comparer des valeurs entre différentes catégories. Chaque barre représente une catégorie et la hauteur (ou la longueur, si c'est un graphique à barres horizontales) de la barre correspond à la valeur de cette catégorie. Ce type de graphique est particulièrement efficace pour faire des comparaisons directes entre plusieurs groupes.

**Utilisations :**

- Comparer les ventes de différents produits.
- Visualiser les résultats d'enquêtes ou de sondages.
- Illustrer la distribution des populations dans différentes régions.

**Exemple :**

```
import plotly.graph_objects as go

Catégories et valeurs correspondantes
x = ['A', 'B', 'C', 'D']
y = [10, 15, 7, 20]

Création du graphique en barres
fig = go.Figure(data=go.Bar(x=x, y=y))

Configuration des titres
fig.update_layout(title='Graphique en barres',
 xaxis_title='Catégories',
 yaxis_title='Valeurs')

Affichage du graphique
fig.show()
```

## Nuage de points

Le nuage de points est utilisé pour explorer la relation entre deux variables numériques. Chaque point sur le graphique représente une observation, avec une coordonnée sur l'axe X et une coordonnée sur l'axe Y. Ce type de graphique permet de déceler des corrélations, des tendances ou des regroupements au sein des données. Il est souvent utilisé en statistique pour visualiser la régression et les relations entre variables.

### Utilisations :

- Analyser la relation entre le revenu et le niveau d'éducation.
- Visualiser la corrélation entre la taille et le poids des individus.
- Identifier des groupes dans des données multivariées.

### Exemple :

```
import plotly.graph_objects as go

Données pour les axes X et Y
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 1, 4]

Création du nuage de points
fig = go.Figure(data=go.Scatter(x=x, y=y, mode='markers'))

Configuration des titres
fig.update_layout(title='Nuage de points',
 xaxis_title='X',
 yaxis_title='Y')

Affichage du graphique
fig.show()
```

## Graphique en secteurs ou circulaire

Le graphique en secteurs, ou circulaire ou camembert, représente des proportions de différentes catégories par rapport à un total. Chaque secteur du graphique représente une catégorie, et la taille de chaque secteur est proportionnelle à la valeur de cette catégorie par rapport au total. Ce type de graphique est utile pour illustrer la part relative de chaque catégorie dans un tout.

### Utilisations :

- Visualiser la répartition des budgets entre différentes dépenses.
- Montrer la part de marché des différentes entreprises dans un secteur.
- Illustrer la composition démographique d'une population.

### Exemple :

```
import plotly.graph_objects as go

Étiquettes et valeurs des secteurs
labels = ['A', 'B', 'C', 'D']
values = [4500, 2500, 1050, 2000]
```

```
Création du graphique en secteurs
fig = go.Figure(data=go.Pie(labels=labels, values=values))

Configuration du titre
fig.update_layout(title='Graphique en secteurs')

Affichage du graphique
fig.show()
```

## Histogramme

L'histogramme est utilisé pour représenter la distribution d'une variable continue en regroupant les données en intervalles (ou "bins"). Chaque barre représente le nombre d'observations (fréquence) dans chaque intervalle. Cela permet de visualiser la forme de la distribution des données, qu'elle soit normale, asymétrique ou bimodale.

### Utilisations :

- Analyser la distribution des notes d'un examen.
- Observer la répartition des âges dans une population.
- Identifier des anomalies ou des outliers dans les données.

### Exemple :

```
import plotly.graph_objects as go
import numpy as np

Génération de données aléatoires
data = np.random.randn(100)

Création de l'histogramme
fig = go.Figure(data=go.Histogram(x=data))

Configuration des titres
fig.update_layout(title='Histogramme',
 xaxis_title='Valeurs',
 yaxis_title='Fréquence')

Affichage du graphique
fig.show()
```

## Graphique en surface

Le **graphique en surface** est utilisé pour visualiser des données tridimensionnelles. Il représente une surface continue dans l'espace, où les axes X et Y représentent les coordonnées et l'axe Z représente une valeur dépendante. Ce type de graphique est utile pour visualiser des fonctions mathématiques et des modèles de données complexes.

### Utilisations :

- Visualiser des données topographiques, comme des élévations géographiques.
- Analyser des phénomènes physiques, comme la distribution de la chaleur ou de la pression.
- Représenter des modèles de données dans l'apprentissage automatique.

**Exemple :**

```
import plotly.graph_objects as go
import numpy as np

Génération de valeurs pour les axes X et Y
x = np.linspace(-3, 3, 50) # 50 points entre -3 et 3
y = np.linspace(-3, 3, 50) # 50 points entre -3 et 3

Création d'une grille de coordonnées
x, y = np.meshgrid(x, y)

Calcul des valeurs Z en utilisant une fonction simple (par exemple, une parabole)
z = x**2 + y**2 # Valeurs Z représentant une surface parabolique

Création du graphique en surface
fig = go.Figure(data=go.Surface(z=z, x=x, y=y))

Configuration des titres
fig.update_layout(title='Graphique en surface simple',
 scene=dict(xaxis_title='X',
 yaxis_title='Y',
 zaxis_title='Z'))

Affichage du graphique
fig.show()
```

**Quelques explications :**

1. **Génération des axes : `np.linspace(-3, 3, 50)`** génère 50 points également espacés entre -3 et 3 pour les axes X et Y.
2. **Grille de coordonnées : `np.meshgrid(x, y)`** crée une grille 2D à partir des vecteurs x et y. cette methode transforme deux vecteurs (ici x et y) en deux matrices 2D. Chaque matrice représente toutes les combinaisons possibles des coordonnées x et y.
3. **Calcul des valeurs Z : `z = x**2 + y**2`** définit une surface parabolique, où chaque point Z est la somme des carrés de X et Y.
4. **Création et affichage :** Le graphique est créé et affiché avec des titres appropriés pour chaque axe.

**Carte géographique**

Une **carte géographique** permet de visualiser des données sur des emplacements géographiques. Chaque point sur la carte peut représenter une observation, avec des coordonnées géographiques (latitude et longitude). Ce type de graphique est particulièrement efficace pour illustrer des données spatiales et pour observer des tendances géographiques.

**Utilisations :**

- Visualiser la répartition des ventes par région.
- Analyser des données démographiques à l'échelle mondiale.
- Montrer des itinéraires de transport ou des données environnementales.

**Exemple :**

```
import plotly.express as px

Données pour la carte
data = dict(
 lat=[3.8480, 4.0511, 4.2634, 2.7740], # Latitude
 lon=[11.5021, 9.7085, 9.7052, 9.7085], # Longitude
 text=['Yaoundé', 'Douala', 'Bamenda', 'Garoua'] # Étiquettes
)

Création de la carte géographique
fig = px.scatter_geo(data, lat='lat', lon='lon', text='text', title='Carte géographique du Cameroun')

Affichage du graphique
fig.show()
```

## III. Introduction à Mayavi

### II.1. Présentation de la bibliothèque

**Mayavi** est une bibliothèque Python puissante utilisée pour la visualisation 3D de données scientifiques. Elle est particulièrement adaptée pour visualiser des données volumétriques, des surfaces, des champs de vecteurs et d'autres types de données complexes. Basée sur le moteur de rendu VTK (Visualization Toolkit), Mayavi offre une interface conviviale pour créer des visualisations interactives et dynamiques.

#### Caractéristiques Principales

- **Visualisation 3D** : Il permet de créer des visualisations en trois dimensions de manière efficace, ce qui est crucial pour explorer des données complexes.
- **Support pour Multiple Types de Données** : Il peut gérer divers formats de données, y compris des données scalaires, vectorielles, et des maillages.
- **Interface Utilisateur Interactive** : Il fournit des outils interactifs pour manipuler les visualisations, y compris le zoom, la rotation et le déplacement.
- **Intégration avec NumPy** : Il fonctionne de manière transparente avec des tableaux NumPy, facilitant la manipulation et la visualisation de données scientifiques.
- **Facilité d'Utilisation** : Il offre une API intuitive qui simplifie le processus de création de visualisations complexes.

#### Applications de Mayavi

- **Visualisation Scientifique** : Utilisé dans des domaines comme la physique, la chimie, et la biologie pour modéliser des phénomènes complexes.
- **Analyse de Données** : Aide à explorer et comprendre des ensembles de données volumineux et multidimensionnels.
- **Éducation** : Outil pédagogique pour illustrer des concepts mathématiques et physiques en 3D.



**Mayavi** est une bibliothèque incontournable pour quiconque souhaite visualiser des données scientifiques en 3D de manière intuitive et interactive. Que ce soit pour des analyses de données, des modélisations ou des présentations, Mayavi offre des outils puissants pour transformer des données complexes en visualisations claires et compréhensibles.

## Installation

Pour utiliser **Mayavi** dans Visual Studio Code (VS Code), il suffit d'ouvrir un terminal dans VS Code et exécutez la commande suivante :

```
pip install mayavi
```

## II.2. Plotly vs Mayavi

Voici un tableau comparatif des différences entre **Plotly** et **Mayavi** :

| Critères                      | Plotly                                                                               | Mayavi                                                                                                 |
|-------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Type de Visualisation</b>  | 2D et 3D, principalement axé sur les graphiques interactifs et les tableaux de bord. | Principalement 3D, avec un accent sur les visualisations scientifiques et volumétriques.               |
| <b>Interface</b>              | Interface web interactive, facile à intégrer dans des applications web.              | Interface de bureau, principalement axée sur des visualisations scientifiques.                         |
| <b>Complexité des Données</b> | Idéal pour les données tabulaires et les graphiques simples à complexes.             | Conçu pour des données volumineuses et complexes, comme des champs scalaires et vectoriels.            |
| <b>Interopérabilité</b>       | Fonctionne bien avec des bibliothèques comme Pandas pour des analyses de données.    | S'intègre bien avec NumPy et SciPy pour des calculs scientifiques.                                     |
| <b>Facilité d'Utilisation</b> | API intuitive, permet de créer des graphiques rapidement sans beaucoup de code.      | Peut nécessiter plus de code pour des visualisations complexes, mais offre une grande flexibilité.     |
| <b>Installation</b>           | Facile à installer via <code>pip</code> .                                            | Installation peut être plus complexe en raison des dépendances (comme VTK).                            |
| <b>Utilisations</b>           | Analyse de données, visualisations interactives, tableaux de bord.                   | Visualisation scientifique, modélisation de phénomènes physiques, exploration de données volumineuses. |
| <b>Documentation</b>          | Documentation riche avec de nombreux exemples et tutoriels.                          | Documentation disponible mais peut être moins fournie que celle de Plotly.                             |

En Conclusion,

- **Plotly** est idéal pour des visualisations interactives et des analyses de données, surtout dans un contexte web.
- **Mayavi** est plus adapté pour des visualisations scientifiques en 3D et pour travailler avec des données volumineuses et complexes.

Le choix entre les deux dépendra de vos besoins spécifiques en matière de visualisation de données.

## II.3. Cas pratique

Vous êtes un ingénieur en géologie chargé d'analyser des données topographiques d'une région montagneuse au Cameroun, comme les Monts Mandara. Vous devez visualiser les élévations de la région pour mieux comprendre la géographie locale et potentiellement planifier des projets de construction.

### Objectif :

Utiliser Mayavi pour créer une visualisation 3D des données d'élévation sur une grille de points.

### Données :

Pour simplifier, nous allons générer des données d'élévation synthétiques. Les données réelles peuvent être obtenues à partir de sources comme les cartes topographiques ou des modèles numériques de terrain.

### Code Python :

```
from mayavi import mlab
import numpy as np

Créer une grille de coordonnées
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
x, y = np.meshgrid(x, y)

Générer des données d'élévation synthétiques (une surface en forme de
montagne)
z = np.sin(np.sqrt(x**2 + y**2)) * 5 # Élévation maximale de 5 unités

Créer la visualisation 3D
mlab.surf(x, y, z, warp_scale='auto', colormap='terrain')

Configuration des titres
mlab.title('Visualisation 3D de l\'élévation des Monts Mandara')
mlab.xlabel('Longitude')
mlab.ylabel('Latitude')
mlab.zlabel('Élévation (m)')

Afficher la visualisation
mlab.show()
```

Si lors de l'exécution, vous rencontrez une erreur du style « **ImportError: Could not import backend for traitsui. Make sure you have a suitable UI toolkit like PyQt/PySide or wxPython installed** », cela indique que Mayavi ne parvient pas à trouver un **backend approprié** pour l'interface utilisateur. Cela peut se produire si les bibliothèques nécessaires comme **PyQt** ou **PySide** ou encore **wxPython** ne sont pas installées.

Un **backend** désigne la bibliothèque ou l'interface graphique qui permet à Mayavi d'afficher ses fenêtres, ses menus, et ses éléments visuels interactifs. Plus concrètement, c'est la couche qui gère l'interaction avec le système d'exploitation et la plateforme graphique utilisée pour afficher l'interface graphique. Pour Python, cela inclut souvent des bibliothèques comme **PyQt** ou **PySide** ou encore **wxPython** qui fournissent des composants visuels pour construire des fenêtres, boutons, menus, etc.

Ainsi, vous devez installer l'une des bibliothèques suivantes :

```
pip install PyQt5
pip install PySide2
pip install wxPython
```